

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\agent-a1424e200dde17a25.jsonl`
- SHA-256: `602346c7ea020c90bacf97cab7f02f8c95c7221700a18b608dedbad16fa95faa`
- Source modified: `2026-06-14T11:05:54+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `subagents`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-14T10:49:54.200Z)

FOUNDATIONAL FIX (owner: "fix the foundational issues before any other task"): the eval harness promotes cues on a windowing PRODUCTION DOESN'T SERVE (eval?serve). The Gate-A +0.074 was measured on the harness's recall-oriented PROPOSAL windowing (vt=0.005, merge_gap=1.0, min_win=0.5), but the SERVED pipeline uses COARSE windowing (vt=0.02, merge_gap=2.0, min_win=2.0), so the lift didn't transfer (it regressed served). Fix the foundation so a cue CANNOT be promoted on a windowing production doesn't serve. Repo cwd C:/Users/avidu/Projects/badminton-highlight-indexer. ISOLATED worktree: `git -C ... fetch origin` then `git -C ... worktree add C:/Users/avidu/Projects/bhi-evalserve -b feat/eval-serve-windowing-alignment origin/master`.

Read: backend/eval/distill_local.py (build_candidates + the windowing params vt / merge_gap / min_win), backend/eval/experiment.py + fusion_compare.py + fusion_golden.py (how offline candidates/windowing are built for eval), backend/pipeline/segmenters/fusion_hybrid.py + how `process\_video`/`load\_config` sets the SERVED windowing (vt=0.02, gap=2.0, min=2.0), scratch/served_gateA_*.py + output/gateA_served_verify/ (the served check that found this), scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md.

DESIGN + BUILD the minimal foundational fix (be surgical; do NOT break existing eval):

1. Parameterize the eval-candidate windowing ? a named **SERVED preset** (vt=0.02/gap=2.0/min=2.0) vs the **PROPOSAL preset** (vt=0.005/gap=1.0/min=0.5), single-sourced so eval and serve reference the SAME served windowing constants.
 2. Encode the RULE: **a cue is promotable only if it wins (or at least does NOT regress) at the SERVED windowing** ? a clear helper / check the ablation + compare promotion path uses (compose with eval_stats + per_user_gate).
 3. Re-run the Gate-A ablation at BOTH windowings and record the contrast (proposal +0.074 vs served ~?0.008) as the worked example proving the fix.
- COMPREHENSIVE tests (the served-vs-proposal windowing produces different candidates; the promotion check rejects an eval-only win). ruff+mypy+pytest green. PR; MERGE on green IF additive/safe; if it materially changes existing eval semantics, leave the PR OPEN for owner review with a crisp summary. Branch-safe, explicit paths, `Co-Authored-By: Claude Opus 4.8 (1M context) <noreply@anthropic.com>`. Return: the fix design, PR number + merge/open status, the Gate-A served-vs-proposal numbers, tests, and whether you left it open for review.

2. assistant (2026-06-14T10:49:56.566Z)

I'll start by following the repo-freshness convention and setting up the isolated worktree, then ramp up on the relevant code.

3. user (2026-06-14T10:49:59.071Z)

e75ec35 feat(eval): LOSO/LOGO ablation runner ? per-signal marginal ?F1 (Phase 0) (#147)
570cf2b feat(fusion): flip Gate-A (held-shuttle guard) to the served default ? config-reversible (#146)
7c6498f docs(quality): personalization Phase-0 checkpoint ? eval/safety rails + locked structural-guard decision (#145)

4. assistant (2026-06-14T10:50:01.277Z)

Let me set up the isolated worktree and read the key files in parallel.

5. user (2026-06-14T10:50:03.289Z)

Preparing worktree (new branch 'feat/eval-serve-windowing-alignment')
branch 'feat/eval-serve-windowing-alignment' set up to track 'origin/master'.
HEAD is now at e75ec35 feat(eval): LOSO/LOGO ablation runner ? per-signal marginal ?F1 (Phase 0) (#147)

6. assistant (2026-06-14T10:50:06.816Z)

Note: the foundational fix prompt references PRs #146 (Gate-A flipped to served default) and #147 (LOSO/LOGO ablation runner) that have landed since the prompt was written. Let me read the key files to understand the current state.

7. user (2026-06-14T10:50:07.015Z)

```
1      """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3      Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4      further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5      using a few resolution/fps-INVARIANT local features, so the model transfers across
6      videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8      Flow (all local at runtime):
9          WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10             filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11             active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12             -> rally windows.
13
14      Training labels come from Gemini full-context labels via the SAME IoU matching the
15      evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16      other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17      compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18      rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20      Run:
21          python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22      """
23      from __future__ import annotations
24
25      import json
26      import argparse
27      from typing import Dict, List, Optional, Tuple
28
29
30      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31      from backend.pipeline.detectors import rally_gate
32      from backend.eval.calibrate_wasb import load_golden_gts
33      from backend.eval.training import label_candidates
34      from backend.eval.classifier import LogisticRegression
35      from backend.eval.metrics import score
36      from backend.eval.gemini_refine import merge_overlaps
37
38      Interval = Tuple[float, float]
39
40      FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
41      # Recall-oriented proposal: deliberately permissive so real rallies are among the
42      # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
43      PROPOSAL = (0.005, 1.0, 0.5)
```

```

44     BASELINE_LOCAL = (0.02, 2.0, 2.0) # today's production-ish windowing, no learning
45
46
47     def build_candidates(trajecory_csv: str, fps: float, frame_width: float,
48                          proposal: Tuple[float, float, float] = PROPOSAL) ->
49     Tuple[List[Interval], List[List[float]]]:
50         """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
51         rows)."""
52         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
53         vt, mg, mwd = proposal
54         wins = TrackNetRunner.trajectory_to_action_windows(
55             points, fps=fps, frame_width=frame_width,
56             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
57         intervals = [(w["start"], w["end"]) for w in wins]
58         gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
59         crossings
60         X: List[List[float]] = []
61         for w, g in zip(wins, gated):
62             dur = max(1e-6, w["end"] - w["start"])
63             win_frames = max(1.0, dur * fps)
64             X.append([
65                 dur, # rally length (sec) ?
66                 float(w["peak_velocity"]), # already normalized by
67                 float(w["mean_velocity"]),
68                 float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ?
69                 float(g.crossings), # net crossings (count) ?
70                 the rally signal
71             ])
72         return intervals, X
73
74     def baseline_windows(trajecory_csv: str, fps: float, frame_width: float) ->
75     List[Interval]:
76         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
77         vt, mg, mwd = BASELINE_LOCAL
78         wins = TrackNetRunner.trajectory_to_action_windows(
79             points, fps=fps, frame_width=frame_width,
80             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
81         return [(w["start"], w["end"]) for w in wins]
82
83     def load_video(spec: Dict, iou: float = 0.5) -> Dict:
84         fps, fw = float(spec["fps"]), float(spec["frame_width"])
85         intervals, X = build_candidates(spec["trajectory"], fps, fw)
86         gts = load_golden_gts(spec["labels"])
87         y = label_candidates(intervals, gts, iou)
88         return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
89         X,
90                 "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
91                 fw)}
92
93     def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
94     List[Interval],
95                       threshold: float = 0.5) -> List[Interval]:
96         if not intervals:
97             return []
98         proba = model.predict_proba(X)
99         pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
100         return merge_overlaps(pos, gap_tol=1.0)

```

```

98     def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
99             model_out: Optional[str] = None) -> dict:
100         videos = [load_video(s, iou) for s in specs]
101         print(f"=== Distilled local rally classifier vs Gemini silver-GT ")
102         f"({len(videos)} videos, IoU={iou}, prob>={threshold}) ===")
103         for v in videos:
104             ceil = score(v["intervals"], v["gts"], iou).recall
105             print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
106                   f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")
107
108         result: dict = {"videos": [v["name"] for v in videos]}
109         if len(videos) >= 2:
110             print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
111 out) ---")
112             clf, base = [], []
113             for i, held in enumerate(videos):
114                 others = [v for j, v in enumerate(videos) if j != i]
115                 X = [row for v in others for row in v["X"]]
116                 y = [t for v in others for t in v["y"]]
117                 model = LogisticRegression().fit(X, y, FEATURE_NAMES)
118                 preds = predict_rallies(model, held["X"], held["intervals"], threshold)
119                 sc = score(preds, held["gts"], iou)
120                 bsc = score(held["baseline"], held["gts"], iou)
121                 clf.append(sc)
122                 base.append(bsc)
123                 print(f" held-out {held['name']}: LEARNED P={sc.precision:.3f}
124 R={sc.recall:.3f} F1={sc.f1:.3f}"
125                       f" | baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
126 F1={bsc.f1:.3f}")
127             def mean(xs, a):
128                 return sum(getattr(s, a) for s in xs) / len(xs)
129             print(f" MEAN held-out: LEARNED P={mean(clf, 'precision'):.3f}
130 R={mean(clf, 'recall'):.3f} "
131                   f"F1={mean(clf, 'f1'):.3f} | baseline P={mean(base, 'precision'):.3f} "
132                   f"F1={mean(base, 'f1'):.3f} R={mean(base, 'recall'):.3f} "
133                   f"F1={mean(base, 'f1'):.3f}")
134             result["mean_learned_f1"] = mean(clf, "f1")
135             result["mean_baseline_f1"] = mean(base, "f1")
136
137             # Final model on ALL videos (for production / running on a fresh held-out video).
138             Xall = [row for v in videos for row in v["X"]]
139             yall = [t for v in videos for t in v["y"]]
140             final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
141             assert final.w is not None # set by fit()
142             weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
143             print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
144             if model_out:
145                 final.save(model_out)
146                 print(f"final model (all videos) -> {model_out}")
147                 result["model_out"] = model_out
148             return result
149
150         def main() -> None:
151             ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
152 classifier.")
153             ap.add_argument("--manifest", required=True,
154                             help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
155 (labels = Gemini CSV)")
156             ap.add_argument("--iou", type=float, default=0.5)
157             ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
158 cutoff")
159             ap.add_argument("--model-out", default="output/local_rally_model.json")
160             args = ap.parse_args()
161             with open(args.manifest, encoding="utf-8") as f:
162                 specs = json.load(f)

```

```

156         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
157
158
159     if __name__ == "__main__":
160         main()
161

```

8. assistant (2026-06-14T10:50:10.338Z)

Now I see the core: `PROPOSAL = (0.005, 1.0, 0.5)` and `BASELINE\_LOCAL = (0.02, 2.0, 2.0)`. Let me read the eval harness files and the fusion segmenter in parallel.

9. user (2026-06-14T10:50:11.208Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin variant/experiment layer that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. No new scoring math lives
7     here; everything defers to:
8
9     - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11    - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12    - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13    - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14    - `backend.eval.calibration`   ? `pct_improvement`
15
16    The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17    (per-frame signals ? run once on the GPU/WSL box, persisted into
18    `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19    signals, where are the rallies). A Variant is a declarative re-scoring recipe;
20    given persisted candidate features it produces List[Interval] deterministically,
21    with no GPU and zero production risk. So most experiments are pure offline
22    re-scoring of stored data and never touch the served pipeline.
23
24    Three modes (`EXPERIMENT_HARNESS.md` §5):
25    - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26      aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27      held-out loop, but variant-parameterised.
28    - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29      variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30      human spot-checks (and what two highlight reels would show side by side).
31    - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32      (exit 0/1/3); rollback = flip the variant/config, never `git revert`.
33
34    Pure + offline + unit-tested. The only DB-touching helper is
35    `load_video_candidates`, which reads persisted candidates via
36    `Database.query_candidate_segments`.
37    """
38    from __future__ import annotations
39
40    import json
41    import logging
42    import os
43    from dataclasses import dataclass, field
44    from datetime import datetime, timezone
45    from hashlib import sha1
46    from typing import Any, Callable, Dict, List, Optional, Sequence
47
48    from backend.eval.calibration import pct_improvement
49    from backend.eval.classifier import LogisticRegression
50    from backend.eval.gemini_refine import merge_overlaps
51    from backend.eval.metrics import Interval, match_intervals, score

```

```

52     from backend.eval.regression import gt_hash
53     from backend.eval.training import label_candidates
54     from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
55     from backend.eval import segmentation_metrics as _segm  # C4: F1@k + segment-count
ratio
56     from backend.eval.score_calibration import fit_isotonic, fit_platt  # C2: calibrate cue
scores
57
58     logger = logging.getLogger(__name__)
59
60     # Where a comparison run appends one reproducible record. Tracked location (NOT under
61     # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
62     # regression.DEFAULT_BASELINE_PATH.
63     DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
64     _METRICS = ("precision", "recall", "f1", "mean_iou")
65
66
67     class ExperimentError(ValueError):
68         """A variant cannot be evaluated as configured (e.g. a learned variant asked to
69         score an unlabeled video with no pinned model, or a gate referencing an absent
70         feature)."""
71
72
73     # ----- Variant
74
75
76     @dataclass
77     class Variant:
78         """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
79
80         A variant turns one video's persisted candidate windows + features into rally
81         intervals. Every knob defaults to the control behaviour (no gate, no learned
82         filter), so a variant that merely carries a new, disabled cue is byte-identical
83         to control ? the core no-regression guarantee.
84
85         Fields:
86         - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87           for the leaderboard and for selecting the served path (the existing
88           ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
89           New cues are recorded here default-OFF.
90         - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
91           ``net_crossings`` feature is  $\geq$  this. 0 = off. This is the eval-only gate from
92           ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93           windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94         - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
95           whose persisted ``players_in_court`` is  $\geq$  this. 0 = off (no-op until the person
96           cue persists that feature).
97         - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
98           ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
99           required for ``compare_unlabeled`` on a learned variant.
100        - ``feature_names`` ? the persisted features the learned filter consumes. When set
101          WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
102          (honest) ? the recipe, not a pinned artifact.
103        - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104          overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
105        """
106
107        name: str
108        segmenter: str = "wasb_hybrid"
109        config_overrides: Dict[str, Any] = field(default_factory=dict)
110        cue_flags: Dict[str, bool] = field(default_factory=dict)
111        min_crossings: int = 0
112        min_players: int = 0
113        classifier_model: Optional[str] = None

```

```

114     feature_names: Optional[List[str]] = None
115     threshold: float = 0.5
116     merge_gap: float = 1.0
117     # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118     # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119     # failure where a fixed 0.5 zeroed out a small-candidate video.
120     tune_threshold: bool = False          # pick the F1-best threshold on the train fold
121     calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate
proba first
122
123     def is_learned(self) -> bool:
124         """True if this variant applies a learned classifier (pinned model or
recipe)."""
125         return self.classifier_model is not None or bool(self.feature_names)
126
127     def to_dict(self) -> dict:
128         return {
129             "name": self.name,
130             "segmenter": self.segmenter,
131             "config_overrides": self.config_overrides,
132             "cue_flags": self.cue_flags,
133             "min_crossings": self.min_crossings,
134             "min_players": self.min_players,
135             "classifier_model": self.classifier_model,
136             "feature_names": self.feature_names,
137             "threshold": self.threshold,
138             "merge_gap": self.merge_gap,
139             "tune_threshold": self.tune_threshold,
140             "calibrate": self.calibrate,
141         }
142
143     @classmethod
144     def from_dict(cls, d: dict) -> "Variant":
145         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146         return cls(**{k: v for k, v in d.items() if k in known})
147
148     def spec_hash(self) -> str:
149         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150         and makes a result reproducible. The pinned **control** variant always hashes
the
151         same, so a regression is always traceable to a recipe change, never to drift."""
152         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
153         return sha1(blob.encode()).hexdigest()[:12]
154
155
156     #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157     #: filter. Always the comparison reference; "rollback" = make this the served variant.
158     CONTROL = Variant(name="control")
159
160
161     # ----- persisted candidates
162
163
164     @dataclass
165     class VideoCandidates:
166         """One video's persisted detection, ready for offline re-scoring.
167
168         ``intervals`` are the candidate windows; ``features`` is column-major
169         (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
170         ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171         with ``load_video_candidates``, or directly in tests.
172         """
173

```

```

174     name: str
175     intervals: List[Interval]
176     features: Dict[str, List[float]] = field(default_factory=dict)
177     gts: Optional[List[Interval]] = None
178
179
180 def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181     """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182     `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
183     col = vc.features.get(name)
184     n = len(vc.intervals)
185     if col is None:
186         logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                        name, vc.name)
188         return [0.0] * n
189     if len(col) != n:
190         raise ExperimentError(
191             f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192     return [float(x) for x in col]
193
194
195 def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
196     cols = [_feature_column(vc, name) for name in feature_names]
197     return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
198
199
200 def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201     """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
202     players)."""
203     n = len(vc.intervals)
204     mask = [True] * n
205     if variant.min_crossings > 0:
206         col = _feature_column(vc, "net_crossings")
207         mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
208     if variant.min_players > 0:
209         col = _feature_column(vc, "players_in_court")
210         mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
211     return mask
212
213 def predict(variant: Variant, vc: VideoCandidates,
214            model: Optional[LogisticRegression] = None,
215            threshold: Optional[float] = None,
216            calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
217     """Variant prediction: gate by persisted features, optionally filter by a learned
218     model, then merge. Pure and deterministic.
219
220     ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
221     fold; if omitted and the variant pins ``classifier_model``, that frozen model is
222     loaded. A learned variant with neither a supplied nor a pinned model raises ? there
223     is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
224     fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
225     them.
226
227     """
228     mask = _gate_mask(variant, vc)
229     if variant.feature_names:
230         if model is None:
231             if variant.classifier_model is None:
232                 raise ExperimentError(
233                     f"variant {variant.name!r} is learned but has no model to predict "
234                     f"with (pass a fitted model or set classifier_model)")
235             model = LogisticRegression.load(variant.classifier_model)
236         proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
237 variant.feature_names))]

```

```

235         if calibrator is not None:
236             proba = [calibrator(p) for p in proba]
237             thr = variant.threshold if threshold is None else threshold
238             mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
239             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
240             return merge_overlaps(kept, gap_tol=variant.merge_gap)
241
242
243     def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
244                           train_videos: Sequence[VideoCandidates], iou: float
245                           ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
246         """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
247         (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
248         (use the variant default). Honest: never looks at the held-out video.
249         """
250         calibrator: Optional[Callable[[float], float]] = None
251         if variant.calibrate:
252             raw: List[float] = []
253             y: List[int] = []
254             for vc in train_videos:
255                 raw.extend(float(p) for p in
256                           model.predict_proba(_feature_matrix(vc, variant.feature_names or
257                           [])))
258                 y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
259             if variant.calibrate == "platt":
260                 calibrator = fit_platt(raw, y)
261             elif variant.calibrate == "isotonic":
262                 calibrator = fit_isotonic(raw, y)
263             else:
264                 raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
265                 'platt'/'isotonic')")
266             threshold: Optional[float] = None
267             if variant.tune_threshold:
268                 best_t, best_f1 = variant.threshold, -1.0
269                 for k in range(1, 20):
270                     # grid 0.05..0.95 on the train
271                     t = round(0.05 * k, 2)
272                     f1s = [score(predict(variant, vc, model=model, threshold=t,
273                     calibrator=calibrator),
274                             vc.gts or [], iou).f1 for vc in train_videos]
275                     mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
276                     if mean_f1 > best_f1:
277                         best_f1, best_t = mean_f1, t
278                     threshold = best_t
279             return calibrator, threshold
280
281     # ----- variant scoring
282
283     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
284         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
285         the
286         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
287         X: List[List[float]] = []
288         y: List[int] = []
289         for vc in videos:
290             if vc.gts is None:
291                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
292             X.extend(_feature_matrix(vc, variant.feature_names or []))
293             y.extend(label_candidates(vc.intervals, vc.gts, iou))
294         if not X:
295             raise ExperimentError("cannot train a learned variant on zero candidates")

```

```

293         return LogisticRegression().fit(X, y, variant.feature_names)
294
295
296     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
297                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
298         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
299         mean aggregate.
300
301         Prediction policy:
302         - **static variant** (no learned filter): predict each video directly ? no
training,
303         so no leakage; per-video scoring is already honest.
304         - **learned, pinned model** (`classifier_model` set): score every video with the
305         SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
306         report "how the shipped artifact does here", not for the promotion decision.
307         - **learned recipe** (`feature_names`, no pinned model) + `honest=True`: LOVO
?
308         train on the OTHER videos, predict the held-out one. The number to trust.
309         - learned recipe + `honest=False`: train on all, predict each (overfit; sanity
only).
310         """
311         for vc in videos:
312             if vc.gts is None:
313                 raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
314
315         per_video: List[Dict[str, Any]] = []
316         predictions: Dict[str, List[Interval]] = {}
317
318         recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
319         pinned_model = (LogisticRegression.load(variant.classifier_model)
320                         if variant.classifier_model is not None else None)
321         all_model = None
322         if recipe_lovo and not honest:
323             all_model = _train(variant, videos, iou)
324
325         for i, vc in enumerate(videos):
326             cal: Optional[Callable[[float], float]] = None
327             thr: Optional[float] = None
328             if recipe_lovo and honest:
329                 others = [v for j, v in enumerate(videos) if j != i]
330                 if not others:
331                     raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
332                 model: Optional[LogisticRegression] = _train(variant, others, iou)
333                 if variant.tune_threshold or variant.calibrate: # operating point from
the train fold
334                     assert model is not None # _train never returns
None
335                     cal, thr = _calibrate_and_tune(variant, model, others, iou)
336             elif recipe_lovo: # honest=False ? one model over all
videos
337                 model = all_model
338             else: # static variant or pinned model
339                 model = pinned_model
340             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341             assert vc.gts is not None # guarded at function top
342             sc = score(preds, vc.gts, iou)
343             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344             predictions[vc.name] = preds
345
346         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)}

```

```

347         for m in _METRICS}
348     return {
349         "variant": variant.name,
350         "spec_hash": variant.spec_hash(),
351         "is_learned": variant.is_learned(),
352         "honest_lovo": recipe_lovo and honest,
353         "per_video": per_video,
354         "aggregate": aggregate,
355         "predictions": predictions,
356     }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
361         mean)."""
362         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
363         _METRICS}
364
365     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
366     Dict[str, Any]:
367         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
368         hides)."""
369         gts_by_name = {v.name: (v.gts or []) for v in videos}
370         overlaps = _segm.DEFAULT_OVERLAPS
371         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
372         ratios: List[float] = []
373         for name, preds in eval_v.get("predictions", {}).items():
374             gts = gts_by_name.get(name, [])
375             got = _segm.f1_at_overlaps(preds, gts, overlaps)
376             for ov in overlaps:
377                 f1k[ov].append(got[ov])
378             ratios.append(_segm.segment_count_ratio(preds, gts))
379
380         def _mean(xs: List[float]) -> float:
381             return sum(xs) / len(xs) if xs else 0.0
382         out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
383         f1k.items()}
384         out["segment_count_ratio"] = _mean(ratios)
385         return out
386
387
388     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
389                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
390         """C1 + C4 honest reporting layered over a compare() pair (additive,
391         EXPERIMENT_HARNESS §6).
392
393         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
394         order),
395         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
396         when
397         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
398         """
399         a_f1 = [p["f1"] for p in eval_a["per_video"]]
400         b_f1 = [p["f1"] for p in eval_b["per_video"]]
401         return {
402             "variant_a_ci": _ci_block(eval_a),
403             "variant_b_ci": _ci_block(eval_b),
404             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
405             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
406                             "variant_b": _segmentation_block(videos, eval_b)},
407         }
408
409
410     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,

```

```

404         iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405     """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
406     golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407
408     The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
409     learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410     `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
411
412     ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413     paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414     existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415     shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416     bare LOVO mean at n?6 is not trustworthy on its own.
417     """
418     if len(videos) < 1:
419         raise ExperimentError("compare needs at least one labeled video")
420     eval_a = evaluate_variant(videos, variant_a, iou, honest)
421     eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423     delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424     delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425
426     by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427     per_video_delta = [
428         {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429         for p in eval_b["per_video"]
430     ]
431
432     # One fingerprint over the whole label set ? detects "the deltas were measured
against
433     # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434     all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436     result: Dict[str, Any] = {
437         "iou": iou,
438         "label_set_hash": gt_hash(all_gts),
439         "num_videos": len(videos),
440         "variant_a": eval_a,
441         "variant_b": eval_b,
442         "delta": delta,
443         "delta_pct": delta_pct,
444         "per_video_delta": per_video_delta,
445     }
446     if honest_stats:
447         result["honest"] = honest_summary(videos, eval_a, eval_b)
448     return result
449
450
451     # ----- SxS on unlabeled video
452
453
454     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455         agree_iou: float = 0.5) -> Dict[str, Any]:
456         """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).

```

```

457
458 With no labels there is no lift to measure ? instead this surfaces where the two
459 variants **agree vs diverge**, which is exactly what a human reviews (and what two
460 highlight reels would show side by side):
461
462 - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
463 - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
464 - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPs:
465   the human / a later golden label adjudicates).
466
467 Both variants must be predictable without labels: a learned variant therefore needs
468 a
469 pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
470 ``metrics.match_intervals`` ? no new matching logic.
471
472 preds_a = predict(variant_a, video)
473 preds_b = predict(variant_b, video)
474 mr = match_intervals(preds_a, preds_b, agree_iou)
475
476 agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
477           for m in mr.matches]
478 agree_iou = [m.iou for m in mr.matches]
479 a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
480 b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
481
482 def _dur(ivs: List[Interval]) -> float:
483     return sum(e - s for s, e in ivs)
484
485 return {
486     "video": video.name,
487     "agree_iou": agree_iou,
488     "variant_a": variant_a.name,
489     "variant_b": variant_b.name,
490     "num_a": len(preds_a),
491     "num_b": len(preds_b),
492     "num_agreed": len(agreed),
493     "mean_agree_iou": (sum(agree_iou) / len(agree_iou)) if agree_iou else 0.0,
494     "duration_a": _dur(preds_a),
495     "duration_b": _dur(preds_b),
496     "agreed": agreed,
497     "a_only": a_only,
498     "b_only": b_only,
499 }
500
501 # ----- leaderboard / DB
502
503
504 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
505                      timestamp: Optional[str] = None) -> Dict[str, Any]:
506     """Append a compact, reproducible record of a `compare()` result to the JSON
507     leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
508     `regression_baseline.json`; deliberately the size of a baseline file, not a service
509     (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
510     """
511     row: Dict[str, Any] = {
512         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
513         "iou": result.get("iou"),
514         "label_set_hash": result.get("label_set_hash"),
515         "num_videos": result.get("num_videos"),
516         "variant_a": {"name": result["variant_a"]["variant"],
517                      "spec_hash": result["variant_a"]["spec_hash"],
518                      "aggregate": result["variant_a"]["aggregate"]},
519         "variant_b": {"name": result["variant_b"]["variant"],
520                      "spec_hash": result["variant_b"]["spec_hash"],

```

```

521         "aggregate": result["variant_b"]["aggregate"]},
522         "delta": result.get("delta"),
523     }
524     # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
the
525     # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
526     honest = result.get("honest") or {}
527     if honest.get("paired_delta_f1"):
528         row["honest_delta_f1"] = honest["paired_delta_f1"]
529     entries: List[Dict[str, Any]] = []
530     if os.path.isfile(path):
531         with open(path, encoding="utf-8") as f:
532             entries = json.load(f)
533     entries.append(row)
534     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
535     with open(path, "w", encoding="utf-8") as f:
536         json.dump(entries, f, indent=2)
537     logger.info("recorded experiment %s vs %s (?f1=%3f) -> %s",
538               row["variant_a"]["name"], row["variant_b"]["name"],
539               (result.get("delta") or {}).get("f1", 0.0), path)
540     return row
541
542
543     def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                             gts: Optional[List[Interval]] = None,
545                             feature_names: Sequence[str] = ()) -> VideoCandidates:
546         """Bridge persisted detection ? `VideoCandidates` via
`Database.query_candidate_segments`.
547
548         ``feature_names`` selects which persisted features to lift out of each row's
`stats`
549         JSON (``"duration"`` is derived from the window bounds). This is the only DB-
touching
550         function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
551         but works against any `Database` (including an in-memory one in tests).
552         """
553         rows = db.query_candidate_segments(video_id, detector=detector)
554         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
555         features: Dict[str, List[float]] = {}
556         for name in feature_names:
557             if name == "duration":
558                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
559             else:
560                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561         return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562

```

10. assistant (2026-06-14T10:50:24.858Z)

Now let me read the fusion segmenter, fusion_compare, fusion_golden, and the served check files in parallel.

11. user (2026-06-14T10:50:25.622Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:

```

```

7
8     - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9       GPU-free, computed from the trajectory we already have); and, only when the person cue
10      is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11      (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12      fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13      torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14      model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15      via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17      (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18      dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19      **Persisted candidates remain the full superset** regardless, so the offline
experiment
20      harness can re-score any variant.
21
22      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26      experiment harness (EXPERIMENT_HARNESS.md), never silently.
27      """
28      from typing import Any, Dict, List, Optional, Tuple
29
30      from backend.pipeline.segmenters.base import segmenter_registry
31      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32      from backend.pipeline.detectors.base import DetectorRunner
33      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35      from backend.pipeline.detectors import rally_gate
36      from backend.pipeline.detectors import fusion_features as ff
37
38
39      class FusionSegmenter(TrajectoryHybridSegmenter):
40          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42          family = "fusion"
43          display_label = "Fusion"
44
45          def __init__(self, db: Any, config: Any):
46              super().__init__(db, config)
47              # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48              self._person: Optional[Any] = None
49              # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50              # over the whole trajectory once per candidate window (it depends only on
`points`).
51              self._axis_cache_key: Optional[int] = None
52              self._axis_cache: Optional[tuple] = None
53
54          @property
55          def name(self) -> str:
56              return "fusion_hybrid"
57

```

```

58     @property
59     def description(self) -> str:
60         return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61             "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62             "AI provider sets semantic boundaries.")
63
64     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65         substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66         if substrate == "tracknet":
67             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68             return TrackNetRunner(cfg), {
69                 "weights_path": cfg.weights_path,
70                 "queue_length": cfg.queue_length,
71                 "fusion_substrate": "tracknet",
72             }
73         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74         return WasbRunner(wcfg, {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"})
75
76     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
77                                frame_width: float, video_path: str,
78                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
79         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
80         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
81         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
82         # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83         gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
84                                             min_crossings=0,
estimate=self._axis_estimate(points))
85         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87         fcfg = idx_cfg.get("fusion", {})
88         if fcfg.get("cue_flags", {}).get("person", False):
89             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90         return stats
91
92     def _axis_estimate(self, points: List[Any]) -> tuple:
93         """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
94         computed once per video, not once per candidate window (it depends only on
points)."""
95         key = id(points)
96         if self._axis_cache_key != key or self._axis_cache is None:
97             self._axis_cache_key = key
98             self._axis_cache = rally_gate.estimate_play_axis(points)
99         est = self._axis_cache
100         assert est is not None
101         return est
102
103     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
104                          frame_width: float, video_path: str,
105                          fcfg: Dict[str, Any]) -> Dict[str, float]:
106         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
107         frame range and compute the scale/translation-invariant person features. Lazily
builds
108         ONE PersonDetector. ? real-video verification is owner-side; this wiring is

```

```

mock-tested."""
109         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
110         from backend.pipeline.detectors.person_detector import PersonDetector
111         if self._person is None:
112             self._person = PersonDetector(self.config)
113
114         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
115         # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
116         # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
117         start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
118         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
119         fw = det["frame_width"] or frame_width
120         fh = det["frame_height"]
121         # If we can't trust the frame dims, every normalised feature would silently
collapse
122         # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
123         if fw <= 0 or fh <= 0:
124             return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
125
126         poly = fcfg.get("court_polygon")
127         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
128         net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
129         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
130         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
131                                         court_polygon=polygon, net=net)
132
133     def _select_for_handoff(self, windows: List[Dict[str, Any]],
134                           window_stats: List[Dict[str, Any]],
135                           idx_cfg: Dict[str, Any]) -> List[int]:
136         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
137         (0) keep every window ? identical to the substrate. Persisted candidates are
138         untouched (full superset)."""
139         fcfg = idx_cfg.get("fusion", {})
140         min_crossings = int(fcfg.get("min_crossings", 0))
141         min_players = int(fcfg.get("min_players", 0))
142         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
143
144         keep: List[int] = []
145         for i, st in enumerate(window_stats):
146             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
147                 continue
148             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
149                 continue
150             keep.append(i)
151         return keep
152
153
154     segmenter_registry.register(FusionSegmenter)
155

```

12. assistant (2026-06-14T10:50:30.116Z)

Now let me look at the trajectory_hybrid base for the served windowing, and the served check scripts.

13. user (2026-06-14T10:50:30.900Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a

```

```

4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7         healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8         -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20    import os
21    import time
22    import logging
23    import concurrent.futures
24    from typing import Any, Dict, List, Optional, Tuple
25
26    import cv2
27
28    from backend.pipeline.segmenters.base import BasePlaySegmenter
29    from backend.utils.video import get_video_duration, extract_video_chunk
30    from backend.sports import sport_registry
31    from backend.providers import provider_registry
32    from backend.pipeline.detectors.base import DetectorRunner
33
34    logger = logging.getLogger(__name__)
35
36
37    def _fmt_ts(seconds: float) -> str:
38        seconds = max(0, int(seconds))
39        h, rem = divmod(seconds, 3600)
40        m, s = divmod(rem, 60)
41        return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44    class TrajectoryHybridSegmenter(BasePlaySegmenter):
45        """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46        boundaries."""
47
48        #: config section under `indexing` (velocity_threshold lives there), the
49        #: output subdir for trajectories, and the metadata detector-tag prefix.
50        family: str = ""
51        #: human-readable label used in log lines.
52        display_label: str = ""
53
54        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55        Any]]:
56            """Return (runner, detector-specific detection_params extras)."""
57            raise NotImplementedError
58
59        @staticmethod
60        def _probe_fps_width(video_path: str) -> tuple:
61            """Return (fps, frame_width) for a video, with sensible fallbacks."""
62            cap = cv2.VideoCapture(video_path)
63            fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
64            width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
65            cap.release()
66            return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
67
68        @staticmethod

```

```

67     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
68         """Provider-reported exchange count, else a duration-based estimate.
69
70         One canonical implementation ? the twins had drifted (wasb relied on
71         ``int(None)`` raising into the fallback; same result, by accident).
72         """
73         shot_count = segment.get("shuttle_exchange_count")
74         if shot_count is None:
75             return max(3, int(duration / 0.8))
76         try:
77             return int(shot_count)
78         except (ValueError, TypeError):
79             return max(3, int(duration / 0.8))
80
81     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
82     -----
83     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
84                                frame_width: float, video_path: str,
85                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
86         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
87         BASE
88         returns exactly the 4 motion keys, so the trajectory twins persist byte-
89         identical
90         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
91         features). ``points`` are the whole-video trajectory points
92         (TrajectoryPoint)."""
93         return {
94             "peak_velocity": cw["peak_velocity"],
95             "mean_velocity": cw["mean_velocity"],
96             "active_frame_count": cw["active_frame_count"],
97             "track_id_count": cw["track_id_count"],
98         }
99
100     def _select_for_handoff(self, windows: List[Dict[str, Any]],
101                            window_stats: List[Dict[str, Any]],
102                            idx_cfg: Dict[str, Any]) -> List[int]:
103         """Indices of the candidate windows that proceed to the AI handoff (and thus may
104         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
105         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
106         Persisted candidates are NOT affected ? they remain the full superset for
107         offline
108         re-scoring."""
109         return list(range(len(windows)))
110
111     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
112                      player_pool: Optional[List[str]] = None,
113                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
114         # override-ignore: the ABC declares the Result contract (Success|Failure)
115         # but every live segmenter still returns a bare list ? the documented
116         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
117         label = self.display_label
118         # --- Sport profile + AI provider wiring (identical across segmenters) ---
119         sport_name = self.config.get("sport", "badminton")
120         try:
121             sport_profile = sport_registry.get(sport_name)
122         except KeyError as e:
123             logger.error(str(e))
124             return []
125
126         idx_cfg = self.config.get("indexing", {})
127         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
128 "gemini"))
129         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
130 "gemini-2.5-flash"))

```

```

125     api_key_env_var = f"{provider_name.upper()}_API_KEY"
126     api_key = os.environ.get(api_key_env_var)
127     if not api_key:
128         logger.error(
129             f"{api_key_env_var} environment variable is not set! "
130             f"To run the {provider_name} handoff, set your API key. "
131             f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
132         )
133     return []
134     try:
135         provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
136     except KeyError as e:
137         logger.error(str(e))
138         return []
139
140     video_duration = get_video_duration(video_path)
141     if video_duration <= 0:
142         logger.error("Invalid video duration.")
143         return []
144     fps, frame_width = self._probe_fps_width(video_path)
145
146     # --- Runner config + windowing thresholds ---
147     runner, runner_params = self._build_runner(idx_cfg)
148
149     velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
150     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
151     min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
152     handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
153     ai_max_workers = idx_cfg.get("ai_max_workers", 5)
154     log_candidates = idx_cfg.get("log_yolo_candidates", True)
155     store_candidates = idx_cfg.get("store_yolo_candidates", True)
156
157     detection_params = {
158         "detector": self.name,
159         "velocity_thresh": velocity_thresh,
160         "merge_gap": merge_gap,
161         "min_action_window_duration": min_window_duration,
162         **runner_params,
163     }
164
165     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
166     ok, msg = runner.healthcheck()
167     if not ok:
168         logger.error("%s WSL environment not ready: %s", label, msg)
169         return []
170     logger.info("%s WSL env OK: %s", label, msg)
171
172     # --- Phase 2: trajectory -> candidate rally windows ---
173     # Trajectory detectors process the whole video in one pass (they carry
174     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
175     t_start = time.time()
176     out_dir = os.path.join("output", self.family)
177     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
178     if not csv_path:
179         logger.error("%s produced no trajectory; aborting.", label)
180         return []
181
182     points = runner.parse_trajectory_csv(csv_path)
183     logger.info("Parsed %d trajectory points (%d visible).",
184                 len(points), sum(1 for p in points if p.visible))
185
186     all_candidate_windows = runner.trajectory_to_action_windows(
187         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
188         velocity_thresh=velocity_thresh, merge_gap=merge_gap,

```

```

189         min_window_duration=min_window_duration, chunk_index=0,
190     )
191     logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
192                 label, time.time() - t_start, len(all_candidate_windows))
193
194     if log_candidates:
195         for cw in all_candidate_windows:
196             logger.info(f"[{label} rally
197 {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])}] "
198                         f"({cw['end'] - cw['start']:.1f}s) |
199 frames={cw['active_frame_count']} "
200                         f"peak_v={cw['peak_velocity']:.3f}
201 mean_v={cw['mean_velocity']:.3f}")
202
203     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
204     fusion
205     # segmenter adds net_crossings + person-cue features. Computed once, reused for
206     both
207     # persistence and the handoff gate below.
208     window_stats = [
209         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
210         idx_cfg)
211         for cw in all_candidate_windows
212     ]
213
214     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
215     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
216     if store_candidates:
217         for i, cw in enumerate(all_candidate_windows):
218             candidate_ids[i] = self.db.add_candidate_segment(
219                 video_id, detector=self.name,
220                 start_time=cw["start"], end_time=cw["end"],
221                 chunk_index=cw["chunk_index"], confidence=min(1.0,
222                 cw["peak_velocity"]),
223                 stats=window_stats[i], detection_params=detection_params,
224             )
225
226     if not all_candidate_windows:
227         return []
228
229     # --- Phase 3: AI provider handoff over padded candidate clips ---
230     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
231     Persisted
232     # candidates above stay the full superset ? only the served play_segments are
233     gated.
234     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
235     idx_cfg)
236
237     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
238     os.makedirs(handoff_dir, exist_ok=True)
239     logger.info("Phase 3: %s validation on %d candidate windows...",
240                 provider_name, len(handoff_indices))
241
242     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
243         w_start = max(0, window["start"] - handoff_padding)
244         w_end = min(video_duration, window["end"] + handoff_padding)
245         w_dur = w_end - w_start
246         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
247         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
248         reencode=True):
249             return []
250
251         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
252         segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
253         label=f"Handoff {wi+1}")
254
255         valid = []
256         for seg in segments:

```

```

243         try:
244             seg["start_time"] = float(seg["start_time"]) + w_start
245             seg["end_time"] = float(seg["end_time"]) + w_start
246             seg["_candidate_id"] = candidate_ids[wi]
247             valid.append(seg)
248         except (ValueError, TypeError, KeyError) as e:
249             logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
250         try:
251             os.remove(w_path)
252         except OSError:
253             pass
254         return valid
255
256     ai_segments: List[dict] = []
257     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
258         futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
259                     for wi in handoff_indices]
260         for future in futures:
261             ai_segments.extend(future.result())
262         try:
263             os.rmdir(handoff_dir)
264         except OSError:
265             pass
266         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
267
268     # --- Phase 4: DB population + candidate linking ---
269     segments_data = []
270     for segment in ai_segments:
271         try:
272             start = float(segment["start_time"])
273             end = float(segment["end_time"])
274             dur = end - start
275             metadata = {
276                 "shot_count": self._shot_count_from(segment, dur),
277                 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
278                 "detector": f"{self.family}-hybrid-{model_name}",
279             }
280             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
281             if segment_id:
282                 candidate_id = segment.get("_candidate_id")
283                 if candidate_id is not None:
284                     self.db.link_candidate_to_segment(candidate_id, segment_id)
285                 segments_data.append({
286                     "id": segment_id, "start_time": start, "end_time": end,
287                     "duration": dur, "metadata": metadata,
288                 })
289         except Exception as e:
290             logger.warning("Skipping invalid segment struct: %s", e)
291
292     return segments_data
293

```

14. assistant (2026-06-14T10:50:34.394Z)

Now I see the served windowing source: line 149-151 ? `velocity\_threshold` defaults to `0.02`, `dead\_time\_merge\_gap` defaults to `2.0`, `min\_action\_window\_duration` defaults to `2.0`. Let me read the served check scripts and the numbers doc, plus the config.

15. user (2026-06-14T10:50:36.399Z)

Exit code 2

```
ls: cannot access 'scratch/served_gateA*.py': No such file or directory
---numbers doc---
ls: cannot access 'scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md': No such file or directory
---output dir---
ls: cannot access 'output/gateA_served_verify/': No such file or directory
```

16. assistant (2026-06-14T10:50:38.617Z)

These are untracked files in the main checkout (gitignored scratch/output), not in the fresh worktree. Let me look in the main checkout.

17. user (2026-06-14T10:50:40.599Z)

```
GATE_AB_NUMBERS_AND_IMPLICATIONS.md
gate_ab_ablation.py
gate_ab_ablation_result.json
gate_ab_detail.py
served_gateA_sweep.py
served_gateA_verify.py
---md---
scratch/ABLATION_FRAMEWORK_PLAN.md
scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md
scratch/HIGH_RISK_REMEDIATION_PLAN.md
scratch/OWNED_MODEL_PHASE0_PLAN.md
scratch/PERSONALIZATION_FEATURE_DESIGN.md
scratch/PERSONALIZATION_PIVOT_ANALYSIS.md
scratch/PR1_BODY.md
scratch/PR1_HANDOFF.md
scratch/PR2_HANDOFF.md
scratch/PR88_FIX.md
---output gateA---
adarsh_baseline_gated.csv
gateA_served_verify
```

18. user (2026-06-14T10:50:44.005Z)

```
1      # Gate-A/B foundational decision ? golden-set numbers + implications
2
3      **Owner's framing:** *NUMBERS INFORM, FOUNDATIONS DECIDE.* Extensibility, scope of
exploration,
4      and rigid foundational choices weigh more than raw ?F1 ? but this must be a **data-
INFLUENCED**
5      choice, so every number below is **actually measured** (no fabrication) on the n=6 owned
golden
6      corpus, IoU ? 0.5, with **honest stats (C1: bootstrap 95% CI + paired sign test; never a
bare
7      mean at n=6)**.
8
9      **Repro (CPU, seconds, no GPU/Gemini):**
10     `PYTHONPATH=<repo> python scratch/gate_ab_ablation.py` ?
`scratch/gate_ab_ablation_result.json`.
11     Reuses ONLY the shipped harness (`backend/eval/experiment.compare` +
`backend/eval/eval_stats`) over
12     the 6 persisted `output/*_golden_features.json`. All 6 JSONs already carry the audio
columns ? no
13     `--augment` step was needed.
14
15     **Corpus** (the 6 labelled matches in `output/human_lovo_manifest.json`):
16
17     | video | candidates | golden rallies | proposal-recall ceiling | note |
18     |---|---|---|---|---|
19     | adarsh_avi_singles | 172 | 96 | 0.833 | clean singles |
20     | targetest_doubles | 86 | 49 | 0.816 | doubles |
```

```

21 | mahadevpura_singles | 57 | 31 | 0.645 | single-court-ish |
22 | gbaaddy | 82 | 48 | **0.312** | multi-court |
23 | testlarge_short | 14 | 6 | 0.500 | short |
24 | kushagra_singles | 20 | 32 | **0.125** | multi-court |
25
26 Multi-court matches are **windowing-capped** (the decision layer cannot recover rallies
the proposal
27 windowing missed) ? their low absolute F1 is a proposal-recall problem, not a
gate/fusion failure.
28
29 ---
30
31 ## A. The two mechanisms (this is the foundational distinction)
32
33 The golden JSONs hold **two structurally different things**, and the Gate-A/B decision
lives in the
34 first one:
35
36 - **GATES = decision masks** over the recall-oriented CONTROL proposal set. `Gate-A =
net_crossings ? 2`
37 and `Gate-B = players_in_court ? N` are hard keep/drop rules applied *before* (or
instead of) any
38 learned scorer. In the harness these are `Variant.min_crossings` /
`Variant.min_players`
39 (`_gate_mask` in `experiment.py`). **This is what the foundational decision is
about.**
40 - **FEATURE COLUMNS = learned-fusion inputs.** shuttle (5: `duration, peak_velocity,
mean_velocity,
41 active_ratio, net_crossings`) + person (5) + audio (3) = **13 columns** a LOVO
LogisticRegression
42 weights. Here a signal contributes "softly" ? the scorer decides how much to trust it.
Removing one
43 group and re-measuring held-out ?F1 = that group's **marginal contribution**.
44
45 Note `net_crossings` appears in BOTH: as a **gate** (hard `?2`, the Gate-A decision) and
as a **soft
46 feature column**. They are not the same lever ? and the numbers below show the *gate*
form is where
47 all the power is.
48
49 ---
50
51 ## B. Comprehensive numbers table
52
53 ### B1. Gate-A ON vs OFF ? the foundational decision (static, CONTROL vs `net_crossings
? 2`)
54
55 | variant | F1 | P | R | seg-count ratio | # predicted segs |
56 |---|---|---|---|---|---|
57 | CONTROL (merge-all, no gate) | 0.388 | 0.313 | 0.539 | 1.68 | 431 |
58 | **Gate-A (`min_crossings ? 2`)** | **0.462** | **0.477** | 0.468 | **1.01** | 235 |
59
60 **?F1 = +0.074**, 95% CI **[+0.042, +0.105]** (clears 0), sign test **6W / 0L, p =
0.031**.
61 Per-video ?F1 (all positive): adarsh +0.125, mahadevpura +0.117, testlarge +0.075,
largetest +0.074,
62 gbaaddy +0.043, kushagra +0.009. **F1@k** all rise (F1@10 0.645?0.748, F1@25
0.603?0.698,
63 F1@50 0.388?0.462). The over-seg ratio **1.68 ? 1.01** is the headline structural
effect: Gate-A
64 nearly eliminates the over-segmentation tIoU barely reflected, by pruning **196 of 431**
low-crossing
65 proposal windows (held-shuttle / single-feed / dead-time FPs ? the owner's bug).
*Exactly reproduces
66 the first-A/B archive (`docs/archives/quality-iterations/2026-06-first-ab-

```

```

experiment/`).*
67
68     ### B2. Gate-B ON vs OFF (static, CONTROL vs `players_in_court ? N`)
69
70     | Gate-B threshold | F1 | ?F1 vs CONTROL | 95% CI | sign test | clears 0 | # segs 431? |
71     |---|---|---|---|---|---|
72     | N ? 1 | 0.388 | +0.000 | [0, 0] | 0W/0L | No | 431 (no-op) |
73     | N ? 2 | 0.389 | +0.000 | [0, +0.001] | 1W/0L | No | 430 (?no-op) |
74     | N ? 3 | 0.307 | **?0.081** | [?0.244, 0] | 0W/1L | No | 274 |
75     | N ? 4 | 0.283 | **?0.105** | [?0.305, +0.002] | 1W/2L | No | 231 |
76
77     **Gate-B is inert-to-harmful** on this corpus. At production-plausible thresholds
(N?1/2) it is a
78     **no-op** ? nearly every window already shows ?1?2 detected persons because there are
**no court
79     polygons**, so `players_in_court` counts **spectators + adjacent-court players**, not
just the two
80     on the active court. Pushing N higher to force discrimination starts **deleting real
rallies**
81     (N?3: ?0.081; N?4: ?0.105). **Stacked on Gate-A, Gate-B adds exactly 0.000** (N?1 and
N?2 both
82     leave Gate-A's 0.462 unchanged). Gate-B is **not measurable as a win until court
polygons exist.**
83
84     ### B3. Leave-one-signal-out ? marginal contribution to the learned 13-column fusion
(LOV0)
85     *(delta = F1(full 13 cols) ? F1(full minus that signal); positive ? the signal helps the
scorer)*
86
87     Full learned LOV0 reference: **F1 = 0.366** (P 0.453 / R 0.341).
88
89     | signal dropped | F1 full | F1 dropped | marginal ?F1 | 95% CI | sign test | clears 0 |
90     |---|---|---|---|---|---|
91     | **audio** (3 cols) | 0.366 | 0.286 | **+0.079** | [?0.054, +0.203] | 4W/2L (p=0.688) |
No |
92     | `net_crossings` (as a soft column) | 0.366 | 0.382 | **?0.014** | [?0.083, +0.027] |
4W/1L (p=0.375) | No |
93     | **person** (5 cols) | 0.366 | 0.376 | **?0.009** | [?0.074, +0.055] | 2W/4L (p=0.688)
| No |
94     | velocity block (peak+mean+active) | 0.366 | 0.364 | +0.002 | [?0.010, +0.018] | 2W/3L
(p=1.000) | No |
95     | `duration` | 0.366 | 0.367 | ?0.001 | [?0.004, 0] | 0W/2L (p=0.500) | No |
96
97     **Reading:** the only signal carrying *positive* marginal information for the learned
scorer is
98     **audio** (+0.079; the scorer loses the most when audio is removed). **`net_crossings`
as a soft
99     column is ~neutral-to-slightly-negative (?0.014)** ? its discriminative power is fully
captured by the
100     **hard Gate-A** form, not the learned column. Person is mildly negative (the no-court-
polygon noise).
101     Velocity/duration are ~zero marginal. **Every CI spans 0** (n=6) ? see the caveat.
102
103     ### B4. Incremental add-onto-shuttle (the P3/P4 framing, for cross-reference)
104
105     | step | F1 A?B | ?F1 | 95% CI | sign test | clears 0 |
106     |---|---|---|---|---|---|
107     | shuttle ? +person (P3) | 0.307 ? 0.286 | **?0.021** | [?0.165, +0.162] | 2W/4L
(p=0.688) | No |
108     | shuttle ? +audio | 0.307 ? 0.375 | **+0.068** | [?0.002, +0.140] | 4W/2L (p=0.688) |
No |
109     | shuttle+person ? +audio (P4) | 0.286 ? 0.366 | **+0.079** | [?0.054, +0.203] | 4W/2L
(p=0.688) | No |
110     | shuttle ? +person+audio | 0.307 ? 0.366 | +0.059 | [?0.039, +0.156] | 4W/2L (p=0.688)
| No |

```

```

111
112     Consistent with the leaderboard (P3 ?0.021, P4 incr. +0.079). Audio is the lead learned
cue; person
113     is a wash on its own. Note **audio added directly to shuttle (+0.068, CI [?0.002,
+0.140])** is the
114     cleanest audio read (drops the person handicap) and its CI **barely** touches 0 ? the
most promising
115     near-promotable signal.
116
117     ### B5. Existing baselines (context ? IoU 0.5 vs human golden, from `eval_baselines/` +
QUALITY_ITERATIONS)
118
119     | path | Mahadevpura (clean-ish) | Kushagra (multi-court) | corpus-mean F1 |
120     |---|---|---|---|
121     | Heuristic (velocity windowing, LOVO n=6) | 0.657 | 0.157 | 0.48 |
122     | Gen-0 owned head (per-frame proto, LOVO n=6) | 0.744 | 0.561 | 0.583 |
123     | Two-stage WASB+Gemini refine | 0.83 | ? | ? |
124     | Gemini wrapper (`gemini-flash-latest`) | **0.93** | **0.66** | ? |
125
126     Wrapper baseline cost note: **well under $1 for two videos** at flash pricing (serial,
~14 chunk
127     requests). Multi-court drops the commodity wrapper 0.93 ? 0.66 ? that 0.66 is the owned
model's ship
128     target. **Caveat on cross-comparing rows:** the fusion/Gate numbers ($B1?B4) are scored
on a
129     *recall-oriented proposal* candidate set (CONTROL ? 0.388 mean), a different operating
regime than the
130     heuristic/Gen-0/wrapper rows; don't read 0.462 vs 0.93 as "Gate-A loses to Gemini" ?
they answer
131     different questions (proposal-decision lift vs end-to-end detector quality).
132
133     ---
134
135     ## C. Gate-A default flip ? implications (numbers AND foundations)
136
137     ### C1. What the numbers say
138     Gate-A is the **only** lever in this whole study that clears the honest promotion bar:
?F1 +0.074
139     with a CI that excludes 0 **and** a 6/0 sign test (p=0.031) ? it improves F1 on
**every** golden
140     match and slashes over-segmentation (1.68?1.01). It is cheap, GPU-free, deterministic,
and was
141     already corroborated on real footage (GX010125 offline re-score: 45 ? 27 rallies, 18
low-crossing FP
142     windows dropped). By the repo's own "promote only on measured lift" rule, **Gate-A
qualifies and
143     Gate-B does not.**
144
145     ### C2. Extensibility ? does defaulting Gate-A ON constrain or enable future signal
fusion?
146
147     **It ENABLES, and here's the structural reason.** Gate-A is a **proposal-stage precision
filter**, not
148     a verdict. It prunes structurally-impossible-to-be-a-rally windows (a held shuttle never
crosses the
149     net twice) *before* the learned scorer / Gemini ever sees them. That is strictly
**complementary** to
150     adding more signals:
151
152     - It **raises the floor every later cue builds on.** The full-13 learned fusion is
measured on the
153     *un-gated* CONTROL set (F1 0.366); a learned fusion measured **on top of a Gate-A-
pruned set** starts
154     from 0.462, with the worst FP class already removed ? every cue's job gets easier, not
harder.

```

155 - It does **not** consume or mask any feature column. The persisted superset
(shuttle+person+audio, 13
156 cols) stays intact in the JSONs; Gate-A only changes which **windows* survive to
scoring*. So audio
157 (the promising +0.079 lead) and any future cue are added exactly as before ? Gate-A is
orthogonal
158 to the fusion layer, living one stage upstream.
159 - The harness already treats it as a first-class, swappable *`Variant`* knob
(*`min\_crossings`*), so a
160 future cue that **subsumes** Gate-A's signal can be A/B'd against it and Gate-A flipped
back off via
161 config ? **the experiment seam is preserved.**
162
163 **The one real constraint to name honestly:** Gate-A is a **hard gate**, not a soft
feature. The
164 leave-one-out numbers (\$B3) show the **soft** *`net\_crossings`* column is ~neutral (?0.014)
while the
165 **hard** *`?2`* gate is the win ? i.e. the value is in the thresholding, which a learned
scorer did **not**
166 reproduce from the column at n=6. Defaulting the hard gate ON therefore bakes in a
rule-based pre-filter
167 alongside the learned path. That is a deliberate two-stage shape (rule-gate ?
learned/Gemini scorer),
168 not pure end-to-end learning. It is reversible (config flag) and standard (proposal-
filtering before
169 scoring is textbook Temporal Action Localization), but it IS a foundational commitment
to a
170 **hybrid rule+learned pipeline**, which the owner should make eyes-open.
171
172 **### C3. Scope of exploration ? what does the default flip open vs close?**
173
174 - **Opens:** a clean, higher-precision substrate for the **next** signal hunt (audio
first). Every future
175 cue gets measured against the Gate-A floor, so a real lift is more likely to be a
real lift and less
176 likely to be "it cleaned up held-shuttle FPs the gate would have caught anyway."
177 - **Opens:** the held-shuttle / single-feed / dead-time FP class is closed structurally,
freeing the
178 exploration budget to chase the **harder, temporal multi-court confound** (the
documented dead end for
179 spatial/loudness heuristics ? only the learned temporal model cracks it).
180 - **Closes nothing important:** because the persisted superset is untouched, no future
ablation is
181 foreclosed. The only thing it "closes" is re-litigating held-shuttle FPs by other
means.
182
183 **### C4. Foundational structure ? is it hard to reverse?**
184
185 **No ? and this is the key de-risking fact.** Gate-A as shipped is
`indexing.fusion.min\_crossings`
186 (default 0 = OFF today). Flipping the default to 2 is a one-line config change; rollback
is flipping it
187 back (the repo's stated discipline: "rollback = flip a config flag, never *`git`*
`revert`"). It does not
188 alter the DB schema, the persisted feature superset, or any registry contract. The
served-side flip
189 is the only part still owner-gated, and only because the production fusion segmenter
(P2) is unverified
190 on a live real-video run ? **that's a verification gap, not a reversibility problem.**
191
192 The Gate-B side IS a foundational question, but the data says **don't build on it yet**:
it is inert
193 without court polygons and harmful when forced. Committing to Gate-B now would bake a
dependency on a
194 court-mask source (manual polygon vs auto-homography ? an open owner decision) into

```

the default path
195     for **zero measured benefit**. Keep Gate-B default-OFF; revisit only after court
polygons exist, then
196     re-measure person *and* Gate-B together (the current ?0.021 person result is partly a
`court_polygon=None`
197     artifact).
198
199     ---
200
201     ## D. Recommendation framing (numbers + foundations, not numbers alone)
202
203     1. **Promote Gate-A to the default on the fusion/proposal path**
(`indexing.fusion.min_crossings = 2`).
204     - *Numbers:* the only lever that clears the honest bar (+0.074, CI clears 0, 6/0,
p=0.031;
205         over-seg 1.68?1.01) ? measured, reproduced, and corroborated on real footage.
206     - *Foundations:* it's an upstream, orthogonal, reversible precision pre-filter that
*raises the floor*
207         for every future cue without consuming any feature or foreclosing any ablation. The
only eyes-open
208         commitment is to a **hybrid rule+learned two-stage shape**, which is standard and
config-reversible.
209     - *Gate on:* the live served-side flip stays owner-gated **only** until one real
fusion-path run
210         confirms P2 behaves as the offline re-score predicts (the GX010125 evidence already
points the
211         right way). This is verification, not a numbers question.
212
213     2. **Keep Gate-B default-OFF.** No measured benefit (no-op at N?2, harmful at N?3,
+0.000 stacked on
214         Gate-A). It is **blocked on court polygons**, not on more data. Do not bake a court-
mask dependency
215         into the default path for zero lift. Re-open Gate-B *and* the person cue together
once polygons exist.
216
217     3. **Make audio the next promotion candidate, not person.** Audio is the strongest
learned cue
218         (leave-out +0.079; add-onto-shuttle +0.068 with a CI that only just touches 0). The
honest blocker is
219         n=6: every fusion CI spans 0. **Growing the golden corpus (Roora) is the highest-ROI
unblock** ? it is
220         the only thing that can turn the +0.07-ish audio lead and the person re-measurement
into confirmable
221         results.
222
223     4. **Be honest about the central caveat:** *at n=6 every learned-fusion CI spans 0.*
Gate-A is the lone
224         exception (its CI clears 0 and the sign test is significant) precisely because it's a
strong, simple,
225         structural rule that wins on all 6 videos ? not a marginal learned weight. So the
numbers **decisively**
226         support Gate-A and **decisively reject Gate-B**, but only **suggest** (cannot
confirm) audio and
227         **cannot exonerate** person until N grows. This is a data-influenced decision where
the data is strong
228         exactly where the foundational choice is being made (Gate-A) and weak where it isn't
(the soft cues).
229
230     ---
231
232     ## E. Is a served end-to-end Gemini run on the golden videos warranted? ? **NO (for the
decision).**
233
234     **Reason.** The Gate-A/B decision lives in the **proposal/fusion layer, upstream of
Gemini**, and is

```

235 already settled by the persisted-feature ablation against golden labels (6/6, p=0.031, CI clears 0).

236 A served Gemini run would measure a **different** thing ? final highlight-reel quality with the gate

237 flipped on in the live path ? and it ***cannot move a numbers-driven decision*** because:

238 - There are ***no golden labels*** for new served-pipeline footage, so the output is divergence

239 (*`a\_only`/`b\_only`* windows), not F1 ? illustrative, not decisive.

240 - The golden videos' served behavior under Gate-A is ***already corroborated*** by the GX010125 offline

241 re-score (45 ? 27 rallies, 18 low-crossing FP windows dropped, ~1.7 min), at zero Gemini cost.

242 - The remaining owner-gate on the served-side flip is a ***verification*** that the production fusion

243 segmenter (P2) behaves as the offline path predicts ? a cheap single real-video run, ***not*** a

244 golden-corpus Gemini sweep.

245

246 ***What a Gemini run WOULD be worth (optional, owner's call ? flagged, not run):*** a **confidence-builder /*

247 *demo**, not a decision input ? run the served fusion pipeline on ONE golden match with *`min\_crossings`*

248 OFF vs 2 and eyeball the two reels to confirm P2 drops the held-shuttle windows end-to-end.

249 - ***Expected cost/time:*** ~1 golden match ? tens of minutes wall-clock (GPU WASB/person pass +

250 ffmpeg + serial Gemini chunks); Gemini spend ***< \$1*** at flash pricing (the wrapper baseline did two

251 videos for under \$1). Two reels (gate OFF/ON) ? ***2× that***, still sub-\$2.

252 - ***Recommendation:*** worth it once, as the ***owner-gated served-side verification*** of P2 (item D1's

253 gate), but ***not*** as input to the Gate-A/B foundational decision ? the CPU ablation already decided

254 that. Do not burn a multi-video Gemini sweep; it adds cost, not decision value.

255

256 ---

257

258 *### Artifacts*

259 - Runner: *`scratch/gate\_ab\_ablation.py`* . detail: *`scratch/gate\_ab\_detail.py`*

260 - Raw measured result: *`scratch/gate\_ab\_ablation\_result.json`*

261 - Reuses: *`backend/eval/experiment.py`*, *`backend/eval/eval\_stats.py`*,

262 *`backend/eval/segmentation\_metrics.py`*, *`output/\*\_golden\_features.json`* (the persisted substrate).

263 - Cross-refs: *`docs/QUALITY\_ITERATIONS.md`*,

264 *`eval\_baselines/experiment\_leaderboard.json`*,

265 *`docs/archives/quality-iterations/2026-06-first-ab-experiment/README.md`*.

19. assistant (2026-06-14T10:50:47.857Z)

This is very informative. Now I understand the crux: the +0.074 Gate-A win was measured on the CONTROL variant whose candidates come from the ***proposal windowing*** (vt=0.005/gap=1.0/min=0.5). Now let me read the served check scripts and config.

20. user (2026-06-14T10:50:48.419Z)

1 *""Served-side verification of the Gate-A default flip (PR #146).*

2

3 Confirms the LIVE *`FusionSegmenter`* served path (not the offline harness's

4 recall-oriented proposal) behaves as the flip promises, WITHOUT re-running WASB

5 (wasb_runner has no cache-hit early-return ? re-running would overwrite the cached

6 CSV). We reuse each golden video's CACHED WASB trajectory and drive the REAL served

7 code: the same production windowing *`process\_video`* uses, then the actual

8 *`FusionSegmenter.\_enrich\_candidate\_stats`* (net_crossings) and *`\_select\_for\_handoff`*

9 (the served Gate-A gate) ? the exact methods Gemini's handoff set is built from.

```

10
11 Two questions (the owner-gated follow-up flagged in #146):
12     (a) At indexing.fusion.min_crossings=0 (ungated control), does the fusion handoff
13         set match a wasb_hybrid run on the same trajectory? (byte-identical seeding +
14         handoff at mc=0 ? the no-regression invariant on the real served path.)
15     (b) At min_crossings=2 (the Gate-A default), does it drop ONLY low-crossing
16         (held-shuttle/feed) windows ? no real-rally loss vs golden labels ? i.e. no
17         F1 regression?
18
19 Config is loaded EXACTLY as production (`load_config()`); segmenters come from the
20 registry. Honest measured numbers only. Artifacts isolated under
21 output/gateA_served_verify/. No GPU, no Gemini, no WASB re-run.
22 ""
23 from __future__ import annotations
24
25 import copy
26 import json
27 import os
28 from typing import Any, Dict, List, Tuple
29
30 from backend.config.loader import load_config
31 from backend.pipeline.segmenters.base import segmenter_registry
32 # Import for side-effect registration of both segmenters.
33 import backend.pipeline.segmenters.fusion_hybrid # noqa: F401
34 import backend.pipeline.segmenters.wasb_hybrid # noqa: F401
35 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
36 from backend.eval.calibrate_wasb import load_golden_gts
37 from backend.eval.training import label_candidates
38 from backend.eval.metrics import score
39
40 Interval = Tuple[float, float]
41 MANIFEST = "output/human_lovo_manifest.json"
42 OUT_DIR = "output/gateA_served_verify"
43 IOU = 0.5
44
45
46 def win_params(idx_cfg: Dict[str, Any], family: str) -> Tuple[float, float, float]:
47     """Reproduce TrajectoryHybridSegmenter.process_video's windowing knobs exactly."""
48     vt = idx_cfg.get(family, {}).get("velocity_threshold", 0.02)
49     mg = idx_cfg.get("dead_time_merge_gap", 2.0)
50     mwd = idx_cfg.get("min_action_window_duration", 2.0)
51     return float(vt), float(mg), float(mwd)
52
53
54 def windows_for(points, fps: float, fw: float, vt: float, mg: float, mwd: float) ->
List[Dict[str, Any]]:
55     return TrackNetRunner.trajectory_to_action_windows(
56         points, fps=fps, frame_width=fw, chunk_start=0.0,
57         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd, chunk_index=0,
58     )
59
60
61 def ivs(windows: List[Dict[str, Any]], idxs) -> List[Interval]:
62     return [(windows[i]["start"], windows[i]["end"]) for i in idxs]
63
64
65 def run_video(spec: Dict[str, Any], fusion, wasb, base_idx_cfg) -> Dict[str, Any]:
66     name = spec["name"]
67     fps, fw = float(spec["fps"]), float(spec["frame_width"])
68     points = TrackNetRunner.parse_trajectory_csv(spec["trajectory"])
69     gts = load_golden_gts(spec["labels"])
70
71     # --- Served windowing for each segmenter, from its own family config ---
72     f_vt, f_mg, f_mwd = win_params(base_idx_cfg, fusion.family) # family="fusion"
73     w_vt, w_mg, w_mwd = win_params(base_idx_cfg, wasb.family) # family="wasb"

```

```

74     f_windows = windows_for(points, fps, fw, f_vt, f_mg, f_mwd)
75     w_windows = windows_for(points, fps, fw, w_vt, w_mg, w_mwd)
76
77     # (a) Same seeding: fusion and wasb produce identical candidate windows.
78     f_iv = [(w["start"], w["end"]) for w in f_windows]
79     w_iv = [(w["start"], w["end"]) for w in w_windows]
80     seeding_identical = f_iv == w_iv
81
82     # Real served enrichment (net_crossings) via the actual FusionSegmenter method.
83     f_stats = [fusion._enrich_candidate_stats(cw, points, fps, fw, "", base_idx_cfg)
84                for cw in f_windows]
85
86     # Handoff selection through the REAL served gate at mc=0 and mc=2.
87     cfg0 = copy.deepcopy(base_idx_cfg); cfg0.setdefault("fusion", {})[ "min_crossings" ] =
88     0
89     cfg2 = copy.deepcopy(base_idx_cfg); cfg2.setdefault("fusion", {})[ "min_crossings" ] =
90     2
91     keep0 = fusion._select_for_handoff(f_windows, f_stats, cfg0)
92     keep2 = fusion._select_for_handoff(f_windows, f_stats, cfg2)
93     # wasb_hybrid's handoff = the base seam (all windows, no gating).
94     wasb_keep = wasb._select_for_handoff(w_windows, [{ } for _ in w_windows],
base_idx_cfg)
95
96     # (a) Handoff identity at mc=0: same windows AND same selected set as wasb_hybrid.
97     handoff_identical = (seeding_identical and list(keep0) ==
list(range(len(f_windows)))
98     and list(wasb_keep) == list(range(len(w_windows))))
99
100    # (b) Label every window once vs golden; classify each Gate-A drop as FP or real-
rally loss.
101    labels = label_candidates(f_iv, gts, IOU)
102    dropped = [i for i in keep0 if i not in set(keep2)]
103    drops = [{"start": round(f_windows[i]["start"], 2), "end":
round(f_windows[i]["end"], 2),
104             "net_crossings": f_stats[i].get("net_crossings", 0.0),
105             "is_real_rally": bool(labels[i])} for i in dropped]
106    real_rally_losses = [d for d in drops if d["is_real_rally"]]
107
108    s0 = score(ivs(f_windows, keep0), gts, IOU)
109    s2 = score(ivs(f_windows, keep2), gts, IOU)
110
111    return {
112        "name": name, "fps": fps,
113        "num_gts": len(gts), "num_windows": len(f_windows),
114        "seeding_identical_to_wasb": seeding_identical,
115        "handoff_identical_at_mc0": handoff_identical,
116        "win_params_fusion": [f_vt, f_mg, f_mwd], "win_params_wasb": [w_vt, w_mg,
w_mwd],
117        "handoff_mc0": len(keep0), "handoff_mc2": len(keep2),
118        "dropped_by_gateA": len(dropped),
119        "real_rally_losses": len(real_rally_losses),
120        "drops": drops,
121        "mc0": {"P": round(s0.precision, 3), "R": round(s0.recall, 3), "F1":
round(s0.f1, 3),
122               "TP": s0.true_positives, "FP": s0.false_positives, "FN":
s0.false_negatives},
123        "mc2": {"P": round(s2.precision, 3), "R": round(s2.recall, 3), "F1":
round(s2.f1, 3),
124               "TP": s2.true_positives, "FP": s2.false_positives, "FN":
s2.false_negatives},
125        "delta_f1": round(s2.f1 - s0.f1, 3),
126    }
127
128    def main() -> None:

```

```

128     os.makedirs(OUT_DIR, exist_ok=True)
129     config = load_config() # exactly production
130     idx_cfg = config.get("indexing", {}) # plain dict, identical to served path
131     FusionCls = segmenter_registry.get("fusion_hybrid")
132     WasbCls = segmenter_registry.get("wasb_hybrid")
133     fusion = FusionCls(None, config)
134     wasb = WasbCls(None, config)
135
136     specs = json.load(open(MANIFEST, encoding="utf-8"))
137     print("-" * 92)
138     print("SERVED-SIDE Gate-A verification ? live FusionSegmenter over CACHED golden
trajectories")
139     print(f"(config.indexing.fusion = {idx_cfg.get('fusion')})")
140     print("-" * 92)
141
142     results = [run_video(s, fusion, wasb, idx_cfg) for s in specs]
143
144     print(f"\n{'video':<22}{ 'win':>4}{ 'mc0':>5}{ 'mc2':>5}{ 'drop':>5}{ 'lost':>5}"
145           f"{'F1@mc0':>8}{ 'F1@mc2':>8}{ '?F1':>7} seed=wasb handoff==wasb@mc0")
146     all_identical = True
147     total_lost = 0
148     for r in results:
149         all_identical &= r["seeding_identical_to_wasb"] and
r["handoff_identical_at_mc0"]
150         total_lost += r["real_rally_losses"]
151
152     print(f"{'name':<22}{ 'num_windows':>4}{ 'handoff_mc0':>5}{ 'handoff_mc2':>5}"
153           f"{'dropped_by_gateA':>5}{ 'real_rally_losses':>5}"
154           f"{'mc0':>8}{ 'F1':>8.3f}{ 'mc2':>8.3f}{ 'delta_f1':>+7.3f}"
155           f"{'seeding_identical_to_wasb':>5}"
156           f"{'handoff_identical_at_mc0':>5}")
157
158     n = len(results)
159     mean_f1_0 = sum(r["mc0"]["F1"] for r in results) / n
160     mean_f1_2 = sum(r["mc2"]["F1"] for r in results) / n
161     mean_p0 = sum(r["mc0"]["P"] for r in results) / n
162     mean_p2 = sum(r["mc2"]["P"] for r in results) / n
163     mean_r0 = sum(r["mc0"]["R"] for r in results) / n
164     mean_r2 = sum(r["mc2"]["R"] for r in results) / n
165
166     print("-" * 92)
167     print(f"MEAN over n={n}: P {mean_p0:.3f}->{mean_p2:.3f} R
{mean_r0:.3f}->{mean_r2:.3f} "
168           f"F1 {mean_f1_0:.3f}->{mean_f1_2:.3f} (? {mean_f1_2 - mean_f1_0:+.3f})")
169     print(f"\n(a) mc=0 seeding+handoff byte-identical to wasb_hybrid on all videos : "
170           f"{'YES ?' if all_identical else 'NO ?'}")
171     print(f"(b) real-rally windows dropped by Gate-A (mc=2) across corpus : "
172           f"{'total_lost' if (total_lost > 0) else 'no drops'}")
173
174     print(f" F1 regression on any video : "
175           f"{'NONE ?' if all(r['delta_f1'] >= 0 for r in results) else 'YES ?'}")
176
177     summary = {
178         "purpose": "served-side Gate-A flip verification (PR #146) via live
FusionSegmenter "
179         "over cached golden trajectories ? no WASB re-run, no Gemini",
180         "config_fusion": idx_cfg.get("fusion"),
181         "iou": IOU, "n": n,
182         "mc0_seeding_handoff_identical_to_wasb_all_videos": all_identical,
183         "total_real_rally_losses_gateA": total_lost,
184         "no_f1_regression_any_video": all(r["delta_f1"] >= 0 for r in results),
185         "mean_f1_mc0": round(mean_f1_0, 3), "mean_f1_mc2": round(mean_f1_2, 3),
186         "mean_delta_f1": round(mean_f1_2 - mean_f1_0, 3),
187         "per_video": results,
188     }

```

```

186         json.dump(summary, open(os.path.join(OUT_DIR, "served_gateA_summary.json"), "w"),
indent=2)
187         print(f"\nwrote {OUT_DIR}/served_gateA_summary.json")
188
189
190     if __name__ == "__main__":
191         main()
192

```

21. assistant (2026-06-14T10:50:51.099Z)

Now let me look at the served verify summary output and the served sweep script to see the actual served numbers.

22. user (2026-06-14T10:50:52.788Z)

```

total 32
drwxr-xr-x 1 avidu 197609      0 Jun 14 16:06 .
drwxr-xr-x 1 avidu 197609      0 Jun 14 16:06 ..
-rw-r--r-- 1 avidu 197609 10412 Jun 14 16:06 served_gateA_summary.json
===SUMMARY===
{
  "purpose": "served-side Gate-A flip verification (PR #146) via live FusionSegmenter over
cached golden trajectories \u2014 no WASB re-run, no Gemini",
  "config_fusion": {
    "substrate": "wasb",
    "velocity_threshold": 0.02,
    "cue_flags": {},
    "min_crossings": 2,
    "min_players": 0,
    "court_polygon": null,
    "net_axis": "y",
    "net_position": 0.5
  },
  "iou": 0.5,
  "n": 6,
  "mc0_seeding_handoff_identical_to_wasb_all_videos": true,
  "total_real_rally_losses_gateA": 12,
  "no_f1_regression_any_video": false,
  "mean_f1_mc0": 0.3,
  "mean_f1_mc2": 0.292,
  "mean_delta_f1": -0.008,
  "per_video": [
    {
      "name": "adarsh_avi_singles",
      "fps": 59.94,
      "num_gts": 96,
      "num_windows": 97,
      "seeding_identical_to_wasb": true,
      "handoff_identical_at_mc0": true,
      "win_params_fusion": [
        0.02,
        2.0,
        2.0
      ],
      "win_params_wasb": [
        0.02,
        2.0,
        2.0
      ],
      "handoff_mc0": 97,
      "handoff_mc2": 76,
      "dropped_by_gateA": 21,
      "real_rally_losses": 7,
      "drops": [

```

```
{
  "start": 152.55,
  "end": 156.99,
  "net_crossings": 1.0,
  "is_real_rally": true
},
{
  "start": 183.77,
  "end": 191.21,
  "net_crossings": 1.0,
  "is_real_rally": false
},
{
  "start": 202.44,
  "end": 204.52,
  "net_crossings": 1.0,
  "is_real_rally": false
}
```

23. assistant (2026-06-14T10:50:57.963Z)

This confirms the exact problem precisely. The numbers are:

- **PROPOSAL windowing** (vt=0.005/gap=1.0/min=0.5): Gate-A ?F1 = **+0.074** (CI clears 0, 6/0 sign test).
- **SERVED windowing** (vt=0.02/gap=2.0/min=2.0): Gate-A ?F1 = **?0.008**, with 12 real-rally losses across the corpus, F1 regression on some videos.

The eval?serve gap is precisely what allowed Gate-A to be "promoted" (#146 flipped it to served default) on a windowing production doesn't serve. Now let me look at the config loader to understand where served defaults live, eval_stats, and per_user_gate.

24. user (2026-06-14T10:50:58.956Z)

```
__init__.py
ablation.py
annotations.py
audio
batch.py
calibrate_local.py
calibrate_wasb.py
calibration.py
classifier.py
distill_local.py
eval_partial_labels.py
eval_stats.py
experiment.py
export_wasb_segments.py
fusion_audio.py
fusion_compare.py
fusion_golden.py
gemini_refine.py
gt_loader.py
harness.py
metrics.py
per_user_eval.py
per_user_gate.py
per_user_regression.py
rally_seq_proto.py
regression.py
score_calibration.py
segmentation_metrics.py
training.py
===config===
__init__.py
loader.py
models.py
```

25. user (2026-06-14T10:51:01.248Z)

```
1      """Per-user promotion gate ? the safety rail for flag-gated personalization.
2
3      The owner adopted personalization as a flag-gated feature on a generalization-
first
4      baseline: a cue/variant B may be flipped ON for one user only when that user's own
5      held-out evidence honestly clears the same small-N bar the global eval uses ? otherwise
we
6      fall back to the shipped baseline (variant A). This module is the decision logic for
that
7      flip; it is pure (no I/O) and reuses backend.eval.eval_stats` for every statistic
8      (paired_delta_ci` + `paired_sign_test`) so the math is never reinvented.
9
10     The counter-case this guards against (and its mitigations, implemented here):
11
12     1. Minimum-N floor. Personalizing off 1-2 of a user's videos is the small-N-lies
regime
13         (see eval_stats` C1). Refuse to promote below min_n` videos ? point estimates
from
14         too few held-out videos are noise, not signal.
15     2. Honest lift, not a single mean. Promote only when the paired ?F1 95% bootstrap CI
16         excludes 0 and the paired sign test agrees (majority of videos win,  $p \leq$ 
threshold).
17         This is the global promotion bar applied per user.
18     3. Structural guards are never disabled on "no lift". A structural guard (e.g.
Gate-A
19         net-crossings ? the held-shuttle false-positive killer) earns its keep through
failure
20         modes a given user's corpus may not contain yet. Measuring "no per-user lift" on a
corpus
21         that simply lacks those failure cases must NEVER turn the guard OFF. For
structural=True`
22         the decision is therefore always "keep ON", independent of the measured delta.
23
24     Below the floor / insufficient evidence ? fall back to the shipped baseline (the
conservative
25     default), never an unsafe promotion. All additive, pure, deterministic (the bootstrap
seed is
26     threaded through). Nothing here changes existing eval behaviour; callers opt in.
27     """
28     from __future__ import annotations
29
30     from dataclasses import dataclass, field
31     from typing import Any, Dict, Sequence
32
33     from backend.eval.eval_stats import paired_delta_ci, paired_sign_test
34
35     __all__ = ["PromotionDecision", "should_promote_for_user"]
36
37
38     @dataclass
39     class PromotionDecision:
40         """Structured result of a per-user promotion decision.
41
42         - promote` ? flip variant B ON for this user (only ever True` for a non-
structural
43         cue that cleared the full bar).
44         - keep_on` ? the cue/guard should be ON for this user. For a structural guard
this is
45         always True` (it is never disabled on "no lift"); for a non-structural cue it
equals
46         promote`.
47         - reason` ? short human-readable explanation of the decision.
48         - the remaining fields surface the evidence (n, mean delta, CI bounds and the two
```

```

tests)
49         so a caller can log/audit exactly why a user was or wasn't personalized.
50         """
51
52     promote: bool
53     keep_on: bool
54     reason: str
55     n: int
56     mean_delta: float
57     ci_low: float
58     ci_high: float
59     ci_excludes_zero: bool
60     sign_test: Dict[str, float] = field(default_factory=dict)
61     floor_met: bool = False
62     structural_protected: bool = False
63
64     def as_dict(self) -> Dict[str, Any]:
65         """Plain-dict view (for JSON logging / telemetry)."""
66         return {
67             "promote": self.promote,
68             "keep_on": self.keep_on,
69             "reason": self.reason,
70             "n": self.n,
71             "mean_delta": self.mean_delta,
72             "ci_low": self.ci_low,
73             "ci_high": self.ci_high,
74             "ci_excludes_zero": self.ci_excludes_zero,
75             "sign_test": self.sign_test,
76             "floor_met": self.floor_met,
77             "structural_protected": self.structural_protected,
78         }
79
80
81     def should_promote_for_user(
82         a_scores: Sequence[float],
83         b_scores: Sequence[float],
84         *,
85         structural: bool = False,
86         min_n: int = 5,
87         p_threshold: float = 0.05,
88         confidence: float = 0.95,
89         n_boot: int = 2000,
90         seed: int = 0,
91         eps: float = 0.0,
92     ) -> PromotionDecision:
93         """Decide whether to flip variant B ON for ONE user from that user's held-out
94         scores.
95
96         ``a_scores`` / ``b_scores`` are the user's per-video held-out F1 (or any score)
97         for
98         variant A (control / shipped baseline) and variant B (with the cue), aligned by
99         video
100         (same order). The decision rules (faithful to the module docstring):
101
102         1. ``structural=True`` ? the guard is never disabled on "no lift": return
103            ``keep_on=True, promote=False`` regardless of the measured per-user delta. Its
104            value is
105            in failure modes the user's corpus may not yet contain.
106            2. Otherwise promote iff ALL hold ? the minimum-N floor (``n >= min_n``), the
107            paired
108            ?F1 bootstrap CI excludes 0, and the paired sign test agrees (majority of
109            win, ``p_value <= p_threshold``). Any miss ? fall back to the shipped baseline.
110            Statistics come straight from ``eval_stats`` (``paired_delta_ci`` /

```

```

``paired_sign_test``);
107     deterministic given ``seed``. Returns a :class:`PromotionDecision`.
108     """
109     n = min(len(a_scores), len(b_scores))
110     floor_met = n >= min_n
111
112     # --- Evidence (reuse eval_stats; never reinvent the statistics)
    -----
113     if n == 0:
114         # No paired videos at all ? nothing to test. Surface an empty-but-honest result.
115         sign_test: Dict[str, float] = paired_sign_test([], eps)
116         mean_delta, ci_low, ci_high, ci_excludes_zero = 0.0, 0.0, 0.0, False
117     else:
118         delta = paired_delta_ci(
119             a_scores, b_scores,
120             confidence=confidence, n_boot=n_boot, seed=seed, eps=eps,
121         )
122         mean_delta = float(delta["mean_delta"])
123         ci_low = float(delta["ci_low"])
124         ci_high = float(delta["ci_high"])
125         ci_excludes_zero = bool(delta["ci_excludes_zero"])
126         sign_test = delta["sign_test"]
127
128     sign_p = float(sign_test.get("p_value", 1.0))
129     sign_wins = float(sign_test.get("wins", 0.0))
130     sign_losses = float(sign_test.get("losses", 0.0))
131     # The sign test "agrees" with a promotion only if it is significant AND the majority
132     # leans the *winning* way (more B-wins than B-losses) ? a significant net *loss*
    must
133     # never satisfy the promotion bar.
134     sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
135
136     # --- Rule 1: structural guards are never disabled on "no lift"
    -----
137     if structural:
138         return PromotionDecision(
139             promote=False,          # nothing to "promote" ? it ships ON by design
140             keep_on=True,          # ALWAYS kept ON, regardless of measured per-user
    lift
141             reason=(
142                 "structural guard kept ON regardless of measured per-user lift "
143                 "(value is in failure modes this user's corpus may not yet contain)"
144             ),
145             n=n,
146             mean_delta=mean_delta,
147             ci_low=ci_low,
148             ci_high=ci_high,
149             ci_excludes_zero=ci_excludes_zero,
150             sign_test=sign_test,
151             floor_met=floor_met,
152             structural_protected=True,
153         )
154
155     # --- Rule 2: minimum-N floor (refuse before even looking at lift)
    -----
156     if not floor_met:
157         return PromotionDecision(
158             promote=False,
159             keep_on=False,          # fall back to the shipped baseline (variant A)
160             reason=(
161                 f"below minimum-N floor (n={n} < min_n={min_n}); "
162                 "falling back to shipped baseline"
163             ),
164             n=n,
165             mean_delta=mean_delta,

```

```

166         ci_low=ci_low,
167         ci_high=ci_high,
168         ci_excludes_zero=ci_excludes_zero,
169         sign_test=sign_test,
170         floor_met=False,
171         structural_protected=False,
172     )
173
174     # --- Rule 2 (cont.): honest lift ? CI excludes 0 AND sign test agrees
175     if ci_excludes_zero and sign_agrees:
176         promote = True
177         reason = (
178             f"promoted: n={n} >= {min_n}, ?F1 95% CI "
179             f"[{ci_low:.4f}, {ci_high:.4f}] excludes 0, "
180             f"sign test {int(sign_wins)}-{int(sign_losses)} (p={sign_p:.4f} <=
181             {p_threshold})"
182         )
183     else:
184         promote = False
185         misses = []
186         if not ci_excludes_zero:
187             misses.append(f"?F1 95% CI [{ci_low:.4f}, {ci_high:.4f}] spans 0")
188         if not sign_agrees:
189             misses.append(
190                 f"sign test does not agree ({int(sign_wins)}-{int(sign_losses)}), "
191                 f"p={sign_p:.4f}"
192             )
193         reason = (
194             "insufficient evidence (" + "; ".join(misses) + "); "
195             "falling back to shipped baseline"
196         )
197     return PromotionDecision(
198         promote=promote,
199         keep_on=promote,          # non-structural: keep_on tracks promote exactly
200         reason=reason,
201         n=n,
202         mean_delta=mean_delta,
203         ci_low=ci_low,
204         ci_high=ci_high,
205         ci_excludes_zero=ci_excludes_zero,
206         sign_test=sign_test,
207         floor_met=True,
208         structural_protected=False,
209     )
210

```

26. assistant (2026-06-14T10:51:05.899Z)

Now let me read eval_stats and fusion_compare/fusion_golden to understand how offline candidates are built, plus the config models/loader for the served constants.

27. user (2026-06-14T10:51:06.501Z)

```

1     """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden
2     set.
3     The fusion-P3 measurement. Loads the per-video feature JSONs written by
4     `fusion_golden.py`
5     and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-
6     out) ?
7     no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the
8     harness's
9     designed seam): the person features are extra columns the learned LogisticRegression

```

```

weights,
7     so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
8
9     Run (no GPU):
10    python -m backend.eval.fusion_compare --features-dir output --record
11    """
12    from __future__ import annotations
13
14    import argparse
15    import glob
16    import json
17    import os
18    from typing import Any, Dict, List
19
20    from backend.eval import experiment
21    from backend.eval.eval_stats import paired_delta_ci
22    from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
23
24
25    def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
26        """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
test
27        (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
28        by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
videos."""
29        by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
30        by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
31        vids = [p["video"] for p in res["variant_a"]["per_video"]]
32        return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
33
34
35    def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
36        """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
37        feature columns + golden gts). The harness derives labels from gts itself."""
38        videos: List[experiment.VideoCandidates] = []
39        for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
40            with open(path, encoding="utf-8") as f:
41                d = json.load(f)
42                cands = d["candidates"]
43                intervals = [(float(c["start"]), float(c["end"])) for c in cands]
44                features: Dict[str, List[float]] = {}
45                for fn in SHUTTLE_FEATURE_NAMES:
46                    features[fn] = [float(c["shuttle"][fn]) for c in cands]
47                for fn in PERSON_FEATURE_NAMES:
48                    features[fn] = [float(c["person"][fn]) for c in cands]
49                gts = [(float(a), float(b)) for a, b in d["gts"]]
50                videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
51                                                            features=features, gts=gts))
52        return videos
53
54
55    def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
56                threshold: float = 0.5) -> Dict[str, Any]:
57        shuttle_only = experiment.Variant(name="shuttle_only",
feature_names=list(SHUTTLE_FEATURE_NAMES),
58                threshold=threshold)
59        fusion = experiment.Variant(name="shuttle_person_fusion",
60                feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES),
61                threshold=threshold)
62        return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)
63
64
65    def format_result(res: Dict[str, Any], iou: float) -> str:

```

```

66         a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
67         ds = delta_stats(res)
68         st = ds["sign_test"]
69         lines = [
70             f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest)
===",
71             f"  shuttle-only : F1={a['f1']:.3f}  P={a['precision']:.3f}  R={a['recall']:.3f}
IoU={a['mean_iou']:.3f}",
72             f"  +person      : F1={b['f1']:.3f}  P={b['precision']:.3f}  R={b['recall']:.3f}
IoU={b['mean_iou']:.3f}",
73             f"  ?F1 = {d['f1']:.3f}  ({res['delta_pct']['f1']:.1f}%)
?P={d['precision']:.3f}  ?R={d['recall']:.3f}",
74             f"  paired ?F1 {ds['mean_delta']:.3f}  [95% CI {ds['ci_low']:.3f},
{ds['ci_high']:.3f}]  "
75             f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
76             f"| CI clears 0: {ds['ci_excludes_zero']}",
77             "  per-video ?F1:",
78         ]
79         for pv in res["per_video_delta"]:
80             lines.append(f"    {pv['video']:<26s} ?F1={pv['f1']:.3f}")
81         return "\n".join(lines)
82
83
84     def main() -> None:
85         ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs
shuttle+person (golden set).")
86         ap.add_argument("--features-dir", default="output")
87         ap.add_argument("--iou", type=float, default=0.5)
88         ap.add_argument("--record", action="store_true", help="append the result to the
experiment leaderboard")
89         args = ap.parse_args()
90
91         videos = load_videos(args.features_dir)
92         if len(videos) < 2:
93             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
{len(videos)})")
94         res = compare(videos, iou=args.iou)
95         print(format_result(res, args.iou))
96         if args.record:
97             row = experiment.record_experiment(res)
98             print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']}")
99
100
101     if __name__ == "__main__":
102         main()
103

```

28. user (2026-06-14T10:51:11.410Z)

```

1      """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3      Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4      candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5      EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6      writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7      CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8      measured by the shipped experiment harness; person features enter only as feature
columns the
9      learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10

```

```

11     Run (on the GPU box):
12     python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14     Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15     `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16     """
17     from __future__ import annotations
18
19     import argparse
20     import json
21     import os
22     import time
23     from typing import Any, Dict, List, Optional
24
25     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26     from backend.pipeline.detectors import rally_gate
27     from backend.pipeline.detectors import fusion_features as ff
28     from backend.eval import distill_local
29     from backend.eval.calibrate_wasb import load_golden_gts
30     from backend.eval.training import label_candidates
31     from backend.utils.run_telemetry import RunTelemetry
32
33     # The 5 fps/resolution-invariant shuttle features distill_local already computes.
34     SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
35     # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
36     PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
37
38
39     def _derive_video_path(spec: Dict[str, Any]) -> str:
40         """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
41         golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
convention)."""
42         if spec.get("video_path"):
43             return str(spec["video_path"])
44         labels = str(spec["labels"])
45         if labels.endswith(".rallies.csv"):
46             return labels[: -len(".rallies.csv")] + ".MP4"
47         raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't
*.rallies.csv")
48
49
50     def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
51                     frame_skip: int = 3, iou: float = 0.5,
52                     log_every_sec: float = 15.0,
53                     telemetry: Optional[Any] = None) -> Dict[str, Any]:
54         """One golden video -> a feature record (shuttle + person features per candidate
window).
55
56         Returns `{name, fps, frame_width,
candidates: [{start, end, shuttle {}, person {}, label}], gts}`.
57         `detector` (anything with `detections_over_windows`) is injectable for GPU-free
tests;
58         when None a real `PersonDetector` is built (loads torchvision on first use).
59         `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
snapshots
60         from the detection progress callback so a hard kill leaves a forensic runlog.
61         """
62         fps, fw = float(spec["fps"]), float(spec["frame_width"])
63         traj = str(spec["trajectory"])
64         points = TrackNetRunner.parse_trajectory_csv(traj)
65         intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
66         gts = load_golden_gts(str(spec["labels"]))

```

```

67     labels = label_candidates(intervals, gts, iou)
68     # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
estimate
69     # (parity with the shuttle net-crossing gate) rather than hard-coding an
orientation.
70     axis, net_px, _span = rally_gate.estimate_play_axis(points)
71     video_path = _derive_video_path(spec)
72
73     if detector is None:
74         from backend.pipeline.detectors.person_detector import PersonDetector
75         detector = PersonDetector({"indexing": {"use_gpu": True}})
76
77     # Single-pass person detection over ALL candidate windows (open the video ONCE, read
78     # forward ? no per-window seek). Then compute features per window (pure, no GPU).
79     n = len(intervals)
80     win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
81     print(f"[fusion_golden]   {spec['name']}: {n} windows ? person-detecting in one
pass...", flush=True)
82     t0 = time.monotonic()
83     state = {"last": t0}
84
85     def _progress(done: int, total: int) -> None:
86         if telemetry is not None:
87             telemetry.snapshot(done, total, phase=spec["name"])
88             now = time.monotonic()
89             if now - state["last"] >= log_every_sec or done >= total:
90                 el = now - t0
91                 rate = done / el if el > 0 else 0.0
92                 eta = (total - done) / rate if rate > 0 else 0.0
93                 pct = 100 * done // total if total else 0
94                 print(f"[fusion_golden]   {spec['name']}: detect frame {done}/{total}
({pct}%) "
95                     f"| {el:.0f}s elapsed | ETA {eta:.0f}s", flush=True)
96                 state["last"] = now
97
98     per_window = detector.detections_over_windows(video_path, win_frames, frame_skip,
99                                                    progress_cb=_progress)
100
101     candidates: List[Dict[str, Any]] = []
102     for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
103         det = per_window[i]
104         det_fw = det.get("frame_width") or fw
105         det_fh = det.get("frame_height") or 0.0
106         # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
107         # normalisation fusion_features uses (x/width, y/height).
108         if axis == "x":
109             net_norm = net_px / det_fw if det_fw > 0 else 0.5
110         else:
111             net_norm = net_px / det_fh if det_fh > 0 else 0.5
112         sf, ef = win_frames[i]
113         shuttle_in = [p for p in points if sf <= p.frame <= ef]
114         person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
115                                             court_polygon=None, net=(axis, net_norm))
116         candidates.append({
117             "start": float(s), "end": float(e),
118             "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
119             "person": {k: float(v) for k, v in person.items()},
120             "label": int(labels[i]),
121         })
122     return {
123         "name": spec["name"], "fps": fps, "frame_width": fw,
124         "candidates": candidates,
125         "gts": [[float(a), float(b)] for a, b in gts],
126     }

```

```

127
128
129     def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
130             fresh: bool = False, smallest_first: bool = False) -> List[str]:
131         with open(manifest_path, encoding="utf-8") as f:
132             specs = json.load(f)
133             if smallest_first:
134                 # Process small videos first for quick feedback (the slow full match lands
135                 # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
136                 # row/frame).
137                 specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
138                                                         if os.path.isfile(s.get("trajectory", ""))
139                                                         else 0))
140             os.makedirs(out_dir, exist_ok=True)
141             # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
142             # keep a tiny
143             # rotating runlog of velocity + machine health for the post-mortem. Constructing it
144             # must never
145             # break extraction, so guard the build and fall back to no telemetry on any error.
146             telem: Optional[RunTelemetry]
147             try:
148                 telem = RunTelemetry(os.path.join(out_dir, "run_telemetry.jsonl"),
149                                     label="fusion_golden")
150             except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
151                 # observes
152                 telem = None
153             written: List[str] = []
154             for vi, spec in enumerate(specs, 1):
155                 out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
156                 # Resumable: skip a video whose JSON already exists (a re-run picks up where a
157                 # prior
158                 # run left off instead of re-detecting). --fresh forces a redo.
159                 if not fresh and os.path.isfile(out):
160                     print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already
161                     extracted "
162                     f"-> {out} (use --fresh to redo)", flush=True)
163                     written.append(out)
164                     continue
165                 print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
166                       flush=True)
167                 t0 = time.monotonic()
168                 if telem is not None:
169                     telem.event("video_start", spec["name"])
170                 try:
171                     data = extract_video(spec, frame_skip=frame_skip, iou=iou, telemetry=telem)
172                 except Exception as e: # noqa: BLE001 - record machine state at death, then re-
173                     raise
174                     if telem is not None:
175                         telem.event("error", spec["name"], err=str(e))
176                     raise
177                 n_pos = sum(c["label"] for c in data["candidates"])
178                 with open(out, "w", encoding="utf-8") as f:
179                     json.dump(data, f, indent=2)
180                 if telem is not None:
181                     telem.event("video_done", spec["name"],
182                               windows=len(data["candidates"]), positives=n_pos)
183                 print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
184                 {time.monotonic() - t0:.0f}s "
185                 f"| {len(data['candidates'])} windows ({n_pos} positive) |
186                 {len(data['gts'])} golden rallies "
187                 f"-> {out}", flush=True)
188                 written.append(out)
189             return written
190
191

```

```

179
180     def main() -> None:
181         ap = argparse.ArgumentParser(description="Extract golden-set shuttle+person features
(fusion P3 GPU step).")
182         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
183         ap.add_argument("--out-dir", default="output")
184         ap.add_argument("--frame-skip", type=int, default=3)
185         ap.add_argument("--iou", type=float, default=0.5)
186         ap.add_argument("--fresh", action="store_true", help="re-extract even if a JSON
already exists")
187         ap.add_argument("--smallest-first", action="store_true",
188             help="process smaller videos first for quick feedback (LOVO is
order-independent)")
189         args = ap.parse_args()
190         run(args.manifest, args.out_dir, args.frame_skip, args.iou, fresh=args.fresh,
191             smallest_first=args.smallest_first)
192
193
194     if __name__ == "__main__":
195         main()
196

```

29. assistant (2026-06-14T10:51:17.744Z)

Now I understand the full picture. The key insight:

- `fusion\_golden.py` calls `distill\_local.build\_candidates(traj, fps, fw)` which uses the **PROPOSAL** preset (vt=0.005/gap=1.0/min=0.5) ? line 65, no `proposal=` override.
- The golden feature JSONs (`output/\*\_golden\_features.json`) are therefore built on the **proposal** windowing.
- `fusion\_compare.py` and the Gate-A ablation re-score those proposal candidates ? the +0.074 Gate-A win.
- But production (`trajectory\_hybrid.process\_video`) uses the **SERVED** windowing (vt=0.02/gap=2.0/min=2.0).

Now let me look at the config models/loader to see where the served windowing constants live, and eval_stats and the ablation runner.

30. user (2026-06-14T10:51:18.338Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280

```

```

26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33     class TrackNetConfig(BaseModel):
34         wsl_distro: str = "Ubuntu"
35         repo_dir: str = "~/models/TrackNetV4"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "TrackNetV4"
38         weights_path: str = ""
39         queue_length: int = 5
40         stage_in_wsl: bool = True
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43         velocity_threshold: float = 0.02
44         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45         # decoded frame cache) are deleted automatically ? they are pure intermediates,
46         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48         keep_frames: bool = False
49         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50         # 0 = disabled.
51         min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"
65         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66         sport: str = "badminton"
67         wsl_stage_dir: str = "~/clips"
68         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69         velocity_threshold: float = 0.02
70         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
71         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
72         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
73         keep_frames: bool = False
74         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75         min_free_gb: float = 0.0
76
77
78     class FusionConfig(BaseModel):
79         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
80
81         **Gate A (the held-shuttle guard) is ON by default** (`min_crossings=2`): a window
82         must show the shuttle crossing the net >= 2 times to reach the AI handoff, dropping
the
83         held-shuttle / one-sided-feed / warm-up false positives. This is the promoted,

```

```

validated
84         operating point (?F1 +0.074, 6/6 golden, sign-test p=0.031, CI [+0.042,+0.105];
over-seg
85         1.68?1.01 ? see docs/QUALITY_ITERATIONS.md). It only takes effect when the served
segmenter
86         is ``fusion_hybrid`` (config.json ``indexing.default_segmenter``); the offline
experiment
87         harness is unaffected (it sets ``Variant.min_crossings`` explicitly, never reading
this).
88
89         **Reversible ? rollback is a single flag:** set ``indexing.fusion.min_crossings: 0``
to
90         restore the pre-Gate-A (ungated) handoff, or ``indexing.default_segmenter:
wasb_hybrid``
91         to revert the served segmenter entirely. With ``min_crossings=0`` the fusion
segmenter
92         reproduces the substrate (wasb_hybrid) byte-for-byte.
93
94         The person cue (Gate B) stays OFF by default: the fusion segmenter persists the
shuttle
95         net-crossing count always, and ? only when ``cue_flags['person']`` is true ? the
person-cue
96         features; ``min_players`` (Gate B) defaults to 0 = OFF. The court mask is optional:
97         ``court_polygon=None`` ? Gate B counts every detection (player-count-only); supplied
?
98         off-court persons (spectators / adjacent courts) are excluded. **The persisted
candidate
99         set is the full superset regardless of any gate** ? gating only filters the AI
handoff, so
100        the offline harness can re-score any variant.
101        """
102        # Which trajectory substrate seeds the windows (shuttle stays primary).
103        substrate: Literal["wasb", "tracknet"] = "wasb"
104        # Windowing velocity threshold for the fusion family (the engine reads
105        # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
106        # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
107        velocity_threshold: float = 0.02
108        # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
109        # enabled ? so the default path never imports torch / touches the GPU.
110        cue_flags: dict[str, bool] = Field(default_factory=dict)
111        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
112        # Default 2 = the PROMOTED held-shuttle guard (a real rally crosses the net >=2x; a
single
113        # service feed / held shuttle crosses ~once). Set 0 to turn the gate OFF (ungated
handoff;
114        # reproduces the substrate exactly). Persisted candidates are ALWAYS the full
superset for
115        # offline re-scoring, regardless of this threshold.
116        min_crossings: int = Field(default=2, ge=0)
117        # Served Gate B: when the person cue is on, drop windows with median in-court
players
118        # < this. 0 = OFF.
119        min_players: int = Field(default=0, ge=0)
120        # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
121        court_polygon: Optional[list[list[float]]] = None
122        # Net line for the person two-sided-motion split (person features only ? the shuttle
123        # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
124        net_axis: Literal["x", "y"] = "y"
125        net_position: float = Field(default=0.5, ge=0.0, le=1.0)
126
127
128        class IndexingConfig(BaseModel):
129            default_segmenter: str = "yolo_hybrid"
130            use_gpu: bool = True

```

```

131     motion_threshold: float = 0.08
132     min_segment_duration: float = 3.0
133     dead_time_merge_gap: float = 2.0
134     chunk_duration: float = 90.0
135     chunk_overlap: float = 10.0
136     detector_model: str = "fasterrcnn_resnet50_fpn"
137     detector_score_threshold: float = 0.5
138     yolo_velocity_threshold: float = 0.15
139     yolo_frame_skip: int = 3
140     yolo_tracker: str = "bytetrack.yaml"
141     yolo_prefetch_workers: int = 2
142     min_action_window_duration: float = 2.0
143     log_yolo_candidates: bool = True
144     store_yolo_candidates: bool = True
145     ai_handoff_padding: float = 2.0
146     ai_max_workers: int = 5
147     # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
148     # chunks of a high-bitrate (4K) source blow past the provider upload cap
149     # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
150     # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
151     # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
152     # Matches gemini_refine's --clip-scale-width default.
153     ai_chunk_scale_width: int = 1280
154     # Optional debug knob: trim every video to the first N seconds before processing.
155     debug_duration: Optional[float] = None
156
157     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
158     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
159     fusion: FusionConfig = Field(default_factory=FusionConfig)
160
161     @field_validator("default_segmenter")
162     @classmethod
163     def segmenter_must_be_registered(cls, v: str) -> str:
164         # Registry check happens at runtime in main/segmenter selection.
165         # Here we just allow any string for forward compatibility.
166         return v
167
168     @model_validator(mode="after")
169     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
170         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
171         # step makes `while t < duration: t += step` loop forever, growing memory before
any
172         # GPU work. Reject the bad config at load time instead.
173         if self.chunk_overlap >= self.chunk_duration:
174             raise ValueError(
175                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
176                 f"({self.chunk_duration}); otherwise chunking never advances."
177             )
178         return self
179
180
181     class OutputConfig(BaseModel):
182         default_highlights_name: str = "highlights.mp4"
183         db_name: str = "sports_indexer.db"
184         # Directory where the DB, highlight reels, and processed clips are written.
185         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
186         output_dir: str = "output"
187
188
189     class WatermarkConfig(BaseModel):
190         """Branding overlay burned into each highlight clip during the per-clip re-encode
191         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
192         (under `stitching.watermark` in config.json) to turn it off. The text is fully

```

```

193     configurable. The concat stage stays stream-copy (-c copy)."""
194     enabled: bool = True
195     text: str = "Generated by Khelsutra.guru"
196     logo_path: str = ""          # optional transparent PNG overlay; "" = no logo
197     font_path: str = ""          # optional .ttf; "" = use the fontconfig `font` family
below
198     font: str = "Sans"           # fontconfig family used when font_path is empty
199     font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
fraction of frame height
200     opacity: float = Field(default=0.85, ge=0.0, le=1.0)
201     box: bool = True             # faint backing box behind the text for legibility
202     margin: int = Field(default=24, ge=0) # px inset from the chosen corner
203     position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
204
205
206     class StitchingConfig(BaseModel):
207         begin_padding: float = 0.5
208         end_padding: float = 0.5
209         use_cache: bool = False
210         min_shot_count: int = 1
211         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
212
213
214     class LoggingConfig(BaseModel):
215         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
216         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
217         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
218         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
219         # them to WARNING; set true to restore full chatter for request-flow debugging.
220         verbose_http: bool = False
221
222
223     class SecurityConfig(BaseModel):
224         # When set, /api/process video_path must resolve under this directory (hosted/tenant
225         # mode). Default None preserves the single-user local 'any local file' workflow.
226         allowed_video_root: Optional[str] = None
227
228         # --- Chunked upload (B2, PR-2) ---
229         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
230         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
231         # contain upload_dir or /api/process will reject the uploaded paths.
232         upload_dir: str = "output/uploads"
233         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
234         # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
235         max_upload_mb: int = 4096
236         max_part_mb: int = 95
237         allowed_upload_extensions: List[str] = ["mp4", "mov"]
238
239
240     class AppConfig(BaseModel):
241         """Root configuration object.
242
243         All runtime configuration should be accessed via an instance of this class
244         (or the loader) rather than raw dict lookups.
245         """
246
247         sport: Sport = "badminton"
248         ai_provider: str = "gemini"
249         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
250         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
251         ai_model: str = "gemini-flash-latest"
252

```

```

253 validation: ValidationConfig = Field(default_factory=ValidationConfig)
254 indexing: IndexingConfig = Field(default_factory=IndexingConfig)
255 output: OutputConfig = Field(default_factory=OutputConfig)
256 stitching: StitchingConfig = Field(default_factory=StitchingConfig)
257 security: SecurityConfig = Field(default_factory=SecurityConfig)
258 logging: LoggingConfig = Field(default_factory=LoggingConfig)
259
260 # Allow extra keys during transition so existing config.json files do not break.
261 model_config = {
262     "extra": "allow",
263     "validate_assignment": True,
264 }
265
266 # Temporary migration shim for legacy dict-style access (config.get("section",
267 {}).get(...)).
268 # Will be removed once all call sites (validators, segmenters, etc.) move to typed
269 # attribute access / the typed config. A nested *section* is returned as a plain
dict
269 # via model_dump, so legacy chained access keeps working ? returning the Pydantic
270 # section model instead breaks it (a model has no .get), which bit every validator
on
271 # real runs. Scalar leaves are returned as-is.
272 def get(self, key: str, default: Any = None) -> Any:
273     _MISSING = object()
274     val: Any = _MISSING
275     if key in type(self).model_fields or hasattr(self, key):
276         val = getattr(self, key, _MISSING)
277     if val is _MISSING:
278         # Legacy convenience: find a leaf key inside any nested section. Sections
are
279         # discovered dynamically from the model fields, so a future top-level
section
280         # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
281         for fname in type(self).model_fields:
282             section = getattr(self, fname, None)
283             if isinstance(section, BaseModel) and hasattr(section, key):
284                 val = getattr(section, key)
285                 break
286         if val is _MISSING:
287             return default
288         if isinstance(val, BaseModel):
289             return val.model_dump(mode="json")
290         return val
291
292 def __getitem__(self, key: str) -> Any:
293     val = self.get(key)
294     if val is None and key not in type(self).model_fields:
295         # Re-raise like a real dict for unknown top-level
296         raise KeyError(key)
297     return val
298
299
300 # Convenience type alias for callers that want to annotate
301 Config = AppConfig
302

```

31. assistant (2026-06-14T10:51:22.324Z)

Now I have the complete picture. The served windowing constants are: `velocity\_threshold=0.02` (in `IndexingConfig.wasb`/`fusion`/`TrackNetConfig`), `dead\_time\_merge\_gap=2.0` (line 133), `min\_action\_window\_duration=2.0` (line 142). These are the served defaults.

Let me read eval_stats and the ablation runner to understand the full promotion path, then read the gate_ab_ablation scratch script.

32. user (2026-06-14T10:51:23.022Z)

```
1      """Honest small-N evaluation statistics (course-corrections C1 + C3).
2
3      The ~6-match golden corpus + a regularised logistic arbiter is exactly the regime where
4      a
5      single leave-one-video-out *mean* lies: leave-one-out anti-correlates each fold's
6      training
7      and held-out label means at small N, and one lucky/unlucky fold swings the headline. The
8      ``compare``/LOVO machinery already produces **per-video held-out scores** ? this module
9      turns those into an honest summary:
10
11      - **C1 ? never a single mean.** ``mean_with_ci`` / ``bootstrap_ci`` report a confidence
12      interval; ``paired_sign_test`` and ``paired_delta_ci`` grade a B?A improvement by how
13      many *videos* it wins on (the distribution-free test the repo already leans on:
14      "sign test needs n ? ~n=10/9-wins for p<0.05"), so the promotion gate can't promote
15      noise.
16
17      - **C3 ? honest stacking.** ``out_of_fold_predictions`` / ``grouped_folds`` give a
18      learned
19      producer's *out-of-fold* outputs so feeding them to the arbiter doesn't leak (group by
20      match, never a held-out window from the same video).
21
22      See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C1+C3. All **additive**, pure (stdlib
23      only),
24      deterministic (bootstrap takes an explicit ``seed``) ? nothing here changes existing
25      ``compare``/``calibration.leave_one_video_out`` behaviour; callers opt in.
26      """
27      from __future__ import annotations
28
29      import math
30      import random
31      import statistics
32      from typing import Any, Callable, Dict, List, Optional, Sequence, Tuple
33
34      Interval = Tuple[float, float]
35      FitPredict = Callable[[List[int], List[int]], List[Any]]
36
37      def bootstrap_ci(values: Sequence[float], confidence: float = 0.95,
38                      n_boot: int = 2000, seed: int = 0) -> Tuple[float, float]:
39          """Percentile bootstrap CI for the mean of ``values``. Deterministic given ``seed``.
40
41          ``n<=1`` returns a degenerate ``(v, v)`` / ``(0, 0)`` interval ? there is no spread
42          to
43          resample. With n=6 (our corpus) this is the honest "how wide could the mean be?"
44          band.
45
46          """
47          vals = [float(v) for v in values]
48          n = len(vals)
49          if n == 0:
50              return (0.0, 0.0)
51          if n == 1:
52              return (vals[0], vals[0])
53          rng = random.Random(seed)
54          means: List[float] = []
55          for _ in range(max(1, n_boot)):
56              means.append(sum(vals[rng.randrange(n)] for _ in range(n)) / n)
57          means.sort()
58          alpha = (1.0 - confidence) / 2.0
59          last = len(means) - 1
60          lo = means[max(0, int(math.floor(alpha * last)))]
61          hi = means[min(last, int(math.ceil((1.0 - alpha) * last)))]
62          return (lo, hi)
```

```

57     def mean_with_ci(values: Sequence[float], confidence: float = 0.95,
58                       n_boot: int = 2000, seed: int = 0) -> Dict[str, float]:
59         """`{mean, std, n, ci_low, ci_high, confidence}` ? report this, never a bare mean
(C1)."""
60         vals = [float(v) for v in values]
61         n = len(vals)
62         mean = statistics.mean(vals) if n else 0.0
63         std = statistics.pstdev(vals) if n > 1 else 0.0
64         lo, hi = bootstrap_ci(vals, confidence, n_boot, seed)
65         return {"mean": mean, "std": std, "n": float(n),
66                "ci_low": lo, "ci_high": hi, "confidence": confidence}
67
68
69     def paired_sign_test(deltas: Sequence[float], eps: float = 0.0) -> Dict[str, float]:
70         """Two-sided exact sign test over per-fold deltas (B ? A).
71
72         ``wins`` = folds where B beat A by more than ``eps``; ``losses`` the reverse; ties
73         excluded. ``p_value`` is the exact two-sided binomial tail under p=0.5 ? the
74         distribution-free "did B win on enough *videos*?" check.
75         """
76         wins = sum(1 for d in deltas if d > eps)
77         losses = sum(1 for d in deltas if d < -eps)
78         ties = len(deltas) - wins - losses
79         n_eff = wins + losses
80         if n_eff == 0:
81             p = 1.0
82         else:
83             k = min(wins, losses)
84             tail = sum(math.comb(n_eff, j) for j in range(k + 1)) * (0.5 ** n_eff)
85             p = min(1.0, 2.0 * tail)
86         return {"wins": float(wins), "losses": float(losses), "ties": float(ties),
87                "n_effective": float(n_eff), "p_value": p}
88
89
90     def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
91                        confidence: float = 0.95, n_boot: int = 2000, seed: int = 0,
92                        eps: float = 0.0) -> Dict[str, Any]:
93         """Paired (by fold) B ? A summary: mean delta + bootstrap CI + the sign test (C1).
94
95         ``a_values``/``b_values`` are per-fold held-out scores for variants A and B, aligned
96         by
97         fold (same video order). A promotion is honest when the delta CI clears 0 *and* the
98         sign
99         test agrees ? not when a single mean ticked up.
100         """
101         n = min(len(a_values), len(b_values))
102         deltas = [float(b_values[i]) - float(a_values[i]) for i in range(n)]
103         summary = mean_with_ci(deltas, confidence, n_boot, seed)
104         return {
105             "mean_delta": summary["mean"],
106             "ci_low": summary["ci_low"],
107             "ci_high": summary["ci_high"],
108             "n": float(n),
109             "ci_excludes_zero": (summary["ci_low"] > 0.0) or (summary["ci_high"] < 0.0),
110             "sign_test": paired_sign_test(deltas, eps),
111         }
112
113     def grouped_folds(groups: Sequence[Any]) -> List[Tuple[List[int], List[int]]]:
114         """Leave-one-group-out splits ``(train_idx, test_idx)`` ? group by match, the honest
115         unit.
116
117         The field's name for "LOVO by match" is leave-one-group-out (LOGO). Group order is
118         first-seen (deterministic). Groups with no complement (a single group) yield no
119         fold.

```

```

117     """
118     seen: List[Any] = []
119     for g in groups:
120         if g not in seen:
121             seen.append(g)
122     n = len(groups)
123     folds: List[Tuple[List[int], List[int]]] = []
124     for g in seen:
125         test = [i for i in range(n) if groups[i] == g]
126         train = [i for i in range(n) if groups[i] != g]
127         if test and train:
128             folds.append((train, test))
129     return folds
130
131
132     def out_of_fold_predictions(groups: Sequence[Any], fit_predict: FitPredict) ->
List[Optional[Any]]:
133         """Out-of-fold predictions for honest stacking (C3).
134
135         For each leave-one-group-out fold, ``fit_predict(train_idx, test_idx)`` trains on
the
136         other groups and returns predictions for ``test_idx`` (in that order); results are
scattered back to a list aligned to the input items. So a *learned* producer's
137         outputs,
138         when used as arbiter features, never include in-sample predictions ? the leakage
path
139         that inflates measured lift on a tiny corpus. Items in a singleton group stay
``None``
140         (needs >= 2 groups to predict).
141         """
142         n = len(groups)
143         oof: List[Optional[Any]] = [None] * n
144         for train_idx, test_idx in grouped_folds(groups):
145             preds = fit_predict(train_idx, test_idx)
146             for pos, ti in enumerate(test_idx):
147                 if pos < len(preds):
148                     oof[ti] = preds[pos]
149         return oof
150

```

33. assistant (2026-06-14T10:51:25.739Z)

Now let me read the gate_ab_ablation scratch script and the ablation.py runner to understand the existing promotion path.

34. user (2026-06-14T10:51:26.378Z)

```

1     """Leave-one-signal-out ablation for the Gate-A/B foundational decision (CPU, offline).
2
3     Reuses ONLY the shipped harness (experiment.compare + eval_stats) over the 6 persisted
4     golden feature JSONs (output/*_golden_features.json). No new scoring math, no GPU, no
Gemini.
5
6     Two kinds of question, two mechanisms (mirrors the repo's own distinction):
7
8     1. LEARNED-FUSION marginal contribution (leave-one-signal-out): the "full" learned
variant
9         uses ALL 13 feature columns (shuttle 5 + person 5 + audio 3). For each signal GROUP
(and
10         each individual shuttle feature where sensible) we drop those columns and measure
held-out
11         ?F1 = full - leave_one_out, with bootstrap 95% CI + paired sign test. ?F1 > 0 means
the
12         signal carries marginal information the learned scorer used.
13

```

```

14     NOTE the sign convention: we report  $\Delta = \text{FULL} - \text{DROPPED}$ , so a POSITIVE  $\Delta$  means
the
15     signal HELPS (dropping it hurts). experiment.compare reports  $B - A$ , so we set
16     variant_a = leave_one_out (dropped), variant_b = full  $\rightarrow B - A = \text{full} - \text{dropped}$ . Good.
17
18     2. GATE marginal contribution (decision masks, NOT learned columns): Gate-A
(min_crossings>=2)
19     and Gate-B (min_players>=N) are re-scoring masks over the recall-oriented CONTROL
proposal
20     set. Measured as static-variant A/B: CONTROL vs CONTROL+gate. Reports F1, ?F1
(CI+sign),
21     and the segment-count / over-seg ratio (C4) that tIoU hides.
22
23     All numbers printed are ACTUAL measured values. Writes a JSON blob to scratch/ for the
doc.
24     """
25     from __future__ import annotations
26
27     import json
28     import os
29     from typing import Any, Dict, List, Sequence
30
31     from backend.eval import experiment
32     from backend.eval.eval_stats import paired_delta_ci
33     from backend.eval import segmentation_metrics as segm
34     from backend.eval.metrics import score
35     from backend.pipeline.detectors import audio_features as af
36     from backend.pipeline.detectors import fusion_features as ff
37     from backend.eval import distill_local
38
39     SHUTTLE = list(distill_local.FEATURE_NAMES)          # 5
40     PERSON = list(ff.PERSON_FEATURE_NAMES)              # 5
41     AUDIO = list(af.AUDIO_FEATURE_NAMES)                # 3
42     ALL_FEATURES = SHUTTLE + PERSON + AUDIO             # 13
43
44     FEATURES_DIR = "output"
45     IOU = 0.5
46     THRESHOLD = 0.5
47
48
49     def load_videos() -> List[experiment.VideoCandidates]:
50         import glob
51         videos: List[experiment.VideoCandidates] = []
52         for path in sorted(glob.glob(os.path.join(FEATURES_DIR, "*_golden_features.json"))):
53             with open(path, encoding="utf-8") as f:
54                 d = json.load(f)
55                 cands = d["candidates"]
56                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
57                 feats: Dict[str, List[float]] = {}
58                 for fn in SHUTTLE:
59                     feats[fn] = [float(c["shuttle"][fn]) for c in cands]
60                 for fn in PERSON:
61                     feats[fn] = [float(c["person"][fn]) for c in cands]
62                 for fn in AUDIO:
63                     feats[fn] = [float(c["audio"][fn]) for c in cands]
64                 gts = [(float(a), float(b)) for a, b in d["gts"]]
65                 videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
66                                                         features=feats, gts=gts))
67         return videos
68
69
70     def paired_from_compare(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
71         """Pair B-A per-video held-out scores  $\rightarrow$  mean  $\Delta$  + CI + sign test."""
72         by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
73         by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}

```

```

74     vids = [p["video"] for p in res["variant_a"]["per_video"]]
75     return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
76
77
78     def learned_lovo(videos, feature_names) -> Dict[str, Any]:
79         """Run a learned LOVO variant over `feature_names`; return aggregate + per-video
80         f1."""
81         v = experiment.Variant(name="v", feature_names=list(feature_names),
82         threshold=THRESHOLD)
83         ev = experiment.evaluate_variant(videos, v, iou=IOU, honest=True)
84         return ev
85
86     def leave_one_out_row(videos, full_features, drop, label) -> Dict[str, Any]:
87         """full (all features) as B; leave-one-out (full minus `drop`) as A.
88         delta = B - A = full - dropped -> positive means the dropped signal HELPS."""
89         kept = [f for f in full_features if f not in set(drop)]
90         a = experiment.Variant(name=f"drop_{label}", feature_names=kept,
91         threshold=THRESHOLD)
92         b = experiment.Variant(name="full", feature_names=list(full_features),
93         threshold=THRESHOLD)
94         res = experiment.compare(videos, a, b, iou=IOU, honest=True) # B-A = full - dropped
95         pd = paired_from_compare(res)
96         st = pd["sign_test"]
97         return {
98             "signal": label,
99             "dropped_cols": list(drop),
100             "f1_full": res["variant_b"]["aggregate"]["f1"],
101             "f1_dropped": res["variant_a"]["aggregate"]["f1"],
102             "marginal_dF1": pd["mean_delta"], # full - dropped
103             "ci_low": pd["ci_low"], "ci_high": pd["ci_high"],
104             "ci_excludes_zero": pd["ci_excludes_zero"],
105             "wins": int(st["wins"]), "losses": int(st["losses"]),
106             "p_value": st["p_value"],
107             "per_video_full": {p["video"]: p["f1"] for p in res["variant_b"]["per_video"]},
108             "per_video_dropped": {p["video"]: p["f1"] for p in
109             res["variant_a"]["per_video"]},
110         }
111
112     def static_predict_segcount(variant, videos) -> Dict[str, float]:
113         """Mean F1@k + segment_count_ratio for a static variant across videos."""
114         overlaps = segm.DEFAULT_OVERLAPS
115         f1k = {ov: [] for ov in overlaps}
116         ratios = []
117         for vc in videos:
118             preds = experiment.predict(variant, vc)
119             got = segm.f1_at_overlaps(preds, vc.gts or [], overlaps)
120             for ov in overlaps:
121                 f1k[ov].append(got[ov])
122             ratios.append(segm.segment_count_ratio(preds, vc.gts or []))
123         out = {f"f1@{int(ov*100)}": (sum(v)/len(v) if v else 0.0) for ov, v in f1k.items()}
124         out["segment_count_ratio"] = sum(ratios)/len(ratios) if ratios else 0.0
125         return out
126
127     def gate_ab(videos, variant_a, variant_b, label) -> Dict[str, Any]:
128         """Static A/B (no learned filter): CONTROL vs CONTROL+gate. Reports F1, ?F1
129         (CI+sign),
130         and over-seg ratio for both."""
131         res = experiment.compare(videos, variant_a, variant_b, iou=IOU, honest=True)
132         pd = paired_from_compare(res)
133         st = pd["sign_test"]
134         seg_a = static_predict_segcount(variant_a, videos)
135         seg_b = static_predict_segcount(variant_b, videos)

```

```

133 # count of predicted segments per variant (for over-seg narrative)
134 def total_preds(v):
135     return sum(len(experiment.predict(v, vc)) for vc in videos)
136 return {
137     "label": label,
138     "f1_a": res["variant_a"]["aggregate"]["f1"],
139     "f1_b": res["variant_b"]["aggregate"]["f1"],
140     "p_a": res["variant_a"]["aggregate"]["precision"],
141     "p_b": res["variant_b"]["aggregate"]["precision"],
142     "r_a": res["variant_a"]["aggregate"]["recall"],
143     "r_b": res["variant_b"]["aggregate"]["recall"],
144     "dF1": pd["mean_delta"], "ci_low": pd["ci_low"], "ci_high": pd["ci_high"],
145     "ci_excludes_zero": pd["ci_excludes_zero"],
146     "wins": int(st["wins"]), "losses": int(st["losses"]), "p_value": st["p_value"],
147     "segratio_a": seg_a["segment_count_ratio"], "segratio_b":
seg_b["segment_count_ratio"],
148     "n_pred_a": total_preds(variant_a), "n_pred_b": total_preds(variant_b),
149     "per_video_dF1": {p["video"]: p["f1"] for p in res["per_video_delta"]},
150     "f1k_a": seg_a, "f1k_b": seg_b,
151 }
152
153
154 def main() -> None:
155     videos = load_videos()
156     print(f"Loaded {len(videos)} golden videos: {[v.name for v in videos]}")
157     result: Dict[str, Any] = {"n_videos": len(videos), "iou": IOU, "threshold":
THRESHOLD}
158
159     # ---- Full learned variant (all 13 cols) reference ----
160     full_ev = learned_lovo(videos, ALL_FEATURES)
161     result["full_learned"] = {
162         "feature_names": ALL_FEATURES,
163         "aggregate": full_ev["aggregate"],
164         "per_video": {p["video"]: p["f1"] for p in full_ev["per_video"]},
165     }
166     print("\n=== FULL learned LOVO (all 13 feature columns) ===")
167     print(f"  F1={full_ev['aggregate']['f1']:.3f} "
P={full_ev['aggregate']['precision']:.3f} "
168         f"R={full_ev['aggregate']['recall']:.3f}")
169
170     # ---- Leave-one-SIGNAL-GROUP-out ----
171     print("\n=== Leave-one-signal-out marginal contribution (delta = full - dropped)
===")
172     groups = [
173         ("person", PERSON),
174         ("audio", AUDIO),
175         ("net_crossings(shuttle-as-feature)", ["net_crossings"]),
176         ("shuttle_velocity_block", ["peak_velocity", "mean_velocity", "active_ratio"]),
177         ("duration", ["duration"]),
178     ]
179     rows = []
180     for label, drop in groups:
181         row = leave_one_out_row(videos, ALL_FEATURES, drop, label)
182         rows.append(row)
183         print(f"  drop {label:<36s} ?F1(full-drop)={row['marginal_dF1']:+.3f} "
184             f"CI[{row['ci_low']:+.3f},{row['ci_high']:+.3f}] "
185             f"{row['wins']}W/{row['losses']}L p={row['p_value']:.3f} "
186             f"clears0={row['ci_excludes_zero']}")
187     result["leave_one_out"] = rows
188
189     # ---- Also: incremental ADD-each-group-onto-shuttle (matches P3/P4 framing) ----
190     print("\n=== Incremental add-onto-shuttle (B - A) ===")
191     add_rows = []
192     incr = [
193         ("shuttle->+person", SHUTTLE, SHUTTLE + PERSON),

```

```

194         ("shuttle->+audio", SHUTTLE, SHUTTLE + AUDIO),
195         ("shuttle+person->+audio", SHUTTLE + PERSON, SHUTTLE + PERSON + AUDIO),
196         ("shuttle->+person+audio", SHUTTLE, ALL_FEATURES),
197     ]
198     for label, a_f, b_f in incr:
199         a = experiment.Variant(name="a", feature_names=a_f, threshold=THRESHOLD)
200         b = experiment.Variant(name="b", feature_names=b_f, threshold=THRESHOLD)
201         res = experiment.compare(videos, a, b, iou=IOU, honest=True)
202         pd = paired_from_compare(res)
203         st = pd["sign_test"]
204         r = {"label": label, "f1_a": res["variant_a"]["aggregate"]["f1"],
205             "f1_b": res["variant_b"]["aggregate"]["f1"], "dF1": pd["mean_delta"],
206             "ci_low": pd["ci_low"], "ci_high": pd["ci_high"],
207             "ci_excludes_zero": pd["ci_excludes_zero"],
208             "wins": int(st["wins"]), "losses": int(st["losses"]), "p_value":
209             st["p_value"]}
210         add_rows.append(r)
211         print(f"    {label:<28s} F1 {r['f1_a']:.3f}->{r['f1_b']:.3f} ?F1={r['dF1']:+.3f}
212             "
213             f"CI[{r['ci_low']:+.3f},{r['ci_high']:+.3f}] {r['wins']}W/{r['losses']}L
214             "
215             f"p={r['p_value']:.3f} clears0={r['ci_excludes_zero']}")
216     result["incremental_add"] = add_rows
217
218     # ---- GATE-A ON vs OFF (static, the foundational decision) ----
219     print("\n=== Gate-A ON vs OFF (static CONTROL vs net_crossings>=2) ===")
220     control = experiment.Variant(name="control") # merge-all, no gate
221     gate_a = experiment.Variant(name="gate_a", min_crossings=2)
222     ga = gate_ab(videos, control, gate_a, "Gate-A (min_crossings>=2)")
223     result["gate_a"] = ga
224     print(f"    CONTROL F1={ga['f1_a']:.3f} (P={ga['p_a']:.3f} R={ga['r_a']:.3f}) "
225           f"segratio={ga['segratio_a']:.2f} n_pred={ga['n_pred_a']}")
226     print(f"    Gate-A F1={ga['f1_b']:.3f} (P={ga['p_b']:.3f} R={ga['r_b']:.3f}) "
227           f"segratio={ga['segratio_b']:.2f} n_pred={ga['n_pred_b']}")
228     print(f"    ?F1={ga['dF1']:+.3f} CI[{ga['ci_low']:+.3f},{ga['ci_high']:+.3f}] "
229           f"{ga['wins']}W/{ga['losses']}L p={ga['p_value']:.3f}
230     clears0={ga['ci_excludes_zero']}")
231
232     # ---- GATE-B ON vs OFF (static; min_players sweep). No court polygons -> caveat.
233     ----
234     print("\n=== Gate-B ON vs OFF (static CONTROL vs players_in_court>=N) ===")
235     gb_rows = []
236     for n in (1, 2, 3, 4):
237         gb = experiment.Variant(name=f"gate_b{n}", min_players=n)
238         r = gate_ab(videos, control, gb, f"Gate-B (min_players>={n})")
239         gb_rows.append(r)
240         print(f"    N>={n}: CONTROL F1={r['f1_a']:.3f} -> Gate-B F1={r['f1_b']:.3f} "
241               f"?F1={r['dF1']:+.3f} CI[{r['ci_low']:+.3f},{r['ci_high']:+.3f}] "
242               f"{r['wins']}W/{r['losses']}L p={r['p_value']:.3f}
243     clears0={r['ci_excludes_zero']} "
244               f"n_pred {r['n_pred_a']}->{r['n_pred_b']}")
245     result["gate_b"] = gb_rows
246
247     # ---- Gate-A AND Gate-B stacked (does B add on top of A?) ----
248     print("\n=== Gate-A + Gate-B stacked vs Gate-A alone ===")
249     gab_rows = []
250     for n in (1, 2):
251         a = experiment.Variant(name="gate_a", min_crossings=2)
252         b = experiment.Variant(name=f"gate_ab{n}", min_crossings=2, min_players=n)
253         res = experiment.compare(videos, a, b, iou=IOU, honest=True)
254         pd = paired_from_compare(res)
255         st = pd["sign_test"]
256         r = {"n": n, "f1_a": res["variant_a"]["aggregate"]["f1"],
257             "f1_b": res["variant_b"]["aggregate"]["f1"], "dF1": pd["mean_delta"],
258             "ci_low": pd["ci_low"], "ci_high": pd["ci_high"],

```

```

253         "ci_excludes_zero": pd["ci_excludes_zero"],
254         "wins": int(st["wins"]), "losses": int(st["losses"]), "p_value":
st["p_value"]}
255         gab_rows.append(r)
256         print(f"    GateA + B(N>={n}) vs GateA: F1 {r['f1_a']:.3f}->{r['f1_b']:.3f}
?F1={r['dF1']:+.3f} "
257             f"CI[{r['ci_low']:+.3f},{r['ci_high']:+.3f}] {r['wins']}W/{r['losses']}L
p={r['p_value']:.3f}")
258         result["gate_ab_stacked"] = gab_rows
259
260         out = os.path.join("scratch", "gate_ab_ablation_result.json")
261         with open(out, "w", encoding="utf-8") as f:
262             json.dump(result, f, indent=2)
263         print(f"\nWrote {out}")
264
265
266     if __name__ == "__main__":
267         main()
268

```

35. assistant (2026-06-14T10:51:29.900Z)

Now I have a complete understanding. Let me check the ablation.py runner (the shipped one referenced in PR #147) and the trajectory_to_action_windows signature.

36. user (2026-06-14T10:51:31.265Z)

```

1     """Leave-one-signal-out / leave-one-group-out ablation runner (Phase 0).
2
3     A *systematic, repeatable* version of the one-off ablation
(``scratch/gate_ab_ablation.py``):
4     run it and it EXCLUDES a single signal, measures whether quality dropped, and reports ?
per
5     signal ? the **marginal contribution** ?F1 (full set vs full-minus-that-signal) with a
bootstrap
6     95% CI + paired sign test + the C4 over-segmentation ratio. See
``scratch/ABLATION_FRAMEWORK_PLAN.md``
7     (the design) and ``docs/ABLATION_LAYER.md`` (the shipped note).
8
9     **Nothing here is new scoring/stats math.** It is a *pure driver* over the shipped
10    ``experiment.compare`` + ``eval_stats`` engine, exactly as ``compare`` was a driver over
11    ``metrics``/``calibration``. For each signal ``s`` the marginal contribution is:
12
13    ?(s) = score(full set S) - score(S \ {s})          # leave-one-out drop, ?F1
primary
14
15    operationally ``experiment.compare(videos, variant_a=without_s, variant_b=full)`` ? A is
the
16    ablated (smaller) variant, B is full, so the reported B?A delta IS ?(s), a per-video-
paired ?F1
17    with bootstrap CI + exact sign test, LOVO-honest for learned variants.
18
19    Two ablation granularities fall out of one inventory:
20
21    - **Leave-one-signal-out (LOSO)** ? remove ONE column at a time (fine-grained).
22    - **Leave-one-GROUP-out (LOGO)** ? remove a whole producer's columns at once (the
owner's
23    "is this old SIGNAL still relevant?" question at cue granularity).
24
25    A signal can be a **feature column** (person/audio/shuttle columns the learned scorer
weights) or
26    a **gate knob** (Gate-A's ``min_crossings`` threshold ? ablated by zeroing the knob, NOT
by
27    dropping a column). The runner handles both uniformly: ``without_s`` subtracts the
columns AND/OR

```

```

28     zeroes the gate knob from ``cfg.full``.
29
30     Honest small-N contract (inherited from ``eval_stats`` / ``per_user_gate``; §6 of the
plan):
31     - never a bare mean ? every row ships a CI + the exact sign test;
32     - a signal earns its keep only when removal drops F1 with CI excluding 0 AND the
sign test
33     agreeing** (the ``per_user_gate`` bar, in reverse);
34     - structural signals are exempt from auto-demotion ? a guard whose value is in
failure modes
35     the golden corpus may lack (Gate-A vs held-shuttle FPs) is marked ``structural`` and
is NEVER
36     flagged dead weight on "no measured lift"; its row reports the (possibly ?0) golden
? honestly
37     but tags ``structural_protected``;
38     - the runner emits ``earns_keep`` / ``suggestive`` / ``inert`` /
``structural_protected`` ? it
39     MUST NOT print "irrelevant" or "proven useless" (one wash at n=6 is noise, not
proof).
40
41     Pure + offline + GPU-free. Reuses the persisted golden feature substrate
(``fusion_golden.py``
42     writes one ``<name>_golden_features.json`` per video). CLI::
43
44     python -m backend.eval.ablation --features-dir output [--pdf out.pdf]
45     """
46     from __future__ import annotations
47
48     import argparse
49     import glob
50     import json
51     import logging
52     import os
53     from dataclasses import dataclass, field
54     from typing import Any, Dict, List, Optional, Sequence, Tuple
55
56     from backend.eval import experiment
57     from backend.eval.experiment import Variant, VideoCandidates
58
59     logger = logging.getLogger(__name__)
60
61     # --- The signal inventory (the things the runner ablates), grouped by producer.
-----
62     # Source: distill_local.FEATURE_NAMES (shuttle, includes net_crossings), fusion_features
63     # (person), audio_features (audio). The shuttle group's velocity block + duration are
the
64     # learned-column shuttle signals; net_crossings is BOTH a column and the Gate-A gate
knob.
65     SHUTTLE_COLUMNS: Tuple[str, ...] = ("duration", "peak_velocity", "mean_velocity",
66                                         "active_ratio", "net_crossings")
67     PERSON_COLUMNS: Tuple[str, ...] = ("players_in_court", "two_sided_motion",
"mean_player_motion",
68                                         "in_court_ratio", "shuttle_player_colocation")
69     AUDIO_COLUMNS: Tuple[str, ...] = ("audio_hit_count", "audio_hit_rate",
"audio_hit_strength_mean")
70
71     # Verdict vocabulary (the no-over-claim contract ? never "irrelevant"/"proven useless").
72     EARNS_KEEP = "earns_keep" # CI clears 0 AND sign test agrees ? removing it
hurts
73     SUGGESTIVE = "suggestive" # point estimate leans positive but CI spans 0
74     INERT = "inert" # wash this run (?0, CI spans 0) ? candidate,
not proof
75     STRUCTURAL_PROTECTED = "structural_protected" # a guard; never auto-demoted on "no
lift"
76

```

```

77 #: Human-readable one-liners for each verdict (used in the PDF + table legend).
78 VERDICT_LABEL = {
79     EARNS_KEEP: "clears-bar",
80     SUGGESTIVE: "suggestive",
81     INERT: "inert",
82     STRUCTURAL_PROTECTED: "structural-protected",
83 }
84
85
86 @dataclass(frozen=True)
87 class Signal:
88     """One unit of ablation ? a feature column (or a named group) and/or a gate knob.
89
90     - ``name`` ? table/row identity ("person.colocation" | "gate_a" | "audio").
91     - ``group`` ? producer ("person" | "gate_a" | "audio" | "shuttle"). LOGO ablates a
whole group.
92     - ``columns`` ? the persisted feature columns this signal contributes (may be empty
for a
93     pure gate signal like Gate-A whose column is shared with the shuttle group).
94     - ``gate_knob`` ? e.g. ``"min_crossings"`` for the Gate-A threshold; ablated by
zeroing it on
95     the full variant rather than dropping a column.
96     - ``structural`` ? a guard whose value is in failure modes the golden corpus may
lack; NEVER
97     flagged dead weight on "no measured lift" (mirrors ``per_user_gate``
``structural=True``).
98     """
99
100     name: str
101     group: str
102     columns: Tuple[str, ...] = ()
103     gate_knob: Optional[str] = None
104     structural: bool = False
105
106
107 @dataclass
108 class AblationConfig:
109     """How to run an ablation over a corpus.
110
111     - ``full`` ? the full-set reference variant (B). For the litmus Gate-A reproduction
this is
112     the STATIC ``Variant(min_crossings=2)`` (no classifier); for a learned ablation it
carries
113     ``feature_names`` = all columns and the gates ON.
114     - ``signals`` ? the inventory to ablate.
115     - ``granularity`` ? ``"signal"`` (LOSO, one column-set at a time) or ``"group"``
(LOGO).
116     - ``iou`` ? temporal-IoU match threshold.
117     - ``honest`` ? LOVO for learned variants (the number to trust).
118     - ``seed`` ? threaded into the ``eval_stats`` bootstrap (determinism).
119     - ``p_threshold`` ? sign-test significance bar for the ``earns_keep`` verdict.
120     """
121
122     full: Variant
123     signals: List[Signal]
124     granularity: str = "group"
125     iou: float = 0.5
126     honest: bool = True
127     seed: int = 0
128     p_threshold: float = 0.05
129
130
131 @dataclass
132 class AblationRow:
133     """One signal's marginal-contribution row (?s) + CI + sign test + C4 + a

```

```

verdict)."""
134
135     signal: str
136     group: str
137     structural: bool
138     dropped_columns: List[str]
139     gate_knob: Optional[str]
140     n: int
141     delta_f1: float
142     ci_low: float
143     ci_high: float
144     ci_excludes_zero: bool
145     sign_wins: int
146     sign_losses: int
147     sign_p_value: float
148     f1_full: float
149     f1_ablated: float
150     seg_ratio_full: float
151     seg_ratio_ablated: float
152     verdict: str
153     structural_protected: bool
154
155     def as_dict(self) -> Dict[str, Any]:
156         return {
157             "signal": self.signal,
158             "group": self.group,
159             "structural": self.structural,
160             "dropped_columns": list(self.dropped_columns),
161             "gate_knob": self.gate_knob,
162             "n": self.n,
163             "delta_f1": self.delta_f1,
164             "ci_low": self.ci_low,
165             "ci_high": self.ci_high,
166             "ci_excludes_zero": self.ci_excludes_zero,
167             "sign_test": {"wins": self.sign_wins, "losses": self.sign_losses,
168                           "p_value": self.sign_p_value},
169             "f1_full": self.f1_full,
170             "f1_ablated": self.f1_ablated,
171             "segment_count_ratio_full": self.seg_ratio_full,
172             "segment_count_ratio_ablated": self.seg_ratio_ablated,
173             "verdict": self.verdict,
174             "structural_protected": self.structural_protected,
175         }
176
177
178     @dataclass
179     class AblationReport:
180         """All signals' rows + the run's provenance ? a structured, JSON-able ablation
181         result."""
182         granularity: str
183         iou: float
184         num_videos: int
185         honest_lovo: bool
186         full_set_signals: List[str]
187         full_spec_hash: str
188         label_set_hash: str
189         rows: List[AblationRow] = field(default_factory=list)
190
191         def sorted_rows(self) -> List[AblationRow]:
192             """Rows by marginal ?F1 desc ? the contribution ranking (the plan's table
193             order)."""
194             return sorted(self.rows, key=lambda r: r.delta_f1, reverse=True)
195
196         def as_dict(self) -> Dict[str, Any]:

```

```

196         return {
197             "granularity": self.granularity,
198             "iou": self.iou,
199             "num_videos": self.num_videos,
200             "honest_lovo": self.honest_lovo,
201             "full_set_signals": list(self.full_set_signals),
202             "full_spec_hash": self.full_spec_hash,
203             "label_set_hash": self.label_set_hash,
204             "rows": [r.as_dict() for r in self.sorted_rows()],
205         }
206
207
208     # ----- the driver
209
210
211     def _ablated_variant(full: Variant, columns: Sequence[str], gate_knob: Optional[str]) ->
Variant:
212         """Build the leave-one-out (smaller) variant from ``full``: drop ``columns`` from
its
213         ``feature_names`` and/or zero the named gate knob. Pure ? derives a new Variant,
never
214         mutates ``full``."""
215         cols = set(columns)
216         fn = full.feature_names
217         ablated_fn = [c for c in fn if c not in cols] if fn is not None else fn
218         overrides: Dict[str, Any] = {"feature_names": ablated_fn}
219         if gate_knob == "min_crossings":
220             overrides["min_crossings"] = 0
221         elif gate_knob == "min_players":
222             overrides["min_players"] = 0
223         elif gate_knob is not None:
224             raise ValueError(f"unknown gate_knob {gate_knob!r} (expected
'min_crossings'/'min_players')")
225         dropped_label = ",".join(sorted(cols)) or (gate_knob or "?")
226         return experiment.Variant.from_dict(**full.to_dict(), **overrides,
227                                           "name": f"?{dropped_label}")
228
229
230     def _classify(structural: bool, ci_excludes_zero: bool, sign_wins: int, sign_losses:
int,
231                  sign_p: float, p_threshold: float) -> Tuple[str, bool]:
232         """The honest verdict (no over-claim). Returns ``(verdict, structural_protected)``.
233
234         A structural guard is reported but never demoted on "no lift". Otherwise: removing
the
235         signal *earns its keep* when the ?F1 CI clears 0 AND the sign test agrees (majority
worse on
236         removal, ``p <= p_threshold``); a positive-leaning but CI-spanning row is
``suggestive``; a
237         wash this run is ``inert`` (a candidate, never "proven useless" at n=6)."""
238         if structural:
239             return STRUCTURAL_PROTECTED, True
240         sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
241         if ci_excludes_zero and sign_agrees:
242             return EARNES_KEEP, False
243         if sign_wins > sign_losses:           # leans toward "removal hurts" but evidence is
thin
244             return SUGGESTIVE, False
245         return INERT, False
246
247
248     def ablate_one(videos: Sequence[VideoCandidates], cfg: AblationConfig,
249                  signal: Signal) -> AblationRow:
250         """One ablation row: marginal ?(s) + CI + sign test + C4 + a verdict.
251

```

```

252 Drops ``signal``'s columns / zeroes its gate knob from ``cfg.full`` to form the
ablated
253 variant A, then calls ``experiment.compare(A, full)`` so the reported B?A ?F1 IS
?(s).
254 Reuses ``compare`` verbatim ? learned variants stay LOVO-honest, the C1+C4 honest
stats
255 come for free."""
256 ablated = _ablated_variant(cfg.full, signal.columns, signal.gate_knob)
257 res = experiment.compare(videos, ablated, cfg.full, iou=cfg.iou, honest=cfg.honest,
258                         honest_stats=True)
259 # Re-pair the per-video ?F1 with the configured bootstrap seed for determinism (the
default
260 # honest block uses seed=0; thread cfg.seed explicitly so a run is reproducible).
261 from backend.eval.eval_stats import paired_delta_ci
262 by_a = {p["video"]: p["f1"] for p in res["variant_a"]["per_video"]}
263 by_b = {p["video"]: p["f1"] for p in res["variant_b"]["per_video"]}
264 vids = [p["video"] for p in res["variant_a"]["per_video"]]
265 pd = paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids],
seed=cfg.seed)
266 st = pd["sign_test"]
267 sign_wins, sign_losses = int(st["wins"]), int(st["losses"])
268 sign_p = float(st["p_value"])
269 verdict, protected = _classify(signal.structural, bool(pd["ci_excludes_zero"]),
270                               sign_wins, sign_losses, sign_p, cfg.p_threshold)
271
272 seg_full = res["honest"]["segmentation"]["variant_b"]["segment_count_ratio"]
273 seg_abl = res["honest"]["segmentation"]["variant_a"]["segment_count_ratio"]
274
275 return AblationRow(
276     signal=signal.name,
277     group=signal.group,
278     structural=signal.structural,
279     dropped_columns=list(signal.columns),
280     gate_knob=signal.gate_knob,
281     n=int(pd["n"]),
282     delta_f1=float(pd["mean_delta"]),
283     ci_low=float(pd["ci_low"]),
284     ci_high=float(pd["ci_high"]),
285     ci_excludes_zero=bool(pd["ci_excludes_zero"]),
286     sign_wins=sign_wins,
287     sign_losses=sign_losses,
288     sign_p_value=sign_p,
289     f1_full=float(res["variant_b"]["aggregate"]["f1"]),
290     f1_ablated=float(res["variant_a"]["aggregate"]["f1"]),
291     seg_ratio_full=float(seg_full),
292     seg_ratio_ablated=float(seg_abl),
293     verdict=verdict,
294     structural_protected=protected,
295 )
296
297
298 def _collapse_to_groups(signals: Sequence[Signal]) -> List[Signal]:
299     """Collapse a LOSO inventory into one Signal per producer GROUP (LOGO): union the
columns,
300     OR the gate knob, OR the structural flag. First-seen group order is preserved."""
301     order: List[str] = []
302     by_group: Dict[str, List[Signal]] = {}
303     for s in signals:
304         if s.group not in by_group:
305             order.append(s.group)
306             by_group[s.group] = []
307         by_group[s.group].append(s)
308     out: List[Signal] = []
309     for g in order:
310         members = by_group[g]

```

```

311         cols: Tuple[str, ...] = tuple(dict.fromkeys(c for m in members for c in
m.columns))
312         knob = next((m.gate_knob for m in members if m.gate_knob), None)
313         structural = any(m.structural for m in members)
314         out.append(Signal(name=g, group=g, columns=cols, gate_knob=knob,
structural=structural))
315         return out
316
317
318     def run_ablation(videos: Sequence[VideoCandidates], cfg: AblationConfig) ->
AblationReport:
319         """Loop the inventory at the configured granularity ? an ``AblationReport`` (sorted
table).
320
321         A pure driver: each row is one ``experiment.compare(without_s, full)``.
``granularity="group"``
322         (LOGO) ablates a whole producer's columns at once; ``"signal"`` (LOSO) one column-
set at a
323         time. The full reference is scored once implicitly per row (cheap re-scoring;
detection is
324         already persisted)."""
325         if len(videos) < 1:
326             raise experiment.ExperimentError("run_ablation needs at least one labeled
video")
327         if cfg.granularity not in ("signal", "group"):
328             raise ValueError(f"granularity must be 'signal' or 'group', got
{cfg.granularity!r}")
329
330         inventory = cfg.signals if cfg.granularity == "signal" else
_collapse_to_groups(cfg.signals)
331         rows = [ablate_one(videos, cfg, s) for s in inventory]
332
333         # Provenance: a full-set evaluation gives the honest-LOVO flag + the label-set
fingerprint.
334         full_eval = experiment.evaluate_variant(videos, cfg.full, iou=cfg.iou,
honest=cfg.honest)
335         from backend.eval.regression import gt_hash
336         all_gts = [iv for vc in videos for iv in (vc.gts or [])]
337         return AblationReport(
338             granularity=cfg.granularity,
339             iou=cfg.iou,
340             num_videos=len(videos),
341             honest_lovo=bool(full_eval["honest_lovo"]),
342             full_set_signals=[s.group for s in _collapse_to_groups(cfg.signals)],
343             full_spec_hash=cfg.full.spec_hash(),
344             label_set_hash=gt_hash(all_gts),
345             rows=rows,
346         )
347
348
349     # ----- default
inventories
350
351
352     def default_signals() -> List[Signal]:
353         """The current 4-group inventory (LOSO granularity): shuttle (velocity block +
duration),
354         gate_a (the structural net-crossings gate AND its column), person, audio.
355
356         Gate-A is marked ``structural`` (the held-shuttle FP guard) so it is reported but
never
357         flagged dead weight on "no measured lift". It ablates via the ``min_crossings`` knob
AND by
358         dropping the ``net_crossings`` column, so the leave-one-out genuinely removes the
whole cue."""

```

```

359     return [
360         Signal("shuttle.duration", "shuttle", ("duration",)),
361         Signal("shuttle.velocity", "shuttle", ("peak_velocity", "mean_velocity",
"active_ratio")),
362         Signal("gate_a", "gate_a", ("net_crossings",), gate_knob="min_crossings",
structural=True),
363         Signal("person", "person", PERSON_COLUMNS),
364         Signal("audio", "audio", AUDIO_COLUMNS),
365     ]
366
367
368     def static_gate_a_signal() -> Signal:
369         """The Gate-A-only structural signal (a pure gate knob over the recall-oriented
proposal
370         set). Ablating it against the static ``Variant(min_crossings=2)`` full reproduces
the
371         hand-measured Gate-A result (?F1 ? +0.074, 6/6, p ? 0.031) ? the framework's litmus
test."""
372         return Signal("gate_a", "gate_a", (), gate_knob="min_crossings", structural=True)
373
374
375     def litmus_config() -> AblationConfig:
376         """The Gate-A reproduction config: full = STATIC ``Variant(min_crossings=2)`` (no
learned
377         classifier), inventory = the single Gate-A gate signal. Ablating it = CONTROL+gate
vs CONTROL
378         ? exactly the historical measurement (``scratch/gate_ab_ablation.py``)."""
379         full = experiment.Variant(name="control_gate_a", min_crossings=2)
380         return AblationConfig(full=full, signals=[static_gate_a_signal()],
granularity="signal")
381
382
383     # ----- corpus
I/O
384
385
386     def load_videos(features_dir: str) -> List[VideoCandidates]:
387         """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals +
shuttle +
388         person + audio columns + golden gts). Audio columns are optional (older JSONs
predate the
389         audio augment) ? a missing block defaults those columns to 0.0 so the loader never
crashes,
390         mirroring ``experiment._feature_column``'s tolerance."""
391         videos: List[VideoCandidates] = []
392         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
393             with open(path, encoding="utf-8") as f:
394                 d = json.load(f)
395                 cands = d["candidates"]
396                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
397                 features: Dict[str, List[float]] = {}
398                 for fn in SHUTTLE_COLUMNS:
399                     features[fn] = [float(c["shuttle"][fn]) for c in cands]
400                 for fn in PERSON_COLUMNS:
401                     features[fn] = [float(c["person"][fn]) for c in cands]
402                 for fn in AUDIO_COLUMNS:
403                     features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cands]
404                 gts = [(float(a), float(b)) for a, b in d["gts"]]
405                 videos.append(VideoCandidates(name=d["name"], intervals=intervals,
features=features, gts=gts))
406
407         return videos
408
409
410     def learned_full_variant(*, threshold: float = 0.5, min_crossings: int = 0) -> Variant:
411         """The learned full-set reference: all 13 columns (shuttle 5 + person 5 + audio 3)

```

```

weighted
412         by the LOVO logistic scorer. Used for the LOSO/LOGO contribution table over the
learned path
413         (distinct from the static Gate-A litmus)."""
414         cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) + list(AUDIO_COLUMNS)
415         return experiment.Variant(name="full_learned", feature_names=cols,
416                                   threshold=threshold, min_crossings=min_crossings)
417
418
419     # ----- table
rendering
420
421
422     def format_table(report: AblationReport) -> str:
423         """Sorted ASCII/markdown ablation table (by marginal ?F1 desc ? the contribution
ranking)."""
424         rows = report.sorted_rows()
425         header = (f"Ablation report ? full set = {{{', '.join(report.full_set_signals)}}}"
426                  f" . n={report.num_videos} . IoU {report.iou}"
427                  f" . {'LOVO-honest' if report.honest_lovo else 'static'}"
428                  f" . {report.granularity.upper()}")
429         cols = ["signal", "?F1(full?ablated)", "95% CI", "sign test", "CI?0", "verdict"]
430         widths = [18, 17, 18, 14, 5, 22]
431
432         def _fmt_row(cells: Sequence[str]) -> str:
433             return " ".join(c.ljust(w) for c, w in zip(cells, widths))
434
435         lines = [header, "", _fmt_row(cols), _fmt_row(["-" * w for w in widths])]
436         for r in rows:
437             ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
438             sign = f"{r.sign_wins}W/{r.sign_losses}L p={r.sign_p_value:.3f}"
439             verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
440             lines.append(_fmt_row([r.signal, f"{r.delta_f1:+.3f}", ci, sign,
441                                   "yes" if r.ci_excludes_zero else "no", verdict]))
442         return "\n".join(lines)
443
444
445     # ----- PDF
export
446
447
448     def export_pdf(report: AblationReport, path: str) -> str:
449         """Render the ``AblationReport`` to a clean reportlab PDF: a table of each signal's
marginal
450         contribution + CI + sign test + a verdict column (clears-bar / suggestive / inert /
451         structural-protected). Reuses the muted booktabs aesthetic of
``scratch/quality_report.py``.
452
453         Returns the written path. ``reportlab`` is an offline, pure-Python dependency (no
GPU)."""
454         from reportlab.lib import colors
455         from reportlab.lib.enums import TA_CENTER, TA_LEFT
456         from reportlab.lib.pagesizes import A4
457         from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
458         from reportlab.lib.units import cm
459         from reportlab.platypus import (Paragraph, SimpleDocTemplate, Spacer, Table,
TableStyle)
460
461         navy = colors.HexColor("#1F3A5F")
462         black = colors.HexColor("#000000")
463         darkgrey = colors.HexColor("#333333")
464         light = colors.HexColor("#F0F0F0")
465         hairline = colors.HexColor("#BFBFBF")
466         row_alt = colors.HexColor("#F7F7F7")
467

```

```

468     base = getSampleStyleSheet()["Normal"]
469     title = ParagraphStyle("Title2", parent=base, fontName="Helvetica-Bold",
fontSize=16,
470                             leading=20, alignment=TA_CENTER, textColor=navy,
spaceAfter=4)
471     subtitle = ParagraphStyle("Sub", parent=base, fontName="Helvetica-Oblique",
fontSize=9.5,
472                             leading=13, alignment=TA_CENTER, textColor=darkgrey,
spaceAfter=10)
473     h2 = ParagraphStyle("H2", parent=base, fontName="Helvetica-Bold", fontSize=11,
leading=15,
474                             textColor=navy, spaceBefore=10, spaceAfter=4)
475     body = ParagraphStyle("Body", parent=base, fontName="Helvetica", fontSize=9,
leading=12.5,
476                             alignment=TA_LEFT, textColor=black, spaceAfter=5)
477     cell = ParagraphStyle("Cell", parent=base, fontName="Helvetica", fontSize=8.2,
leading=10.5,
478                             textColor=black)
479     cell_b = ParagraphStyle("CellB", parent=base, fontName="Helvetica-Bold",
fontSize=8.2,
480                             leading=10.5, textColor=black)
481     head = ParagraphStyle("Head", parent=base, fontName="Helvetica-Bold", fontSize=8.2,
482                             leading=10.5, textColor=black)
483
484     rows = report.sorted_rows()
485     story: List[Any] = [
486         Paragraph("Signal Ablation & Feature-Enablement Report", title),
487         Paragraph(
488             f"Leave-one-{'group' if report.granularity == 'group' else 'signal'}-out
marginal "
489             f"contribution &mdash; n={report.num_videos} golden matches, IoU
{report.iou}, "
490             f"{'honest LOVO' if report.honest_lovo else 'static re-score'}. "
491             "Honest measured results only.",
492             subtitle),
493     ]
494
495     full_set = ", ".join(report.full_set_signals)
496     story.append(Paragraph("Run provenance", h2))
497     story.append(Paragraph(
498         f"Full signal set: <b>{{{full_set}}}</b>. Variant spec_hash "
499         f"<font face='Courier'>{report.full_spec_hash}</font>, label-set hash "
500         f"<font face='Courier'>{report.label_set_hash}</font>. Each row is the held-out
"
501         "&Delta;F1 of removing one signal from the full set (full &minus; ablated), with
a "
502         "bootstrap 95% CI and an exact paired sign test &mdash; a signal earns its keep
only "
503         "when removing it drops F1 with the CI clearing 0 <i>and</i> the sign test
agreeing.",
504         body))
505
506     story.append(Paragraph("Marginal contribution per signal", h2))
507     table_data: List[List[Any]] = [[
508         Paragraph("Signal", head), Paragraph("Group", head),
509         Paragraph("&Delta;F1 (full&minus;abl.)", head), Paragraph("95% CI", head),
510         Paragraph("Sign test", head), Paragraph("CI&ne;0", head),
511         Paragraph("Over-seg", head), Paragraph("Verdict", head),
512     ]]
513     for r in rows:
514         ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
515         sign = f"{r.sign_wins}W/{r.sign_losses}L p={r.sign_p_value:.3f}"
516         overseg = f"{r.seg_ratio_full:.2f}&rarr;{r.seg_ratio_ablated:.2f}"
517         verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
518         df1_style = cell_b if r.verdict == EARNS_KEEP else cell

```

```

519         table_data.append([
520             Paragraph(r.signal, cell), Paragraph(r.group, cell),
521             Paragraph(f"{r.delta_f1:+.3f}", df1_style), Paragraph(ci, cell),
522             Paragraph(sign, cell), Paragraph("yes" if r.ci_excludes_zero else "no",
cell),
523             Paragraph(overseg, cell), Paragraph(verdict, cell),
524         ])
525     col_widths = [2.7 * cm, 1.7 * cm, 2.4 * cm, 2.7 * cm, 2.6 * cm, 1.1 * cm, 1.9 * cm,
2.6 * cm]
526     tbl = Table(table_data, colWidths=col_widths, repeatRows=1)
527     last = len(table_data) - 1
528     tbl.setStyle(TableStyle([
529         ("BACKGROUND", (0, 0), (-1, 0), light),
530         ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
531         ("TOPPADDING", (0, 0), (-1, -1), 4),
532         ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
533         ("LEFTPADDING", (0, 0), (-1, -1), 4),
534         ("RIGHTPADDING", (0, 0), (-1, -1), 4),
535         ("LINEABOVE", (0, 0), (-1, 0), 1.0, black),
536         ("LINEBELOW", (0, 0), (-1, 0), 0.8, black),
537         ("LINEBELOW", (0, 1), (-1, max(1, last - 1)), 0.3, hairline),
538         ("LINEBELOW", (0, last), (-1, last), 1.0, black),
539         ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, row_alt]),
540     ]))
541     story.append(tbl)
542     story.append(Spacer(1, 0.3 * cm))
543
544     story.append(Paragraph("Verdict key", h2))
545     story.append(Paragraph(
546         "<b>clears-bar</b> &mdash; removing the signal drops F1 with the &Delta;F1 CI
clearing 0 "
547         "and the sign test agreeing (the signal earns its keep). "
548         "<b>suggestive</b> &mdash; the point estimate leans toward 'removal hurts' but
the CI "
549         "spans 0 (keep, watch as the corpus grows). "
550         "<b>inert</b> &mdash; removal is a wash this run; a candidate for retirement
only after a "
551         "trend across multiple runs, never on one wash at small n. "
552         "<b>structural-protected</b> &mdash; a guard whose value is in failure modes the
golden "
553         "corpus may lack (e.g. Gate-A vs held-shuttle false positives); reported
honestly but "
554         "never auto-flagged dead weight on 'no measured lift'.",
555         body))
556
557     doc = SimpleDocTemplate(path, pagesize=A4, leftMargin=2 * cm, rightMargin=2 * cm,
558                             topMargin=1.8 * cm, bottomMargin=1.8 * cm,
559                             title="Signal Ablation & Feature-Enablement Report",
560                             author="badminton-highlight-indexer")
561     doc.build(story)
562     logger.info("wrote ablation PDF -> %s", path)
563     return path
564
565
566     # -----
CLI
567
568
569     def main() -> None:
570         ap = argparse.ArgumentParser(
571             description="LOSO/LOGO ablation runner over the golden feature substrate (pure
driver "
572                 "over experiment.compare + eval_stats; no new scoring math).")
573         ap.add_argument("--features-dir", default="output",
574             help="dir of *_golden_features.json (the persisted golden

```

```

substrate)")
575         ap.add_argument("--granularity", choices=("signal", "group"), default="group",
576                         help="leave-one-SIGNAL-out (column) or leave-one-GROUP-out
(producer)")
577         ap.add_argument("--iou", type=float, default=0.5)
578         ap.add_argument("--seed", type=int, default=0, help="bootstrap seed (determinism)")
579         ap.add_argument("--litmus", action="store_true",
580                         help="reproduce the static Gate-A measurement (CONTROL vs
min_crossings=2) "
581                             "instead of the learned full-set ablation")
582         ap.add_argument("--pdf", default=None, help="also write a PDF ablation report to
this path")
583         ap.add_argument("--json", default=None, help="also write the structured report JSON
here")
584         args = ap.parse_args()
585
586         videos = load_videos(args.features_dir)
587         if len(videos) < 2:
588             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} "
589                             f"(found {len(videos)})")
590
591         if args.litmus:
592             cfg = litmus_config()
593             cfg.iou, cfg.seed = args.iou, args.seed
594         else:
595             cfg = AblationConfig(full=learned_full_variant(), signals=default_signals(),
596                                 granularity=args.granularity, iou=args.iou, seed=args.seed)
597
598         report = run_ablation(videos, cfg)
599         print(format_table(report))
600
601         if args.json:
602             with open(args.json, "w", encoding="utf-8") as f:
603                 json.dump(report.as_dict(), f, indent=2)
604             print(f"\nwrote report JSON -> {args.json}")
605         if args.pdf:
606             export_pdf(report, args.pdf)
607             print(f"wrote PDF report -> {args.pdf}")
608
609
610         if __name__ == "__main__":
611             main()
612

```

37. assistant (2026-06-14T10:51:36.348Z)

Now I have a comprehensive understanding of the entire architecture. Let me check the `trajectory\_to\_action\_windows` signature and look at existing tests to understand test patterns.

38. user (2026-06-14T10:51:38.875Z)

```

252:     def trajectory_to_action_windows(
===tests dir===
test_ablation.py
test_distill_local.py
test_experiment.py
test_experiment_honest.py
test_fusion_compare.py
test_fusion_features.py
test_fusion_golden_telemetry.py
test_fusion_hybrid.py
===run_all_tests===
run_all_tests.py

```

39. user (2026-06-14T10:51:42.004Z)

```
252     def trajectory_to_action_windows(  
253         points: List[TrajectoryPoint],  
254         fps: float,  
255         frame_width: float,  
256         chunk_start: float = 0.0,  
257         velocity_thresh: float = 0.02,  
258         merge_gap: float = 2.0,  
259         min_window_duration: float = 2.0,  
260         chunk_index: Optional[int] = None,  
261     ) -> List[Dict[str, Any]]:  
262         """Convert a shuttle trajectory into candidate rally windows.  
263  
264         A frame is "active" when the shuttle is visible AND its inter-frame  
265         displacement (normalised by frame width, per second) exceeds  
266         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within  
267         ``merge_gap`` seconds of each other are grouped into one window; short  
268         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict  
269         shape feeding the AI-handoff phase is identical.  
270         """  
271         if fps <= 0:  
272             fps = 30.0  
273         if frame_width <= 0:  
274             frame_width = 1280.0  
275  
276         # Compute per-frame normalised velocity from the last *visible* point.  
277         active = [] # (frame_idx, velocity)  
278         prev: Optional[TrajectoryPoint] = None  
279         for p in points:  
280             if not p.visible:  
281                 prev = None # break the trajectory; absent shuttle = not in flight  
282                 continue  
283             if prev is not None:  
284                 dframes = p.frame - prev.frame  
285                 if dframes > 0:  
286                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5  
287                     velocity = (dist / frame_width) * (fps / dframes)  
288                     if velocity > velocity_thresh:  
289                         active.append((p.frame, velocity))  
290             prev = p  
291  
292         if not active:  
293             return []  
294  
295         max_gap_frames = int(merge_gap * fps)  
296  
297         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:  
298             vels = [v for _, v in group]  
299             return {  
300                 "start": group[0][0] / fps + chunk_start,  
301                 "end": group[-1][0] / fps + chunk_start,  
302                 "active_frame_count": len(group),  
303                 "peak_velocity": max(vels),  
304                 "mean_velocity": sum(vels) / len(vels),  
305                 "track_id_count": 1, # single shuttle; kept for window-shape parity  
306                 "chunk_index": chunk_index,  
307             }  
308  
309         windows = []  
310         group = [active[0]]  
311         for af in active[1:]:
```

40. assistant (2026-06-14T10:51:50.742Z)

Now I have everything I need. Let me design and build the fix.

The fix design

****Root cause:**** The windowing presets are scattered as bare tuples (``PROPOSAL = (0.005, 1.0, 0.5)``, ``BASELINE_LOCAL = (0.02, 2.0, 2.0)``) in ``distill_local.py``, and the served windowing constants (``velocity_threshold=0.02``, ``dead_time_merge_gap=2.0``, ``min_action_window_duration=2.0``) live independently in ``config/models.py`` and are read in ``trajectory_hybrid.process_video``. There is no single source linking them, and nothing prevents a cue from being promoted on the recall-oriented PROPOSAL windowing that production never serves.

****Minimal foundational fix:****

1. A new single-source module ``backend/eval/windowing.py`` defining named ``WindowingPreset``'s ? ``SERVED`` (sourced from the same config defaults production reads) and ``PROPOSAL`` ? plus a ``served_windowing_from_config()`` so eval and serve reference the SAME served constants.
2. A ``promotion_guard`` helper encoding the rule: a cue is promotable only if it does NOT regress at the SERVED windowing (composes with ``eval_stats`` + ``per_user_gate``).
3. Re-point ``distill_local``'s presets at the single source (back-compat), and re-run the Gate-A ablation at both windowings.

Let me first verify the current test baseline is green, then build.

41. user (2026-06-14T10:51:55.465Z)

```
3.13.13 (tags/v3.13.13:011104ce, Apr  7 2026, 19:25:48) [MSC v.1944 64 bit (AMD64)]
..... [100%]
21 passed in 1.92s
```

42. assistant (2026-06-14T10:52:02.212Z)

Tests pass. Now let me check ruff/mypy availability and the existing `distill_local` test to understand test conventions.

43. user (2026-06-14T10:52:04.649Z)

```
ruff 0.15.16
mypy 2.1.0 (compiled: yes)
===mypy config===
ls: cannot access 'setup.cfg': No such file or directory
ls: cannot access 'pyproject.toml': No such file or directory
mypy.ini
===check distill_local in mypy scope===
grep: pyproject.toml: No such file or directory
grep: setup.cfg: No such file or directory
```

44. user (2026-06-14T10:52:07.937Z)

```
1      [mypy]
2      # Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean
3      # modules and pins an explicit quarantine list below ? not a typing crusade.
4      # Ratchet: when you touch a quarantined module, try removing its entry.
5      ignore_missing_imports = True
6      # backend/ is a namespace package (no root __init__.py) ? required so mypy
7      # maps files to backend.* module names instead of duplicating top-level names.
8      explicit_package_bases = True
9
10     # --- Quarantine (known-loose modules, each with a reason) ---
11
12     # Optional module globals (db_instance/global_config set only in main());
13     # properly fixed by the deferred app-factory/lifespan refactor (BACKLOG).
14     [mypy-backend.main]
15     ignore_errors = True
16
```

```

17     # Legacy demo segmenter (fabricated metadata; coverage-omitted too).
18     [mypy-backend.pipeline.segmenters.motion]
19     ignore_errors = True
20
21     # yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code
22     # to detectors/person_detector.py (which IS type-clean, no quarantine). What remains
23     # here is pre-existing orchestration debt unrelated to that move (implicit-Optional
24     # process_video defaults, the Success|Failure ABC-override mismatch, a Future[...]
25     # annotation) ? ratchet those out in a focused typing pass, not here.
26     # (wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)
27     [mypy-backend.pipeline.segmenters.yolo_hybrid]
28     ignore_errors = True
29
30     [mypy-backend.pipeline.segmenters.gemini]
31     ignore_errors = True
32
33     # Registry builds cls(None, {}) probes; typing it honestly needs a Protocol.
34     [mypy-backend.pipeline.segmenters.base]
35     ignore_errors = True
36

```

45. assistant (2026-06-14T10:52:11.210Z)

Now let me check how ruff is configured (line length etc.) by looking at an existing file's style and the ruff config.

46. user (2026-06-14T10:52:12.726Z)

```

ls: cannot access '.ruff.toml': No such file or directory
ruff.toml
All checks passed!
===how tests run===
#!/usr/bin/env python
"""Run the FULL test suite for the badminton-highlight-indexer.

```

A single, discoverable entry point so you (or a reviewer / CI) can verify the whole suite before opening a PR:

```

python run_all_tests.py           # full suite, quiet
python run_all_tests.py -v        # verbose (passthrough to pytest)
python run_all_tests.py -k config # filter by keyword (passthrough)
python run_all_tests.py --cov=backend --cov-report=term-missing # coverage

```

Anything after the script name is passed straight through to pytest.

Notes

```

-----
- The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
- The observability **reporting** tests skip cleanly if the optional ``obsreport``
  package isn't installed (they use ``pytest.importorskip``). To exercise them too:
  pip install -e ../sports-obsreport --no-build-isolation
- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
"""

```

```

from __future__ import annotations

```

```

import importlib.util
import subprocess
import sys

```

```

def _installed(module: str) -> bool:
    try:

```

47. user (2026-06-14T10:52:14.932Z)

```

1      # Lint-only lane (no `ruff format` enforcement ? a whole-repo reformat would
2      # bury blame history for zero behavior value).
3      #
4      # Selected: pyflakes + pycodestyle errors (ruff defaults) + pycodestyle
5      # warnings (W) + bugbear (B). Deferred with intent: I (import-sort) and UP
6      # (pyupgrade) ? both are large mechanical diffs; adopt via a dedicated PR.
7      target-version = "py38"
8
9      # scratch/ = throwaway debugging scripts that may hit real APIs; it is also
10     # excluded from pytest collection (see pytest.ini).
11     extend-exclude = ["scratch"]
12
13     [lint]
14     extend-select = ["W", "B"]
15
16     [lint.per-file-ignores]
17     # Entrypoint scripts deliberately run setup (load_dotenv, logging config,
18     # sys.path) before importing backend modules ? import order is load-bearing.
19     "backend/main.py" = ["E402"]
20     "scripts/run_phase3_eval.py" = ["E402"]
21     "scripts/run_process_and_stitch.py" = ["E402"]
22

```

48. assistant (2026-06-14T10:52:18.786Z)

I note `target-version = "py38"` ? so I must use py38-compatible code (the CLAUDE.md also says Python 3.8+). The existing code uses `from \_\_future\_\_ import annotations` for newer typing syntax. Now let me build the foundational fix.

First, the single-source windowing module.

49. user (2026-06-14T10:52:19.759Z)

<tool_use_error>Error: No such tool available: Mark_chapter</tool_use_error>

50. user (2026-06-14T10:52:23.257Z)

Chapter marked: "Build windowing single-source fix". Continue your current work.

51. assistant (2026-06-14T10:52:27.055Z)

Now let me create the single-source windowing module. The key design constraint: the SERVED preset must reference the SAME constants production reads from `config/models.py` (the Pydantic defaults), so they can never drift.

52. assistant (2026-06-14T10:53:04.716Z)

"""Single-sourced candidate-windowing presets ? the eval?serve foundation fix.

The bug this closes. The offline eval substrate (`distill\_local.build\_candidates` ? `fusion\_golden` ? the golden feature JSONs the ablation/compare path re-scores) was built on a *recall-oriented PROPOSAL* windowing (`vt=0.005, merge\_gap=1.0, min\_window=0.5`): deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED pipeline (`TrajectoryHybridSegmenter.process\_video`) windows COARSELY (`vt=0.02, merge\_gap=2.0, min\_window=2.0` ? the `config.indexing` defaults). A cue measured on the proposal set is measured on candidates **production never serves**, so a "win" need not transfer. This is exactly how Gate-A scored **+0.074** on the proposal set yet **?0.008** on the served set (`scratch/GATE\_AB\_NUMBERS\_AND\_IMPLICATIONS.md` vs `output/gateA\_served\_verify`).

The fix. One module owns the two named windowings as :class:`WindowingPreset`s, and the SERVED preset is **single-sourced** from the same Pydantic config defaults production reads* (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift. Helpers build candidates under either preset, and :func:`served\_windowing\_from\_config` lifts the **actual** served windowing out of a live config (overrides included), so an A/B can target the

operating point a given deployment truly serves.

The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED windowing* ? lives in :mod:`backend.eval.promotion` (which composes ``eval\_stats`` + ``per\_user\_gate``). This module is the **what windowing**; that one is the **promotion gate**.

Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until a caller opts in (``distill\_local`` re-points its module-level presets here, byte-identically).

```
from __future__ import annotations
```

```
from dataclasses import dataclass
```

```
from typing import Any, Dict, List, Mapping, Tuple
```

```
from backend.config.models import IndexingConfig
```

```
Interval = Tuple[float, float]
```

```
__all__ = [  
    "WindowingPreset",  
    "SERVED",  
    "PROPOSAL",  
    "PRESETS",  
    "served_windowing_from_config",  
    "build_windows",  
    "windows_to_intervals",  
]
```

```
@dataclass(frozen=True)
```

```
class WindowingPreset:
```

```
    """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating point.
```

```
    ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame counts as  
    "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one window;  
    ``min_window_duration`` ? windows shorter than this are dropped. These are exactly the three  
    knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset *is* the windowing.  
    """
```

```
    name: str
```

```
    velocity_thresh: float
```

```
    merge_gap: float
```

```
    min_window_duration: float
```

```
    def as_tuple(self) -> Tuple[float, float, float]:
```

```
        """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple shape the  
        ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak."""
```

```
        return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
```

```
    def to_dict(self) -> Dict[str, Any]:
```

```
        return {
```

```
            "name": self.name,
```

```
            "velocity_thresh": self.velocity_thresh,
```

```
            "merge_gap": self.merge_gap,
```

```
            "min_window_duration": self.min_window_duration,
```

```
        }
```

```
    def _served_preset_from_defaults() -> WindowingPreset:
```

```
        """The SERVED windowing, lifted from the SAME Pydantic config defaults production reads.
```

```
        ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
```

```
        - velocity: ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
```

```
wasb/fusion/tracknet)
```

```
        - merge_gap: ``idx_cfg.dead_time_merge_gap`` (default 2.0)
```

```
- min window: ``idx_cfg.min_action_window_duration`` (default 2.0)
```

Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is the whole point: change a served default in ``config/models.py`` and the eval SERVED preset follows, so the two can never silently diverge again.

```
"""
```

```
idx = IndexingConfig() # construct with defaults ? the served operating point
return WindowingPreset(
    name="served",
    velocity_thresh=float(idx.wasb.velocity_threshold), # == fusion/tracknet default 0.02
    merge_gap=float(idx.dead_time_merge_gap),
    min_window_duration=float(idx.min_action_window_duration),
)
```

```
#: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A cue is
#: only promotable if it holds up HERE ? the operating point real users get.
```

```
SERVED: WindowingPreset = _served_preset_from_defaults()
```

```
#: The recall-oriented PROPOSAL windowing the offline substrate is built on (deliberately
#: permissive: propose many, let the classifier/gate filter). NOT what production serves ? a
#: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
#: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay valid.
```

```
PROPOSAL: WindowingPreset = WindowingPreset(
    name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
)
```

```
#: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
```

```
PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
```

```
def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
```

```
    """The SERVED windowing a *specific* live config serves (overrides honoured).
```

```
    Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against an
    arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter ``family``
    (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a served
    default in ``config.json`` ? the module-level :data:`SERVED` is the *default* operating
```

```
point;
```

```
this is *this config's* operating point. Falls back to the served defaults for any absent
key.
```

```
"""
```

```
idx = _as_mapping(config, "indexing")
family_cfg = _as_mapping(idx, family)
return WindowingPreset(
    name=f"served:{family}",
    velocity_thresh=float(family_cfg.get("velocity_threshold", SERVED.velocity_thresh)),
    merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
    min_window_duration=float(idx.get("min_action_window_duration",
SERVED.min_window_duration)),
)
```

```
def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
```

```
    """``obj[key]`` as a plain mapping, whether ``obj`` is a dict or a typed config section
    (``AppConfig.get`` returns nested sections as dicts). Empty mapping if absent/non-
    mapping."""
```

```
    if obj is None:
```

```
        return {}
```

```
    if isinstance(obj, Mapping):
```

```
        sub = obj.get(key, {})
```

```
    elif hasattr(obj, "get"):
```

```
        sub = obj.get(key, {})
```

```
    else:
```

```
        sub = getattr(obj, key, {})
```

```

return sub if isinstance(sub, Mapping) else {}

def build_windows(points: List[Any], fps: float, frame_width: float,
                  preset: WindowingPreset) -> List[Dict[str, Any]]:
    """Trajectory ``points`` ? candidate windows under ``preset`` (the one windowing call site).

    A thin wrapper over ``TrackNetRunner.trajectory_to_action_windows`` that takes a named
    preset
    instead of three loose floats, so eval candidate-building and the served path reference the
    SAME knobs by name. Imported lazily to keep this module import-light (no detector deps)."""
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

    vt, mg, mwd = preset.as_tuple()
    return TrackNetRunner.trajectory_to_action_windows(
        points, fps=fps, frame_width=frame_width,
        velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)

def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
    """``[start, end, ...]`` ? ``[(start, end)]`` ? the interval shape the scorer/gate
    speak."""
    return [(w["start"], w["end"]) for w in windows]

```

53. user (2026-06-14T10:53:04.732Z)

File created successfully at: C:/Users/avidu/Projects/bhi-evalserve/backend/eval/windowing.py
(file state is current in your context ? no need to Read it back)

54. assistant (2026-06-14T10:53:22.119Z)

Now the promotion guard module that encodes the rule. Let me first verify
`should\_promote\_for\_user`'s exact signature is the right thing to compose with, and check how
the served-vs-proposal scores would feed in.

55. assistant (2026-06-14T10:54:05.882Z)

"""Promotion gate with the eval?serve guard: a cue is promotable ONLY if it does not regress
at the SERVED windowing.

Why this exists. ``per\_user\_gate.should\_promote\_for\_user`` already grades a B?A lift
honestly (min-N floor + ?F1 CI excludes 0 + paired sign test). But it grades whatever scores
it is handed ? and the foundational bug was that those scores came from the *recall-oriented
PROPOSAL* windowing (``backend.eval.windowing.PROPOSAL``), which production never serves. So a
cue could clear the bar on a windowing nobody runs (Gate-A: proposal ?F1 **++0.074**) while
regressing on the served one (**??0.008**). Grading harder isn't the fix; **measuring on the
served windowing** is.

This module composes ? it adds no new statistics. It layers ONE rule on top of
`should\_promote\_for\_user`:

A cue is promotable only if, at the **SERVED** windowing, it WINS or at least DOES NOT
REGRESS (its served paired ?F1 mean ? ``-served\_regress\_eps``, and ? when its served CI is
estimable ? that CI's lower bound does not sit clearly in regression territory).

A proposal-windowing win is still *reported* (it's useful signal for where to look next), but it
can never carry a promotion on its own. The decision returns BOTH the served and the proposal
verdicts so the eval?serve contrast is explicit and auditable.

Pure, deterministic, stdlib-only over ``eval\_stats``/``per\_user\_gate``. Additive: existing eval
paths are untouched until a caller routes its A/B through :func:`promote\_if\_served\_safe`.

```

from __future__ import annotations

```

```

from dataclasses import dataclass, field
from typing import Any, Dict, Optional, Sequence

from backend.eval.eval_stats import paired_delta_ci
from backend.eval.per_user_gate import PromotionDecision, should_promote_for_user

__all__ = ["ServedGateResult", "promote_if_served_safe", "served_regresses"]

def served_regresses(a_scores: Sequence[float], b_scores: Sequence[float], *,
                     eps: float = 0.0, n_boot: int = 2000, seed: int = 0) -> Dict[str, Any]:
    """Does variant B REGRESS variant A at the served windowing? (the eval?serve guard core).

    ``a_scores``/``b_scores`` are per-video held-out scores **measured at the SERVED windowing**
    (aligned by video). Returns the paired B?A summary (mean ?F1 + bootstrap CI + sign test)
    plus
    a single ``regresses`` boolean:

    - ``regresses=True`` when the served mean ?F1 is below ``-eps`` (a real served drop), OR
      the served ?F1 CI lies **entirely** below ``-eps`` (a confidently-negative band).
    - ``regresses=False`` when the cue holds the line at the served operating point (mean ?
      ``-eps`` and the CI is not a confident net-negative).

    ``eps`` is the regression tolerance: 0.0 = "must not drop at all on the served mean". A tiny
    positive ``eps`` lets noise-level wobble through (e.g. 0.005 ? half an F1 point at n?6).
    """
    delta = paired_delta_ci(a_scores, b_scores, n_boot=n_boot, seed=seed)
    mean_delta = float(delta["mean_delta"])
    ci_low = float(delta["ci_low"])
    ci_high = float(delta["ci_high"])
    # Regression if the served mean dropped beyond tolerance, OR the whole CI is net-negative.
    mean_regresses = mean_delta < -eps
    ci_confidently_negative = ci_high < -eps
    regresses = bool(mean_regresses or ci_confidently_negative)
    return {
        "regresses": regresses,
        "mean_delta": mean_delta,
        "ci_low": ci_low,
        "ci_high": ci_high,
        "ci_excludes_zero": bool(delta["ci_excludes_zero"]),
        "sign_test": delta["sign_test"],
        "n": int(delta["n"]),
        "eps": float(eps),
    }

@dataclass
class ServedGateResult:
    """A promotion decision that BOTH measures the cue on the served windowing AND reports the
    proposal-windowing view, so the eval?serve contrast is never hidden.

    - ``promote`` ? final verdict. ``True`` only when the underlying honest gate would promote
      *and* the served check did not veto (the cue does not regress at the served windowing).
    - ``served_safe`` ? the eval?serve guard's verdict in isolation (cue does not regress
    served).
    - ``vetoed_by_served`` ? ``True`` when the proposal/base gate WOULD have promoted but the
      served regression check overruled it (the exact Gate-A failure mode this module prevents).
    - ``base_decision`` ? the ``PromotionDecision`` from ``should_promote_for_user`` on the
      *promotion-grade* scores (served by default ? see :func:`promote_if_served_safe`).
    - ``served`` / ``proposal`` ? the paired B?A summary at each windowing (means + CIs + sign
      tests), so a reviewer sees "+0.074 proposal / ?0.008 served" side by side.
    - ``reason`` ? short human-readable explanation of the final verdict.
    """

    promote: bool

```

```

served_safe: bool
vetoed_by_served: bool
reason: str
base_decision: PromotionDecision
served: Dict[str, Any]
proposal: Optional[Dict[str, Any]] = field(default=None)

def as_dict(self) -> Dict[str, Any]:
    return {
        "promote": self.promote,
        "served_safe": self.served_safe,
        "vetoed_by_served": self.vetoed_by_served,
        "reason": self.reason,
        "base_decision": self.base_decision.as_dict(),
        "served": self.served,
        "proposal": self.proposal,
    }

def promote_if_served_safe(
    served_a: Sequence[float],
    served_b: Sequence[float],
    *,
    proposal_a: Optional[Sequence[float]] = None,
    proposal_b: Optional[Sequence[float]] = None,
    structural: bool = False,
    min_n: int = 5,
    p_threshold: float = 0.05,
    served_regress_eps: float = 0.0,
    n_boot: int = 2000,
    seed: int = 0,
) -> ServedGateResult:
    """Promote a cue ONLY if it does not regress at the SERVED windowing (the foundational
rule).

The honest bar (`should_promote_for_user`: min-N floor + ?F1 CI excludes 0 + sign test) is
applied to the served scores ? the operating point production runs ? so a proposal-only
win can never carry a promotion. The optional `proposal_` scores are scored too and
reported alongside, purely to surface the eval?serve contrast (they NEVER promote on their
own).

Args:
    served_a/served_b: per-video held-out scores for variant A (control) and B (the cue),
        measured at the SERVED windowing. These drive the decision.
    proposal_a/proposal_b: the same scores at the PROPOSAL windowing (optional). Reported for
        contrast; the served regression guard still has the final say.
    structural: forwarded to `should_promote_for_user` ? a structural guard (e.g. Gate-A vs
        held-shuttle FPs) is never disabled on "no lift", but it is ALSO not auto-promoted
to
        the served default if it regresses served. So for `structural=True` we report
        `keep_on` honestly but `promote` reflects the served-safety check.
    served_regress_eps: served regression tolerance (0.0 = "no served-mean drop allowed").

Returns a :class:`ServedGateResult` with the final verdict + both windowings' evidence.
"""
# 1) The honest gate, applied to the SERVED scores (never the proposal ones).
base = should_promote_for_user(
    served_a, served_b,
    structural=structural, min_n=min_n, p_threshold=p_threshold,
    n_boot=n_boot, seed=seed,
)

# 2) The eval?serve guard: does B regress A at the served windowing?
served = served_regresses(served_a, served_b, eps=served_regress_eps,
                          n_boot=n_boot, seed=seed)

```

```

served_safe = not served["regresses"]

# 3) Optional proposal-windowing view (reported, never promoting on its own).
proposal: Optional[Dict[str, Any]] = None
if proposal_a is not None and proposal_b is not None:
    proposal = served_regresses(proposal_a, proposal_b, eps=served_regress_eps,
                                n_boot=n_boot, seed=seed)

# 4) Compose the verdict. The served guard can only ever REMOVE a promotion, never add one.
if structural:
    # A structural guard ships ON regardless of measured lift (base.keep_on stays True), but
    # it is only auto-promoted to the SERVED default when it is served-safe. If it regresses
    # served, promote=False (don't bake an eval-only win into production) while keep_on
holds.
    promote = served_safe
    vetoed = False
    if served_safe:
        reason = ("structural guard kept ON; served-safe (does not regress at the served "
                  f"windowing, ?F1 {served['mean_delta']:+.4f}) ? eligible for served "
default")
    else:
        reason = ("structural guard kept ON, but NOT promoted to the served default: it "
                  f"REGRESSES at the served windowing (?F1 {served['mean_delta']:+.4f}); "
                  "an eval-only win must not be served")
    else:
        would_promote = base.promote
        if would_promote and not served_safe:
            promote = False
            vetoed = True
            reason = ("VETOED by the served-windowing guard: the cue clears the honest bar but "
                      f"REGRESSES at the served windowing (served ?F1 "
{served['mean_delta']:+.4f}"
                      f", CI [{served['ci_low']:+.4f}, {served['ci_high']:+.4f}]). A win on a "
                      "windowing production doesn't serve cannot promote.")
        elif would_promote:
            promote = True
            vetoed = False
            reason = ("promoted: clears the honest bar AND is served-safe (served ?F1 "
                      f"{served['mean_delta']:+.4f}). " + base.reason)
        else:
            promote = False
            vetoed = False
            reason = base.reason

return ServedGateResult(
    promote=promote,
    served_safe=served_safe,
    vetoed_by_served=vetoed,
    reason=reason,
    base_decision=base,
    served=served,
    proposal=proposal,
)

```

56. user (2026-06-14T10:54:05.904Z)

File created successfully at: C:/Users/avidu/Projects/bhi-evalserve/backend/eval/promotion.py
(file state is current in your context ? no need to Read it back)

57. assistant (2026-06-14T10:54:09.749Z)

Now re-point `distill\_local`'s presets at the single source for byte-identical back-compat. Let me make the surgical edit.

58. user (2026-06-14T10:54:18.221Z)

The file C:/Users/avidu/Projects/bhi-evalserve/backend/eval/distill_local.py has been updated successfully. (file state is current in your context ? no need to Read it back)

59. assistant (2026-06-14T10:54:21.160Z)

Now let me verify the byte-identical preservation and that imports work, then run the existing tests.

60. user (2026-06-14T10:54:26.763Z)

```
PROPOSAL      (0.005, 1.0, 0.5) == (0.005, 1.0, 0.5)? True
BASELINE_LOCAL (0.02, 2.0, 2.0) == (0.02, 2.0, 2.0)? True
SERVED preset {'name': 'served', 'velocity_thresh': 0.02, 'merge_gap': 2.0,
'min_window_duration': 2.0}
PROPOSAL preset {'name': 'proposal', 'velocity_thresh': 0.005, 'merge_gap': 1.0,
'min_window_duration': 0.5}
PRESETS keys ['served', 'proposal']
```

61. assistant (2026-06-14T10:54:33.527Z)

Byte-identical, and SERVED is correctly sourced from the config defaults. Now let me verify `served\_windowing\_from\_config` works against both a typed config and a plain dict, and that the `process\_video` served knobs match.

62. user (2026-06-14T10:54:37.620Z)

```
typed served (wasb)  (0.02, 2.0, 2.0)
typed served (fusion) (0.02, 2.0, 2.0)
dict override        (0.03, 1.5, 2.5)
dict missing keys    (0.02, 2.0, 2.0) (falls back to SERVED defaults)
```

63. assistant (2026-06-14T10:54:42.731Z)

Everything works. Now let me make `build\_candidates` in `distill\_local` and `fusion\_golden` able to take a preset, so the eval substrate can be built under either windowing. The current `build\_candidates` already takes a `proposal` tuple param ? I'll make it accept a `WindowingPreset` too, keeping back-compat. Let me check what would be cleanest.

The simplest surgical change: `build\_candidates` already accepts a tuple. The cleanest path is to let `fusion\_golden` / a new served-vs-proposal ablation pass the SERVED preset's tuple. Let me verify `build\_candidates` works with `windowing.SERVED.as\_tuple()` and then run the existing tests.

64. user (2026-06-14T10:54:43.930Z)

All checks passed!

65. user (2026-06-14T10:54:59.269Z)

```
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 3 source files
```

66. assistant (2026-06-14T10:55:06.654Z)

Clean. Now let me wire the eval?serve guard into the ablation runner's promotion path so the rule actually composes with the existing promotion path. Let me look at how the ablation `\_classify` produces verdicts and add a served-aware variant there. First, let me check what the ablation's `litmus\_config` does and think about where to add served-awareness.

The most surgical and meaningful integration: add a ****served-vs-proposal ablation helper**** that builds candidates under both windowings and runs the promotion guard. This is the "worked example proving the fix" (item 3). Let me create that as a focused function in the ablation module, and add the contrast runner.

Let me first verify the golden feature JSONs exist (they're built on proposal) and understand what's available for the worked example.

67. user (2026-06-14T10:55:09.104Z)

```
output/adarsh_avi_singles_golden_features.json
output/gbaaddy_golden_features.json
output/kushagra_singles_golden_features.json
output/largetest_doubles_golden_features.json
output/mahadevpura_singles_golden_features.json
output/testlarge_short_golden_features.json
===manifest===
6 videos
  adarsh_avi_singles | traj: C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/adarsh_avi_full_traj.csv | labels: C:/Users/avidu/Projects/Annotation
Setup/Collect/Golden Labelled/Adarsh and Avi.rallies.csv
  kushagra_singles | traj: C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/kushagra_traj.csv | labels: C:/Users/avidu/Projects/Annotation
Setup/Collect/Golden Labelled/KushagraYashAviFirstVideo.rallies.csv
  largetest_doubles | traj: C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/largetestvideo_traj.csv | labels: C:/Users/avidu/Projects/Annotation
Setup/Collect/Golden Labelled/LargeTestVideo.rallies.csv
  gbaaddy | traj: C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/gbaaddy_traj.csv
| labels: C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/gBaaddy20May2026.rallies.csv
  testlarge_short | traj: C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/testlarge_traj.csv | labels: C:/Users/avidu/Projects/Annotation
Setup/Collect/Golden Labelled/TestLargeVideo.rallies.csv
  mahadevpura_singles | traj: C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/mahadevpura_traj.csv | labels: C:/Users/avidu/Projects/Annotation
Setup/Collect/Golden Labelled/MahadevpuraSingles.rallies.csv
```

68. assistant (2026-06-14T10:55:17.951Z)

The trajectory CSVs exist locally (in the Annotation Setup sibling dir). This means I can build candidates under BOTH windowings from the raw trajectories ? that's the worked example. The golden feature JSONs are proposal-windowed; for the served windowing I need to re-window from the trajectory CSVs.

Now let me add the worked-example runner. I'll create a focused script in the worktree's scratch-equivalent. But scratch/ is gitignored and excluded ? for a reproducible, tracked worked example I should add a small CLI to the ablation module or a standalone tracked eval script. Let me look at how the existing Gate-A ablation builds the served-windowed candidates with net_crossings.

The cleanest approach: add a ``served_vs_proposal_gate_a`` function to ``backend/eval/windowing.py``'s consumers ? actually, it belongs as a tracked, reproducible runner. Let me add it as a function in a new tracked module ``backend/eval/serve_contrast.py`` that builds Gate-A candidates under both presets and runs the promotion guard. Let me first check ``rally_gate.gate_windows`` and ``estimate_play_axis`` signatures.

69. user (2026-06-14T10:55:19.324Z)

```
1: """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
32: crossings: int
55: def estimate_play_axis(points) -> Tuple[str, float, float]:
69: def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
72:     crossings = 0
82:         crossings += 1
```

```

84:     return crossings
91: def gate_windows(points, windows: List[Interval], fps: float,
92:                 min_crossings: int = 2, deadband_frac: float = 0.06,
95:                 keep_decision (crossings >= min_crossings). Deadband is a fraction of the play
106:                 c = count_net_crossings(wp, axis, net, deadband)
108:                 out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109:                                     kept=c >= min_crossings))
114:                 min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
117:                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]

```

70. user (2026-06-14T10:55:22.393Z)

```

91     def gate_windows(points, windows: List[Interval], fps: float,
92                     min_crossings: int = 2, deadband_frac: float = 0.06,
93                     estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
94         """Annotate each (start,end) window with its shuttle net-crossing count and a
95         keep decision (crossings >= min_crossings). Deadband is a fraction of the play
96         axis span. Net/axis are estimated once over the whole trajectory (stable).
97
98         `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
99         `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't
redone
100         per window ? same result, avoids the per-window re-estimation."""
101         axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
102         deadband = deadband_frac * span
103         out: List[GatedWindow] = []
104         for (s, e) in windows:
105             wp = _points_in(points, fps, s, e)
106             c = count_net_crossings(wp, axis, net, deadband)
107             vis = sum(1 for p in wp if p.visible)
108             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109                                 kept=c >= min_crossings))
110         return out
111
112
113     def apply_gate(points, windows: List[Interval], fps: float,
114                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115         """Convenience: return only the windows that pass the crossing gate."""
116         return [(g.start, g.end) for g in
117                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118

```

71. assistant (2026-06-14T10:55:26.013Z)

Now I'll create the tracked worked-example runner: `backend/eval/serve\_contrast.py`. It builds Gate-A candidates from the raw golden trajectories under BOTH the SERVED and PROPOSAL presets, scores Gate-A (control vs min_crossings?2) at each, and runs the promotion guard ? proving the served regression veto kicks in.

72. user (2026-06-14T10:55:26.952Z)

```

1     """Pure temporal interval-matching metrics for rally evaluation.
2
3     No I/O, no DB, no global state ? every function here is deterministic and unit
4     tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5     start < end. We score a list of *predicted* rally intervals against a list of
6     *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7     """
8
9     from dataclasses import dataclass, field
10    from typing import List, Tuple, Optional
11
12    Interval = Tuple[float, float]

```

```

13
14
15     def interval_iou(a: Interval, b: Interval) -> float:
16         """Temporal intersection-over-union of two intervals.
17
18         Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19         overlap / union, both measured as durations on the timeline.
20         """
21         a_start, a_end = a
22         b_start, b_end = b
23         if a_end <= a_start or b_end <= b_start:
24             return 0.0
25         inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26         if inter <= 0.0:
27             return 0.0
28         union = (a_end - a_start) + (b_end - b_start) - inter
29         return inter / union if union > 0 else 0.0
30
31
32     @dataclass
33     class Match:
34         """One matched predicted?ground-truth pair."""
35         pred_index: int
36         gt_index: int
37         iou: float
38
39
40     @dataclass

```

73. assistant (2026-06-14T10:55:29.271Z)

Now let me create the tracked worked-example runner module.

74. assistant (2026-06-14T10:56:04.301Z)

"""The eval?serve worked example: score a cue at BOTH windowings and run the promotion guard.

This is the reproducible proof of the foundational fix. For the Gate-A held-shuttle guard it builds candidate windows directly from each golden trajectory CSV under BOTH named windowings (:data:`backend.eval.windowing.PROPOSAL` and :data:`~backend.eval.windowing.SERVED`), scores CONTROL vs Gate-A (`net\_crossings >= 2`) at each, and feeds the per-video held-out ?F1 into :func:`backend.eval.promotion.promote\_if\_served\_safe`.

It demonstrates the bug *and* the fix in one run:

- PROPOSAL windowing: Gate-A wins (historically **?F1 ? +0.074**, CI clears 0, 6/0 sign test).
- SERVED windowing: Gate-A does **not** win (historically **?F1 ? ?0.008**, real-rally loss).
- The promotion guard therefore **vetoes** a promotion that would otherwise pass on the proposal numbers ? a cue cannot be promoted on a windowing production doesn't serve.

Unlike the persisted ``\*\_golden\_features.json`` (which are PROPOSAL-windowed only), this re-windows from the raw trajectory CSVs so the SERVED candidate set is genuinely different. Pure CPU, GPU-free, no Gemini, no DB. Reuses only shipped pieces (windowing, rally_gate, metrics, promotion). Owner runs it against the real golden CSVs; tests exercise it on synthetic trajectories.

CLI::

```

python -m backend.eval.serve_contrast --manifest output/human_lovo_manifest.json
"""
from __future__ import annotations

import argparse
import json
from typing import Any, Dict, List, Sequence, Tuple

```

```

from backend.eval import windowing as W
from backend.eval.calibrate_wasb import load_golden_gts
from backend.eval.gemini_refine import merge_overlaps
from backend.eval.metrics import score
from backend.eval.promotion import promote_if_served_safe
from backend.eval.training import label_candidates
from backend.pipeline.detectors import rally_gate
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

Interval = Tuple[float, float]
IOU = 0.5
MIN_CROSSINGS = 2

def gate_a_scores_at(points: List[Any], gts: List[Interval], fps: float, frame_width: float,
                    preset: W.WindowingPreset, iou: float = IOU,
                    min_crossings: int = MIN_CROSSINGS, merge_gap: float = 1.0
                    ) -> Tuple[float, float]:
    """One video's (CONTROL F1, Gate-A F1) at ``preset``'s windowing.

    Builds candidates under ``preset``, then scores two static variants vs ``gts``:
    - CONTROL ? every candidate window, overlap-merged (the un-gated baseline).
    - Gate-A ? keep only windows whose shuttle net-crossing count >= ``min_crossings``.

    ``merge_gap`` is the post-keep overlap-merge gap (the harness's ``merge_overlaps`` default),
    independent of the windowing preset's own coalescing gap. Returns the two F1 scores."""
    windows = W.build_windows(points, fps, frame_width, preset)
    intervals = W.windows_to_intervals(windows)
    if not intervals:
        return (0.0, 0.0)
    est = rally_gate.estimate_play_axis(points)
    gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0, estimate=est)
    crossings = [g.crossings for g in gated]

    control = merge_overlaps(intervals, gap_tol=merge_gap)
    kept = [iv for iv, c in zip(intervals, crossings) if c >= min_crossings]
    gate_a = merge_overlaps(kept, gap_tol=merge_gap)
    return (score(control, gts, iou).f1, score(gate_a, gts, iou).f1)

def contrast_video(spec: Dict[str, Any], iou: float = IOU,
                  min_crossings: int = MIN_CROSSINGS) -> Dict[str, Any]:
    """Per-video CONTROL/Gate-A F1 at BOTH the proposal and served windowings."""
    fps, fw = float(spec["fps"]), float(spec["frame_width"])
    points = TrackNetRunner.parse_trajectory_csv(str(spec["trajectory"]))
    gts = load_golden_gts(str(spec["labels"]))
    p_control, p_gate = gate_a_scores_at(points, gts, fps, fw, W.PROPOSAL, iou, min_crossings)
    s_control, s_gate = gate_a_scores_at(points, gts, fps, fw, W.SERVED, iou, min_crossings)
    return {
        "name": spec["name"],
        "proposal": {"control_f1": p_control, "gate_a_f1": p_gate, "delta_f1": p_gate -
p_control},
        "served": {"control_f1": s_control, "gate_a_f1": s_gate, "delta_f1": s_gate -
s_control},
    }

def run_contrast(specs: Sequence[Dict[str, Any]], iou: float = IOU,
                min_crossings: int = MIN_CROSSINGS, min_n: int = 5,
                served_regress_eps: float = 0.0) -> Dict[str, Any]:
    """Full corpus: per-video contrast + the served-guarded promotion decision for Gate-A.

```

Gate-A is a *structural* guard (held-shuttle FP killer), so the promotion gate keeps it ON regardless of measured lift ? but it is only auto-promoted to the SERVED default when it is served-safe. This run shows it ISN'T (it regresses served), so the guard vetoes the

```

eval-only proposal win exactly as the foundation demands."""
per_video = [contrast_video(s, iou, min_crossings) for s in specs]
served_a = [v["served"]["control_f1"] for v in per_video]
served_b = [v["served"]["gate_a_f1"] for v in per_video]
proposal_a = [v["proposal"]["control_f1"] for v in per_video]
proposal_b = [v["proposal"]["gate_a_f1"] for v in per_video]

decision = promote_if_served_safe(
    served_a, served_b,
    proposal_a=proposal_a, proposal_b=proposal_b,
    structural=True, min_n=min_n, served_regress_eps=served_regress_eps,
)
return {
    "iou": iou,
    "min_crossings": min_crossings,
    "num_videos": len(per_video),
    "per_video": per_video,
    "decision": decision.as_dict(),
}

def format_contrast(result: Dict[str, Any]) -> str:
    d = result["decision"]
    prop, serv = d["proposal"], d["served"]
    lines = [
        f"=== Gate-A served-vs-proposal windowing contrast (n={result['num_videos']}, "
        f"IoU>={result['iou']}, min_crossings={result['min_crossings']}) ===",
        f" {'video':<24}{ 'PROPOSAL ?F1':>14}{ 'SERVED ?F1':>14}",
    ]
    for v in result["per_video"]:
        lines.append(f"    {v['name']:<24}{v['proposal']['delta_f1']:>+14.3f}"
                    f"{v['served']['delta_f1']:>+14.3f}")
    lines += [
        " " + "-" * 50,
        f" PROPOSAL: mean ?F1 {prop['mean_delta']:+.4f} CI [{prop['ci_low']:+.4f}, "
        f"{prop['ci_high']:+.4f}] {int(prop['sign_test']['wins'])}W/"
        f"{int(prop['sign_test']['losses'])}L regresses={prop['regresses']}",
        f" SERVED : mean ?F1 {serv['mean_delta']:+.4f} CI [{serv['ci_low']:+.4f}, "
        f"{serv['ci_high']:+.4f}] {int(serv['sign_test']['wins'])}W/"
        f"{int(serv['sign_test']['losses'])}L regresses={serv['regresses']}",
        "",
        f" promote={d['promote']} served_safe={d['served_safe']} "
        f"vetoed_by_served={d['vetoed_by_served']}",
        f" -> {d['reason']}",
    ]
    return "\n".join(lines)

def main() -> None:
    ap = argparse.ArgumentParser(
        description="Gate-A eval?serve worked example: score CONTROL vs Gate-A at the PROPOSAL "
        "and SERVED windowings, then run the served-guarded promotion gate.")
    ap.add_argument("--manifest", default="output/human_lovo_manifest.json",
        help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]")
    ap.add_argument("--iou", type=float, default=IOU)
    ap.add_argument("--min-crossings", type=int, default=MIN_CROSSINGS)
    ap.add_argument("--min-n", type=int, default=5, help="promotion min-N floor")
    ap.add_argument("--served-regress-eps", type=float, default=0.0,
        help="served regression tolerance (0.0 = no served-mean drop allowed)")
    ap.add_argument("--json", default=None, help="also write the structured result JSON here")
    args = ap.parse_args()

    with open(args.manifest, encoding="utf-8") as f:
        specs = json.load(f)
    result = run_contrast(specs, iou=args.iou, min_crossings=args.min_crossings,

```

```

        min_n=args.min_n, served_regress_eps=args.served_regress_eps)
print(format_contrast(result))
if args.json:
    with open(args.json, "w", encoding="utf-8") as f:
        json.dump(result, f, indent=2)
    print(f"\nwrote {args.json}")

if __name__ == "__main__":
    main()

```

75. user (2026-06-14T10:56:04.321Z)

File created successfully at: C:/Users/avidu/Projects/bhi-evalserve/backend/eval/serve_contrast.py (file state is current in your context ? no need to Read it back)

76. assistant (2026-06-14T10:56:07.971Z)

Now let me run the worked example against the real golden trajectories from the main checkout (which has the CSVs and manifest). I'll run it from the main checkout's data but using the worktree code via PYTHONPATH.

77. user (2026-06-14T10:56:09.666Z)

C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe: No module named backend.eval.serve_contrast

78. assistant (2026-06-14T10:56:16.310Z)

PYTHONPATH ordering ? the cwd's `backend` shadows the worktree. Let me run from the worktree but point the manifest/data paths. The manifest uses absolute paths for trajectories/labels, so I can run from the worktree directly if the manifest is copied. Let me copy the manifest into the worktree's output dir.

79. user (2026-06-14T10:56:23.059Z)

=== Gate-A served-vs-proposal windowing contrast (n=6, IoU>=0.5, min_crossings=2) ===

video	PROPOSAL ?F1	SERVED ?F1
adarsh_avi_singles	+0.125	-0.004
kushagra_singles	+0.009	+0.000
largetest_doubles	+0.074	-0.051
gbaaddy	+0.043	+0.003
testlarge_short	+0.075	+0.000
mahadevpura_singles	+0.117	+0.007

```

-----
PROPOSAL: mean ?F1 +0.0739  CI [+0.0419, +0.1043]  6W/0L  regresses=False
SERVED   : mean ?F1 -0.0077  CI [-0.0259, +0.0031]  2W/2L  regresses=True

```

```

promote=False  served_safe=False  vetoed_by_served=False

```

-> structural guard kept ON, but NOT promoted to the served default: it REGRESSES at the served windowing (?F1 -0.0077); an eval-only win must not be served

wrote output/serve_contrast_result.json

80. assistant (2026-06-14T10:56:30.516Z)

This is the exact worked example proving the fix:

- **PROPOSAL windowing**: Gate-A mean ?F1 = **+0.0739** (CI [+0.0419, +0.1043] clears 0, 6W/0L) ? reproduces the +0.074 headline.
- **SERVED windowing**: Gate-A mean ?F1 = **-0.0077** (? -0.008, the served regression that the verify script found), 2W/2L, `regresses=True`.
- **Promotion guard verdict**: `promote=False` ? the structural guard is kept ON but NOT

promoted to the served default because it regresses served. **The eval-only win cannot be served.**

The per-video served deltas precisely match the served verify (`mean\_delta\_f1: -0.008`). The numbers are honest and reproduced exactly. Let me clean up the stray scratch file and the manifest copy (output/ is gitignored so it won't be committed, but let me keep things tidy).

81. user (2026-06-14T10:56:31.958Z)

```
M backend/eval/distill_local.py
?? backend/eval/promotion.py
?? backend/eval/serve_contrast.py
?? backend/eval/windowing.py
===check output is gitignored===
output/serve_contrast_result.json
output/human_lovo_manifest.json
```

82. assistant (2026-06-14T10:56:35.149Z)

Good ? `output/` is gitignored so the manifest copy and result won't be committed. Now let me write comprehensive tests. Let me look at an existing test file to match conventions.

83. user (2026-06-14T10:56:35.692Z)

```
1      """Distilled local rally classifier driver (pure parts; no WSL/GPU/Gemini)."""
2      import csv
3
4      from backend.eval import distill_local as dl
5      from backend.eval.classifier import LogisticRegression
6
7
8      def _write_traj(path):
9          rows = [("Frame", "Visibility", "X", "Y")]
10         for i in range(0, 30):
11             rows.append((i, 1, i * 5, 50))
12         for i in range(30, 50):
13             rows.append((i, 0, -1, -1))
14         for i in range(50, 80):
15             rows.append((i, 1, (i - 50) * 5, 50))
16         with open(path, "w", newline="") as f:
17             csv.writer(f).writerows(rows)
18         return str(path)
19
20
21     def test_build_candidates_feature_shape(tmp_path):
22         intervals, X = dl.build_candidates(_write_traj(tmp_path / "t.csv"), fps=10.0,
frame_width=100.0)
23         assert len(intervals) == len(X) >= 1
24         assert all(len(row) == len(dl.FEATURE_NAMES) for row in X)    # 5 invariant features
25         assert all(d > 0 for d, *_ in X)                               # duration positive
26
27
28     def test_predict_rallies_respects_threshold(tmp_path):
29         intervals, X = dl.build_candidates(_write_traj(tmp_path / "t.csv"), fps=10.0,
frame_width=100.0)
30         y = [1] + [0] * (len(intervals) - 1)
31         model = LogisticRegression(epochs=100).fit(X, y, dl.FEATURE_NAMES)
32         none = dl.predict_rallies(model, X, intervals, threshold=1.01) # impossible cutoff
-> nothing
33         assert none == []
34         some = dl.predict_rallies(model, X, intervals, threshold=0.0)   # everything passes
-> merged windows
35         assert len(some) >= 1
36
```

84. assistant (2026-06-14T10:56:38.110Z)

Now let me look at the experiment test for how `VideoCandidates` / `Variant` are set up in tests, and the per_user_gate test if it exists.

85. user (2026-06-14T10:56:39.639Z)

```
test_eval_stats.py
test_per_user_eval.py
test_per_user_gate.py
test_per_user_regression.py
test_rally_gate.py
===head test_experiment_honest===
"""Tests for the honest-reporting wiring in experiment.compare (C1 + C4, additive).

Pins: compare() ADDS a `honest` block by default (CIs + paired ?F1 sign test + F1@k +
segment-count ratio) WITHOUT changing the existing fields; `honest_stats=False` restores
the legacy shape; and record_experiment carries the honest ?F1.
"""

from backend.eval.experiment import CONTROL, Variant, VideoCandidates, compare,
record_experiment

GATE = Variant(name="gateA", min_crossings=2)

def _vid(name, net_crossings):
    # 3 candidates; the MIDDLE is a false fire (net_crossings 0) absent from the golden set,
    # so Gate-A (>=2) drops it: precision/F1 up, over-segmentation 1.5 -> 1.0.
    return VideoCandidates(
        name=name,
        intervals=[(0.0, 10.0), (20.0, 30.0), (40.0, 50.0)],
        features={"net_crossings": net_crossings},
        gts=[(0.0, 10.0), (40.0, 50.0)],
    )

def _corpus():
    return [_vid("v1", [5.0, 0.0, 4.0]), _vid("v2", [3.0, 0.0, 6.0])]

def test_compare_adds_honest_block_without_touching_legacy_fields():
    res = compare(_corpus(), CONTROL, GATE)
    # legacy fields intact (no regression)
    for k in ("iou", "label_set_hash", "num_videos", "variant_a", "variant_b", "delta",
              "delta_pct", "per_video_delta"):
        assert k in res
    h = res["honest"]
    assert {"variant_a_ci", "variant_b_ci", "paired_delta_f1", "segmentation"} <= set(h)

    pd = h["paired_delta_f1"]
    assert pd["mean_delta"] > 0.0 # Gate-A removes the FP ? F1 up
    assert pd["ci_excludes_zero"] is True
    assert pd["sign_test"]["wins"] == 2.0 and pd["sign_test"]["losses"] == 0.0
```

86. assistant (2026-06-14T10:56:42.965Z)

Let me also peek at the per_user_gate test to match the style.

87. user (2026-06-14T10:56:43.816Z)

```
1     """Tests for backend.eval.per_user_gate ? the per-user personalization promotion gate.
2
3     GPU-free, fully synthetic per-video scores, deterministic (the bootstrap seed is
threaded
```

```

4     through ``should_promote_for_user``). Pins the four safety properties of the counter-
case
5     mitigations:
6
7     - promotes a non-structural cue on a clear, above-floor win;
8     - refuses below the minimum-N floor *even with* a positive point estimate (small-N
lies);
9     - refuses when the ?F1 CI spans 0 / the sign test disagrees (the realistic n=6 near-
null);
10    - **never** disables a structural guard, even given a measured per-user LOSS;
11    - falls back to the shipped baseline (``keep_on=False``) whenever it refuses to promote.
12    """
13    from backend.eval.per_user_gate import PromotionDecision, should_promote_for_user
14
15
16    def test_promotes_clear_win_above_floor():
17        """n=8, B beats A on every video by a healthy margin ? promote ON."""
18        a = [0.50, 0.55, 0.60, 0.52, 0.58, 0.54, 0.56, 0.53]
19        b = [0.62, 0.67, 0.71, 0.63, 0.70, 0.66, 0.69, 0.65]
20        d = should_promote_for_user(a, b, min_n=5, seed=0)
21        assert isinstance(d, PromotionDecision)
22        assert d.promote is True
23        assert d.keep_on is True          # non-structural: keep_on tracks promote
24        assert d.floor_met is True
25        assert d.ci_excludes_zero is True
26        assert d.mean_delta > 0.0
27        assert d.sign_test["wins"] == 8.0 and d.sign_test["losses"] == 0.0
28        assert d.structural_protected is False
29
30
31    def test_refuses_below_floor_even_with_positive_point_estimate():
32        """n=3 with a strongly positive point estimate is still REFUSED ? small-N lies.
33
34        The floor is checked before lift, so even though this CI would 'exclude 0' the gate
must
35        fall back to the baseline rather than personalize off too few videos.
36        """
37        a = [0.50, 0.55, 0.60]
38        b = [0.65, 0.70, 0.72]          # B wins all 3, big positive mean delta
39        d = should_promote_for_user(a, b, min_n=5, seed=0)
40        assert d.mean_delta > 0.0      # point estimate genuinely looks good ...
41        assert d.promote is False      # ... but we refuse on too-few videos
42        assert d.keep_on is False      # ? fall back to shipped baseline
43        assert d.floor_met is False
44        assert d.n == 3
45        assert "floor" in d.reason.lower()
46
47
48    def test_refuses_n6_near_null_ci_spans_zero_and_sign_test_disagrees():
49        """The realistic n=6 regime: tiny mixed deltas ? CI spans 0 and sign test is 3-3."""
50        a = [0.50, 0.55, 0.60, 0.52, 0.58, 0.54]

```

88. assistant (2026-06-14T10:56:47.348Z)

Now I have a clear test style. Let me write the three test files: `test\_windowing.py`, `test\_promotion.py`, and `test\_serve\_contrast.py`.

89. assistant (2026-06-14T10:57:07.341Z)

"""Tests for backend.eval.windowing ? the single-sourced SERVED/PROPOSAL presets.

GPU-free. Pins the foundational invariants:

- the SERVED preset is the SAME windowing production serves (sourced from the config defaults, never a hand-typed tuple that can drift);
- PROPOSAL and SERVED are genuinely DIFFERENT operating points ? different candidate sets;

```

- distill_local's module-level presets stay byte-identical to the legacy tuples;
- served_windowing_from_config reproduces process_video's knob reads (overrides honoured).
"""
import csv

from backend.config.models import AppConfig, IndexingConfig
from backend.eval import distill_local as dl
from backend.eval import windowing as W
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

def _write_traj(path):
    """A trajectory with a SHORT fast burst then a LONG slow drift. The short/slow content
    survives the permissive PROPOSAL windowing but is dropped by the coarse SERVED one, so the
    two presets produce a different number of candidate windows (the whole point)."""
    rows = [("Frame", "Visibility", "X", "Y")]
    # 0-12: short fast burst (large jumps) ? a brief flurry.
    for i in range(0, 12):
        rows.append((i, 1, i * 40, 50))
    # 12-20: occluded (breaks the trajectory).
    for i in range(12, 20):
        rows.append((i, 0, -1, -1))
    # 20-80: long slow drift (tiny jumps) ? low velocity, long duration.
    for i in range(20, 80):
        rows.append((i, 1, 500 + (i - 20) * 2, 50))
    with open(path, "w", newline="") as f:
        csv.writer(f).writerows(rows)
    return str(path)

def test_served_preset_matches_config_defaults():
    """SERVED is single-sourced from IndexingConfig defaults ? exactly what process_video
    reads."""
    idx = IndexingConfig()
    assert W.SERVED.velocity_thresh == idx.wasb.velocity_threshold
    assert W.SERVED.merge_gap == idx.dead_time_merge_gap
    assert W.SERVED.min_window_duration == idx.min_action_window_duration
    # The historically-served operating point.
    assert W.SERVED.as_tuple() == (0.02, 2.0, 2.0)

def test_proposal_preset_is_the_recall_oriented_windowing():
    assert W.PROPOSAL.as_tuple() == (0.005, 1.0, 0.5)
    assert W.PRESETS == {"served": W.SERVED, "proposal": W.PROPOSAL}

def test_distill_local_presets_byte_identical_to_legacy_tuples():
    """The eval substrate / golden JSONs must stay valid ? presets unchanged in value."""
    assert dl.PROPOSAL == (0.005, 1.0, 0.5)
    assert dl.BASELINE_LOCAL == (0.02, 2.0, 2.0)
    assert dl.PROPOSAL == W.PROPOSAL.as_tuple()
    assert dl.BASELINE_LOCAL == W.SERVED.as_tuple()

def test_served_and_proposal_produce_different_candidates(tmp_path):
    """The core eval?serve fact: the two windowings window the SAME trajectory differently."""
    points = TrackNetRunner.parse_trajectory_csv(_write_traj(tmp_path / "t.csv"))
    proposal_w = W.build_windows(points, fps=10.0, frame_width=100.0, preset=W.PROPOSAL)
    served_w = W.build_windows(points, fps=10.0, frame_width=100.0, preset=W.SERVED)
    p_iv = W.windows_to_intervals(proposal_w)
    s_iv = W.windows_to_intervals(served_w)
    # Permissive proposal finds at least as many windows as the coarse served one ...
    assert len(p_iv) >= len(s_iv)
    # ... and the two candidate SETS genuinely differ (so a cue scored on one ? the other).
    assert p_iv != s_iv

```

```

def test_served_windowing_from_typed_config():
    cfg = AppConfig()
    for fam in ("wasb", "fusion", "tracknet"):
        assert W.served_windowing_from_config(cfg, fam).as_tuple() == (0.02, 2.0, 2.0)

def test_served_windowing_from_config_honours_overrides():
    """A deployment that tuned a served default ? served_windowing_from_config follows it."""
    d = {"indexing": {"wasb": {"velocity_threshold": 0.03,
                              "dead_time_merge_gap": 1.5,
                              "min_action_window_duration": 2.5}}
    assert W.served_windowing_from_config(d, "wasb").as_tuple() == (0.03, 1.5, 2.5)

def test_served_windowing_from_config_falls_back_to_defaults_on_missing_keys():
    assert W.served_windowing_from_config({}, "wasb").as_tuple() == W.SERVED.as_tuple()
    assert W.served_windowing_from_config(None, "wasb").as_tuple() == W.SERVED.as_tuple()

def test_preset_roundtrip_dict():
    d = W.SERVED.to_dict()
    assert d == {"name": "served", "velocity_thresh": 0.02, "merge_gap": 2.0,
                 "min_window_duration": 2.0}

```

90. user (2026-06-14T10:57:07.355Z)

File created successfully at: C:/Users/avidu/Projects/bhi-evalserve/tests/test_windowing.py
(file state is current in your context ? no need to Read it back)

91. assistant (2026-06-14T10:57:36.182Z)

"""Tests for backend.eval.promotion ? the eval?serve promotion guard.

GPU-free, synthetic per-video scores, deterministic (bootstrap seed threaded). Pins the foundational RULE: a cue is promotable only if it does NOT regress at the SERVED windowing ? an eval-only (proposal-windowing) win is vetoed.

The Gate-A scenario is the worked example baked into a unit: proposal ?F1 strongly positive (would promote), served ?F1 negative (must veto).

"""

```

from backend.eval.promotion import (
    ServedGateResult,
    promote_if_served_safe,
    served_regresses,
)

```

```

def test_served_regresses_detects_a_served_drop():
    """B drops A on the served scores ? regresses=True."""
    a = [0.50, 0.55, 0.60, 0.52, 0.58, 0.54]
    b = [0.49, 0.53, 0.57, 0.51, 0.56, 0.53] # B uniformly a touch worse
    r = served_regresses(a, b, seed=0)
    assert r["regresses"] is True
    assert r["mean_delta"] < 0.0

```

```

def test_served_regresses_passes_a_clear_served_win():
    a = [0.50, 0.55, 0.60, 0.52, 0.58, 0.54]
    b = [0.62, 0.67, 0.71, 0.63, 0.70, 0.66]
    r = served_regresses(a, b, seed=0)
    assert r["regresses"] is False
    assert r["mean_delta"] > 0.0

```

```

def test_eval_only_win_is_vetoed_by_served_guard():

```

```

"""The Gate-A failure mode: clears the honest bar on PROPOSAL, regresses on SERVED ? vetoed.

served_a/served_b regress (B slightly worse), so even though proposal_a/proposal_b is a
clean 6/0 win, the cue must NOT promote ? a win on a windowing production doesn't serve
cannot carry a promotion.
"""
served_a = [0.50, 0.55, 0.60, 0.52, 0.58, 0.54]
served_b = [0.49, 0.54, 0.58, 0.51, 0.57, 0.53] # served: mild regression
proposal_a = list(served_a)
proposal_b = [0.62, 0.67, 0.71, 0.63, 0.70, 0.66] # proposal: big clean win

d = promote_if_served_safe(served_a, served_b,
                          proposal_a=proposal_a, proposal_b=proposal_b,
                          min_n=5, seed=0)
assert isinstance(d, ServedGateResult)
assert d.served_safe is False
assert d.promote is False
# The proposal view is reported for contrast (the eval?serve numbers travel together).
assert d.proposal is not None
assert d.proposal["mean_delta"] > 0.0 # proposal: a real (eval-only) win ...
assert d.served["mean_delta"] < 0.0 # ... that does NOT hold up served.

def test_non_structural_clear_served_win_promotes():
    """When the SERVED scores themselves clear the bar, the cue promotes (guard doesn't
    block)."""
    served_a = [0.50, 0.55, 0.60, 0.52, 0.58, 0.54]
    served_b = [0.62, 0.67, 0.71, 0.63, 0.70, 0.66]
    d = promote_if_served_safe(served_a, served_b, min_n=5, seed=0)
    assert d.served_safe is True
    assert d.promote is True
    assert d.vetoed_by_served is False
    assert d.base_decision.promote is True

def test_vetoed_flag_only_when_base_would_have_promoted():
    """vetoed_by_served distinguishes 'guard overruled a would-be promotion' from 'never
    qualified'.
```

Here the served scores are a near-null wash (CI spans 0) so the base gate wouldn't promote anyway ? promote=False but NOT a veto (nothing to veto).

```

"""
served_a = [0.50, 0.55, 0.60, 0.52, 0.58, 0.54]
served_b = [0.51, 0.54, 0.61, 0.51, 0.59, 0.53] # tiny mixed deltas ? wash
d = promote_if_served_safe(served_a, served_b, min_n=5, seed=0)
assert d.promote is False
assert d.vetoed_by_served is False # base never qualified; not a veto

def test_structural_guard_kept_on_but_not_served_promoted_when_it_regresses():
    """A structural guard (Gate-A) is NEVER disabled on 'no lift' ? keep_on stays True ? but it
    is also NOT auto-promoted to the served default when it regresses served."""
    served_a = [0.50, 0.55, 0.60, 0.52, 0.58, 0.54]
    served_b = [0.49, 0.54, 0.58, 0.51, 0.57, 0.53] # served regression
    d = promote_if_served_safe(served_a, served_b, structural=True, min_n=5, seed=0)
    assert d.base_decision.keep_on is True # structural: always kept ON
    assert d.base_decision.structural_protected is True
    assert d.served_safe is False
    assert d.promote is False # not promoted to served default
    assert "REGRESS" in d.reason.upper()

def test_structural_guard_served_safe_is_eligible_for_served_default():
    """A structural guard that ALSO holds the line served ? eligible for the served default."""
    served_a = [0.50, 0.55, 0.60, 0.52, 0.58, 0.54]

```

```

served_b = [0.55, 0.60, 0.65, 0.57, 0.63, 0.59] # served win
d = promote_if_served_safe(served_a, served_b, structural=True, min_n=5, seed=0)
assert d.base_decision.keep_on is True
assert d.served_safe is True
assert d.promote is True

def test_served_regress_eps_lets_noise_level_wobble_through():
    """A tiny eps tolerates sub-noise served wobble (so a wash isn't called a regression)."""
    served_a = [0.500, 0.550, 0.600, 0.520, 0.580, 0.540]
    served_b = [0.499, 0.549, 0.599, 0.519, 0.579, 0.539] # -0.001 each: trivially worse
    strict = served_regresses(served_a, served_b, eps=0.0, seed=0)
    lenient = served_regresses(served_a, served_b, eps=0.005, seed=0)
    assert strict["regresses"] is True # zero tolerance: any drop is a regression
    assert lenient["regresses"] is False # half-an-F1-point tolerance: wobble allowed

def test_as_dict_carries_both_windowings():
    d = promote_if_served_safe([0.5, 0.6, 0.55, 0.52, 0.58], [0.5, 0.6, 0.55, 0.52, 0.58],
                              proposal_a=[0.5] * 5, proposal_b=[0.6] * 5, min_n=5, seed=0)
    out = d.as_dict()
    assert {"promote", "served_safe", "vetoed_by_served", "served", "proposal",
            "base_decision", "reason"} <= set(out)
    assert out["proposal"] is not None

```

92. user (2026-06-14T10:57:36.198Z)

File created successfully at: C:/Users/avidu/Projects/bhi-evalserve/tests/test_promotion.py
(file state is current in your context ? no need to Read it back)

93. assistant (2026-06-14T10:57:41.290Z)

Now the serve_contrast test. It needs synthetic trajectories + golden labels. Let me check
`load\_golden\_gts` format and `parse\_trajectory\_csv` to craft a synthetic spec.

94. user (2026-06-14T10:57:42.763Z)

```

33: def load_golden_gts(csv_path: str) -> List[Interval]:
===
def load_golden_gts(csv_path: str) -> List[Interval]:
    """Rally [start,end] seconds from a golden CSV ? delegates to the canonical loader.

    Accepts BOTH golden conventions now (start_time/end_time AND the collector export's
    start,end ? the historical fork is closed; see gt_loader.py / ADR-008). Keeps this
    driver's legacy lenient semantics (skip bad rows), but skipped rows are now logged.
    """
    from backend.eval.gt_loader import load_gt_intervals

```

95. user (2026-06-14T10:57:47.047Z)

```

4:- ``annotations.load_annotations``: positional ``start,end`` (the collector's
6:- ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
10:error. This module accepts BOTH conventions (header-mapped, else positional), so every
15:- header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
17:- no header: columns 0,1 positionally.
30:_START_NAMES = ("start", "start_time")
34: def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
35:     """If `row` is a header naming start/end columns, return their indices, else None."""
58:     saw_header = False
64:         if cols is None and not saw_header:

```

96. assistant (2026-06-14T10:57:55.739Z)

Good ? golden CSV accepts `start,end` header. Now let me write the serve_contrast test with

synthetic trajectories + labels, verifying the served-vs-proposal contrast and the veto.

97. assistant (2026-06-14T10:58:12.995Z)

```
"""Tests for backend.eval.serve_contrast ? the Gate-A eval?serve worked example.
```

```
GPU-free, synthetic trajectories + golden labels written to tmp files. Pins:
```

- a cue is scored at BOTH the proposal and served windowings (genuinely different candidates);
- run_contrast feeds those into the served-guarded promotion gate;
- a structural guard that regresses served is kept ON but NOT promoted to the served default.

```
"""
```

```
import csv
```

```
from backend.eval import serve_contrast as sc
```

```
from backend.eval import windowing as W
```

```
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
```

```
def _write_traj(path, segments):
```

```
    """Write a trajectory CSV. `segments` = list of (start_frame, end_frame, step, visible).
```

```
    A larger `step` = faster shuttle (higher velocity)."""
```

```
    rows = [("Frame", "Visibility", "X", "Y")]
```

```
    for (s, e, step, vis) in segments:
```

```
        x = 50
```

```
        for i in range(s, e):
```

```
            if vis:
```

```
                x += step
```

```
                rows.append((i, 1, x, 50))
```

```
            else:
```

```
                rows.append((i, 0, -1, -1))
```

```
    with open(path, "w", newline="") as f:
```

```
        csv.writer(f).writerows(rows)
```

```
    return str(path)
```

```
def _write_labels(path, intervals):
```

```
    rows = [("start", "end")] + [(s, e) for (s, e) in intervals]
```

```
    with open(path, "w", newline="") as f:
```

```
        csv.writer(f).writerows(rows)
```

```
    return str(path)
```

```
def _spec(tmp_path, name, segments, labels, fps=10.0, fw=100.0):
```

```
    return {
```

```
        "name": name,
```

```
        "trajectory": _write_traj(tmp_path / f"{name}.csv", segments),
```

```
        "labels": _write_labels(tmp_path / f"{name}.rallies.csv", labels),
```

```
        "fps": fps, "frame_width": fw,
```

```
    }
```

```
def test_gate_a_scores_at_returns_two_f1s(tmp_path):
```

```
    # A long fast rally (0-60 frames = 0-6s, big steps) labelled as a rally.
```

```
    spec = _spec(tmp_path, "v", [(0, 60, 40, True)], [(0.0, 6.0)])
```

```
    points = TrackNetRunner.parse_trajectory_csv(spec["trajectory"])
```

```
    from backend.eval.calibrate_wasb import load_golden_gts
```

```
    gts = load_golden_gts(spec["labels"])
```

```
    c_f1, g_f1 = sc.gate_a_scores_at(points, gts, 10.0, 100.0, W.SERVED)
```

```
    assert 0.0 <= c_f1 <= 1.0 and 0.0 <= g_f1 <= 1.0
```

```
def test_contrast_video_scores_both_windowings(tmp_path):
```

```
    spec = _spec(tmp_path, "v", [(0, 60, 40, True)], [(0.0, 6.0)])
```

```
    r = sc.contrast_video(spec)
```

```
    assert set(r["proposal"]) == {"control_f1", "gate_a_f1", "delta_f1"}
```

```

assert set(r["served"]) == {"control_f1", "gate_a_f1", "delta_f1"}
# delta_f1 is internally consistent.
assert abs(r["served"]["delta_f1"]
           - (r["served"]["gate_a_f1"] - r["served"]["control_f1"])) < 1e-9

def test_run_contrast_structural_guard_kept_on(tmp_path):
    """End-to-end: a tiny corpus ? the structural Gate-A guard is kept ON; the decision carries
    both windowings' evidence and the served-safety verdict."""
    specs = [
        _spec(tmp_path, f"v{i}", [(0, 80, 30, True)], [(0.0, 8.0)])
        for i in range(6)
    ]
    res = sc.run_contrast(specs, min_n=5)
    d = res["decision"]
    assert res["num_videos"] == 6
    assert len(res["per_video"]) == 6
    # Structural guard: ALWAYS kept ON regardless of measured lift.
    assert d["base_decision"]["keep_on"] is True
    assert d["base_decision"]["structural_protected"] is True
    # Both windowings' evidence is present (the eval?serve contrast is auditable).
    assert d["served"] is not None and d["proposal"] is not None

def test_format_contrast_is_stringy(tmp_path):
    specs = [_spec(tmp_path, f"v{i}", [(0, 80, 30, True)], [(0.0, 8.0)]) for i in range(5)]
    res = sc.run_contrast(specs, min_n=5)
    out = sc.format_contrast(res)
    assert "PROPOSAL" in out and "SERVED" in out and "promote=" in out

```

98. user (2026-06-14T10:58:13.019Z)

File created successfully at: C:/Users/avidu/Projects/bhi-evalserve/tests/test_serve_contrast.py
(file state is current in your context ? no need to Read it back)

99. assistant (2026-06-14T10:58:16.601Z)

Now let me run the three new test files.

100. user (2026-06-14T10:58:21.025Z)

```

...F..... [100%]
===== FAILURES =====
_____ test_served_and_proposal_produce_different_candidates _____

```

tmp_path = WindowsPath('C:/Users/avidu/AppData/Local/Temp/pytest-of-avidu/pytest-369/test_served_and_proposal_produ0')

```

def test_served_and_proposal_produce_different_candidates(tmp_path):
    """The core eval?serve fact: the two windowings window the SAME trajectory
    differently."""
    points = TrackNetRunner.parse_trajectory_csv(_write_traj(tmp_path / "t.csv"))
    proposal_w = W.build_windows(points, fps=10.0, frame_width=100.0, preset=W.PROPOSAL)
    served_w = W.build_windows(points, fps=10.0, frame_width=100.0, preset=W.SERVED)
    p_iv = W.windows_to_intervals(proposal_w)
    s_iv = W.windows_to_intervals(served_w)
    # Permissive proposal finds at least as many windows as the coarse served one ...
    assert len(p_iv) >= len(s_iv)
    # ... and the two candidate SETS genuinely differ (so a cue scored on one ? the other).
> assert p_iv != s_iv
E     assert [(0.1, 7.9)] != [(0.1, 7.9)]

```

tests\test_windowing.py:70: AssertionError
===== short test summary info =====

FAILED tests/test_windowing.py::test_served_and_proposal_produce_different_candidates
1 failed, 20 passed in 2.42s

101. assistant (2026-06-14T10:58:26.801Z)

My synthetic trajectory in the windowing test happens to collapse to the same single window under both presets (the fast burst dominates). Let me craft a trajectory that genuinely diverges ? a short isolated fast burst (survives proposal min_window=0.5s but is dropped by served min_window=2.0s), plus a separated long rally. Let me check the windowing math: at fps=10, min_window=0.5s needs span ? ... actually windows are defined by active-frame groups. Let me design more carefully with a fast-but-short burst that is < 2s.

102. user (2026-06-14T10:58:34.938Z)

The file C:/Users/avidu/Projects/bhi-evalserve/tests/test_windowing.py has been updated successfully. (file state is current in your context ? no need to Read it back)

103. user (2026-06-14T10:58:42.704Z)

```
proposal [(0.1, 0.7), (3.1, 7.9)]
served   [(3.1, 7.9)]
differ? True
```

104. assistant (2026-06-14T10:58:45.187Z)

Now proposal finds 2 windows, served finds 1 ? genuinely different. Let me run all three new test files again.

105. user (2026-06-14T10:58:49.231Z)

```
..... [100%]
21 passed in 1.93s
```

106. assistant (2026-06-14T10:58:51.456Z)

All 21 pass. Now let me run ruff + mypy on the new files and tests, then the full suite.

107. user (2026-06-14T10:58:55.654Z)

```
=== RUFF ===
35 | from backend.eval.promotion import promote_if_served_safe
36 | from backend.eval.training import label_candidates
   |                                     ^^^^^^^^^^^^^^^^^^^^^
37 | from backend.pipeline.detectors import rally_gate
38 | from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
   |
help: Remove unused import: `backend.eval.training.label_candidates`

Found 1 error.
[*] 1 fixable with the `--fix` option.
=== MYPY ===
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 3 source files
```

108. assistant (2026-06-14T10:58:59.583Z)

Mypy clean. Ruff flagged an unused import in serve_contrast (`label\_candidates` ? I don't actually use it since scoring uses `score` directly). Let me remove it.

109. user (2026-06-14T10:59:01.491Z)

The file C:/Users/avidu/Projects/bhi-evalserve/backend/eval/serve_contrast.py has been updated successfully. (file state is current in your context ? no need to Read it back)

110. user (2026-06-14T10:59:05.527Z)

All checks passed!

111. assistant (2026-06-14T10:59:10.230Z)

Now run the full test suite to confirm no regressions.

112. user (2026-06-14T11:00:05.502Z)

```
=====
Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
obsreport (optional reporting dep): available
=====
.....SS..... [ 11%]
..... [ 22%]
..... [ 34%]
..... [ 45%]
..... [ 56%]
..... [ 68%]
..... [ 79%]
..... [ 91%]
..... [100%]
630 passed, 2 skipped in 52.29s
=====
RESULT: PASS - all tests green
=====
```

113. assistant (2026-06-14T11:00:08.139Z)

All 630 tests pass (2 skipped are optional). Let me also run the full mypy gate as CI would (over the whole backend, since mypy.ini has a clean baseline).

114. user (2026-06-14T11:00:31.280Z)

```
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 94 source files
```

115. assistant (2026-06-14T11:00:34.531Z)

Full backend mypy clean. Let me check how CI is configured to make sure I match the gate exactly (ruff target, test command).

116. user (2026-06-14T11:00:36.260Z)

```
ci.yml
===CI steps===
16: # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17: # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
27:     python-version: "3.13"
28:     - name: Install ruff
29:       run: python -m pip install ruff
31:       run: ruff check . --output-format github
```

```

33: # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
35: # up. Typed core deps are installed so mypy sees their real signatures
44:     python-version: "3.13"
45:     - name: Install mypy + typed core deps
46:     run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-genai
numpy
48:     run: mypy backend training
61:     python-version: "3.13"
63:     run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-genai
65:     run: python -m pytest tests/test_import_smoke.py -v
74:     python-version: "3.13"
83:     run: python -m pip install -r requirements.txt pytest pytest-cov httpx # httpx:
fastapi TestClient
85:     # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
92:     run: python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-
fail-under=70

```

117. assistant (2026-06-14T11:00:43.408Z)

CI runs `ruff check .`, `mypy backend training`, and `python run\_all\_tests.py` with a **coverage floor of 70%** (`--cov-fail-under=70`). Let me verify my new modules are covered (they are well-tested) and run the exact CI commands: ruff over `.`, mypy over `backend training`, and check coverage doesn't drop. Let me run the coverage gate.

118. user (2026-06-14T11:00:47.640Z)

```

=== ruff check . ===
All checks passed!
=== mypy backend training ===
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 97 source files

```

119. user (2026-06-14T11:02:13.616Z)

```

Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q --cov=backend --cov=training --cov-report=term --cov-fail-under=70
backend\eval\promotion.py           52      3    94%
backend\eval\serve_contrast.py      68     18    74%
backend\eval\windowing.py          41      1    98%
TOTAL                             6673   1266    81%
RESULT: PASS - all tests green

```

120. assistant (2026-06-14T11:02:18.866Z)

All CI gates pass: ruff clean, mypy clean, 630 tests pass, total coverage 81% (well above the 70% floor). My new modules are well-covered: windowing 98%, promotion 94%, serve_contrast 74% (the uncovered lines are the `main()` CLI and `format\_contrast` branches, which are exercised in the real worked-example run).

Now let me commit. First, the branch-safety re-verification per the global protocol.

121. user (2026-06-14T11:02:20.276Z)

```

feat/eval-serve-windowing-alignment
e75ec35 feat(eval): LOS0/LOG0 ablation runner ? per-signal marginal ?F1 (Phase 0) (#147)
=== commits ahead of origin/master ===
=== status ===
M backend/eval/distill_local.py
?? backend/eval/promotion.py
?? backend/eval/serve_contrast.py
?? backend/eval/windowing.py

```

```
?? tests/test_promotion.py
?? tests/test_serve_contrast.py
?? tests/test_windowing.py
```

122. assistant (2026-06-14T11:02:22.949Z)

On the correct branch, HEAD is origin/master's tip (no sibling commits), clean to commit. Let me stage explicit paths only and commit.

123. user (2026-06-14T11:02:24.478Z)

```
warning: in the working copy of 'backend/eval/promotion.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'backend/eval/serve_contrast.py', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'backend/eval/windowing.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_promotion.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'tests/test_serve_contrast.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_windowing.py', LF will be replaced by CRLF the next
time Git touches it
M backend/eval/distill_local.py
A backend/eval/promotion.py
A backend/eval/serve_contrast.py
A backend/eval/windowing.py
A tests/test_promotion.py
A tests/test_serve_contrast.py
A tests/test_windowing.py
```

124. user (2026-06-14T11:02:38.903Z)

```
[feat/eval-serve-windowing-alignment b84ec9b] fix(eval): close the eval?serve windowing gap ?
single-source SERVED preset + served-regression promotion guard
7 files changed, 852 insertions(+), 3 deletions(-)
create mode 100644 backend/eval/promotion.py
create mode 100644 backend/eval/serve_contrast.py
create mode 100644 backend/eval/windowing.py
create mode 100644 tests/test_promotion.py
create mode 100644 tests/test_serve_contrast.py
create mode 100644 tests/test_windowing.py
```

125. assistant (2026-06-14T11:02:46.668Z)

Now let me consider the merge-vs-open decision. The task says: MERGE on green IF additive/safe; if it materially changes existing eval semantics, leave OPEN for owner review.

Let me think carefully about this. My changes are **additive** (new modules + a byte-identical re-pointing of distill_local's tuples). However, this fix surfaces a finding with **strategic implications**: it demonstrates that the already-merged PR #146 (which flipped Gate-A to the served default) was promoted on the proposal windowing and **regresses the served pipeline** (?0.008, 12 real-rally losses). That's a product/strategy implication touching a shipped default, not just an additive engineering change.

Per the working agreement (Claude may merge its own good-engineering-fix PRs on green CI; product/strategy/licensing stay owner-gated), the **code** is additive and safe, but the **finding** materially bears on a shipped served default and the Gate-A foundational decision. This is exactly the kind of thing the owner should review. Let me push and open the PR but **leave it OPEN** for owner review, with a crisp summary. Let me push first.

126. user (2026-06-14T11:02:49.797Z)

remote:

```
remote: Create a pull request for 'feat/eval-serve-windowing-alignment' on GitHub by visiting:
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/feat/eval-serve-
windowing-alignment
remote:
branch 'feat/eval-serve-windowing-alignment' set up to track 'origin/feat/eval-serve-windowing-
alignment'.
To https://github.com/avidullu/badminton-highlight-indexer.git
* [new branch]      feat/eval-serve-windowing-alignment -> feat/eval-serve-windowing-alignment
```

127. assistant (2026-06-14T11:02:53.776Z)

Now let me create the PR with a crisp body, leaving it open for owner review.

128. user (2026-06-14T11:03:20.625Z)

<https://github.com/avidullu/badminton-highlight-indexer/pull/154>

129. assistant (2026-06-14T11:03:25.596Z)

PR #154 created. Let me verify CI is running/green on the PR and confirm its open status.

130. user (2026-06-14T11:03:29.049Z)

```
{"mergeable":"MERGEABLE","number":154,"state":"OPEN","title":"fix(eval): close the eval?serve
windowing gap ? single-source SERVED preset + served-regression promotion guard"}
=== CI checks ===
Full test suite (offline, mocked)      pending 0      https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27496789688/job/81272231697
Mypy (lenient baseline) pending 0      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27496789688/job/81272231704
Ruff lint      pending 0      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27496789688/job/81272231726
Syntax sweep + import smoke (no GPU stack)      pending 0
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27496789688/job/81272231729
```

131. assistant (2026-06-14T11:03:32.788Z)

CI is running. Let me wait for it to complete using a background monitor so I can report the final status.

132. user (2026-06-14T11:05:30.503Z)

```
Full test suite (offline, mocked)      pass    1m59s  https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27496789688/job/81272231697
Mypy (lenient baseline) pass    31s    https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27496789688/job/81272231704
Ruff lint      pass    9s    https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27496789688/job/81272231726
Syntax sweep + import smoke (no GPU stack)      pass    17s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27496789688/job/81272231729
DONE
Full test suite (offline, mocked)      pass    1m59s  https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27496789688/job/81272231697
Mypy (lenient baseline) pass    31s    https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27496789688/job/81272231704
Ruff lint      pass    9s    https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27496789688/job/81272231726
Syntax sweep + import smoke (no GPU stack)      pass    17s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27496789688/job/81272231729
```

133. assistant (2026-06-14T11:05:34.611Z)

All 4 CI checks pass on PR #154, which is OPEN and mergeable. The fix is complete. Let me clean up the worktree's copied manifest (gitignored, but tidy) and give the final report.

134. user (2026-06-14T11:05:35.430Z)

--- worktree clean (output/ gitignored) ---

135. assistant (2026-06-14T11:05:54.602Z)

Done. Here is the report.

Foundational fix complete ? eval?serve windowing gap closed

****Root cause confirmed.**** The offline eval substrate (``distill_local.build_candidates`` ? ``fusion_golden`` ? the golden feature JSONs the ablation/compare path re-scores) is built on the recall-oriented ****PROPOSAL**** windowing (``vt=0.005/gap=1.0/min=0.5``), while the served pipeline (``TrajectoryHybridSegmenter.process_video``, reading ``config.indexing`` defaults) windows coarsely (``vt=0.02/gap=2.0/min=2.0``). The two presets lived as scattered, unlinked constants, so nothing prevented promoting a cue on a windowing production never serves.

The fix (3 additive pieces, existing eval semantics unchanged)

1. ****`backend/eval/windowing.py`**** ? named ``WindowingPreset``'s ``SERVED`` vs ``PROPOSAL``. ****`SERVED`** is single-sourced from the same Pydantic config defaults production reads** (``IndexingConfig``), so eval and serve cannot drift. ``distill_local``'s ``PROPOSAL``/``BASELINE_LOCAL`` now derive from these presets, ****byte-identical**** to the legacy tuples (golden JSONs stay valid). ``served_windowing_from_config()`` lifts the actual served windowing from a live config.
2. ****`backend/eval/promotion.py`**** ? encodes the RULE: ``*promotable only if it does not regress at the SERVED windowing.*`` ``promote_if_served_safe()`` composes ``per_user_gate.should_promote_for_user`` + ``eval_stats``; a proposal-only win is reported but ****vetoed****; a structural guard stays kept-ON but isn't auto-promoted to the served default when it regresses served.
3. ****`backend/eval/serve\_contrast.py`**** ? the reproducible worked example.

Gate-A served-vs-proposal numbers (n=6 golden, reproduced exactly)

windowing	mean ?F1	95% CI	sign test	regresses?
--- --- --- --- ---				
PROPOSAL	**+0.0739**	[+0.0419, +0.1043]	6W/0L	No
SERVED	**?0.0077**	[?0.0259, +0.0031]	2W/2L	**Yes**

? ``promote=False``: the structural guard is kept ON but ****not promoted to the served default**** ? an eval-only win must not be served. The served numbers match ``output/gateA_served_verify/`` (?0.008) precisely.

Tests / gates

3 new test files (21 tests): presets are genuinely different candidate sets; the guard vetoes an eval-only win; structural-guard kept-on-but-not-served-promoted on regression. ****Full CI green on the PR****: ruff ?, mypy ?, 630 passed / 2 skipped ?, coverage 81% (windowing 98%, promotion 94%).

PR status: ****#154, left OPEN for owner review**** (<https://github.com/avidullu/badminton-highlight-indexer/pull/154>)

The code is additive/safe, but I did ****not**** auto-merge because the finding has a product/strategy implication: it shows that already-merged ****PR #146**** (flipped Gate-A to the served default ``min_crossings=2``) was promoted on the proposal +0.074 and ****regresses the served pipeline**** (?0.008, 12 real-rally losses across the corpus). That surfaces an owner decision the PR deliberately does not take ? revert the #146 served default to ``min_crossings=0``, or make the served windowing more proposal-like so Gate-A's structural win transfers. Per the working agreement (engineering fixes auto-mergeable; product/strategy owner-gated), this one stays open.

Worktree: ``C:/Users/avidu/Projects/bhi-evalserve`` (branch ``feat/eval-serve-windowing-alignment``), clean.